



GreenLab

Intelligent Laboratory Energy Management System

A Graduation Project Report

Submitted in Partial Fulfilment of the Requirements for the Degree of
Bachelor of Science in Computer Science

Faculty of Engineering
Mansoura University

Prepared by:
Mohamed Hamada

Supervised by:
[Supervisor Name]

Academic Year 2025–2026

Abstract

GREENLAB is an intelligent energy management system designed for computer science faculty laboratories. The system combines embedded sensing, automated appliance control, computer-vision occupancy estimation, backend decision logic, role-based monitoring, and manual override through a web dashboard. The hardware layer is centred on an ESP32 microcontroller connected to environmental sensors and a relay module. The artificial-intelligence layer uses camera frames and a YOLO-based model to estimate the number of occupants. The backend layer receives sensor and occupancy data, applies operational rules, stores audit records, and exposes secure REST endpoints. The frontend layer presents real-time status, laboratory assignments, manual controls, and administrative actions through an Angular single-page application dashboard.

The project addresses the limitations of purely manual control and purely schedule-based automation by providing a multimodal, governed approach that integrates authentication, authorisation, override management, transfer approvals, fault handling, and audit logging within a single four-tier architecture.

Keywords: IoT, energy management, smart building, ESP32, YOLO, occupancy detection, laboratory automation, Angular, ASP.NET Core

Contents

Abstract	iii
I Foundation and Analysis	1
1 Introduction	3
1.1 Background and Motivation	3
1.2 Problem Statement	4
1.3 Project Aim	5
1.4 Project Objectives	5
1.5 Project Timeline	8
1.6 Scope	8
1.7 Project Organisation	9
2 Literature Review	11
2.1 Review Strategy	11
2.2 Related Work	11
2.2.1 Challa et al. (2025) — Multimodal Classroom Occupancy for HVAC	12
2.2.2 Chaudhari et al. (2024) — Occupancy Detection Survey for Smart	13
2.2.3 Zhang et al. (2026) — Modified Lightweight YOLOv8 for Indoor	15
2.2.4 Waghchoure et al. (2025) — IoT Classroom Appliance Control	17
2.2.5 Wibowo et al. (2023) — IoT Energy Management in a University	18
2.2.6 Ahmed et al. (2020) — IoT Classroom Automation with Sensor-	20
2.2.7 Ingole et al. (2024) — ESP32 Classroom Automation	22
2.2.8 SmartHaus — Angular and REST API IoT Dashboard	23
2.3 Comparative Analysis	25
2.4 Literature Gap Addressed by GreenLab	26
3 System Analysis	29
3.1 Introduction to the Chapter	29
3.2 System Context	30
3.2.1 Temporal Dimension	31
3.2.2 Governance Dimension	31
3.2.3 Resilience and Safe Defaults	32

3.3	Business Model	32
3.3.1	Value Proposition and Customer Focus	33
3.3.2	Key Partners, Activities, and Resources	33
3.3.3	Channels	34
3.3.4	Cost Structure and Revenue Streams	34
3.4	Stakeholder Analysis	35
3.5	Use Case Analysis	36
3.5.1	explained, GreenLab’s governance model hinges on knowing exactly which actions are	36
3.5.2	Actors	37
3.5.3	Human-Triggered Use Cases	37
3.5.4	Automatic Use Cases	38
3.5.5	onward) that keeps SensorReadings current and is the precondition for every other	38
3.5.6	Use Case Summary Table	38
3.6	Functional Requirements	39
3.6.1	Area: System Initialisation and Startup	40
3.6.2	Area: Entry and Exit Detection	40
3.6.3	Area: Occupancy Detection and Classification	41
3.6.4	Area: Lighting Control	41
3.6.5	Area: Ventilation Control	42
3.6.6	Area: Monitoring and Dashboard	42
3.6.7	Area: Manual Override	42
3.6.8	Area: Fault Handling and Safety	43
3.6.9	Area: Authentication and Access Control	43
3.7	Non-Functional Requirements	44
3.7.1	Performance	44
3.7.2	Reliability	44
3.7.3	Security	45
3.7.4	Usability	45
3.7.5	Maintainability	45
3.7.6	Scalability	46
3.8	Sequence Diagram Analysis	46
3.8.1	Occupancy-Triggered Automation Workflow	46
3.8.2	Live Camera Streaming Workflow	47
3.8.3	Manual Override (Relay Control) Workflow	48
3.8.4	Laboratory Transfer Request Workflow	50
3.9	Chapter Summary	51

II Design and Implementation 53

4	System Design	55
4.1	Four-Tier Architecture Overview	55

4.1.1	Data Flow Diagrams	56
4.2	Hardware Layer Design	57
4.2.1	Component Topology	57
4.3	V reference)	57
4.3.1	Firmware Architecture	58
4.4	Python AI Service Design	59
4.4.1	Service Architecture	59
4.4.2	Model and Inference Pipeline	59
4.5	low-confidence boxes while retaining seated	60
4.5.1	Camera Source Abstraction	60
4.5.2	Camera Lifecycle Management	60
4.5.3	Backend Integration Contract	60
4.6	Database Design	60
4.6.1	Schema Overview	60
4.6.2	Table Specifications	61
4.7	Automation Engine Design	62
4.7.1	Occupancy Classification	62
4.7.2	Lighting Decision Matrix	62
4.8	off)	63
4.8.1	Fan Activation Table	63
4.9	REST API Design	63
4.9.1	API Structure	63
4.9.2	Complete Endpoint Catalogue	63
4.9.3	Sensor Event Request and Response Contract	65
4.10	Security Design	66
4.10.1	Authentication Architecture	66
4.10.2	Role-Based Access Control	66
4.10.3	Audit Logging Design	66
4.11	Angular Dashboard Design	67
4.11.1	View Architecture	67
4.11.2	View Inventory	67
4.11.3	Real-Time Update Strategy	68
4.12	Chapter Summary	68
4.13	inventoried the twelve Angular dashboard views with their routes, roles, API sources,	69
5	Implementation	71
5.1	Introduction	71
5.2	Hardware Layer Implementation — ESP32 Firmware	72
5.2.1	Library Includes, Hardware Pin Definitions, and Global Constants	72
5.2.2	Sensor Reading Functions	75
5.3	V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore	76
5.3.1	Local HTTP Server — Direct Relay Control Endpoints	76

5.3.2	Device Authentication — JWT Token Lifecycle	78
5.3.3	Motion-Triggered Sensor Reporting — sendMotionRequest() . .	80
5.3.4	Main Firmware Logic — setup() and loop()	81
5.4	Python AI Processing Service Implementation	85
5.4.1	Stage 1: Single-Camera Prototype (cout_code.txt)	85
5.4.2	Camera Source Abstraction Module (camera_sources.py)	90
5.4.3	Stage 2: Production Multi-Camera Tracking API (droidip.py) .	93
5.5	allows a camera that has not been polled for five minutes to be garbage- collected;	94
5.6	Administration and Configuration Interface	101
5.6.1	Add Lab — Laboratory Provisioning Form	101
5.6.2	Devices — Hardware Registry	101
5.6.3	Users — Account List	102
5.6.4	Add User — Account Creation Form	102
5.7	Monitoring Dashboard Implementation	103
5.7.1	Login Screen	103
5.7.2	System Dashboard	104
5.7.3	All Labs — Per-Laboratory View	104
5.8	Implementation Traceability Matrix	105
5.9	Worked End-to-End Example: One Motion Event Across	106
5.9.1	Tier 1 — ESP32 Sensor Reads (loop() iteration N)	106
5.9.2	Tier 1 → Tier 2: HTTP POST to the ASP.NET Core Backend .	106
5.9.3	Tier 2 → Tier 3: AI Service Poll (concurrent with backend evalu-	107
5.9.4	Tier 2 → Tier 1: Backend Response	107
5.9.5	Tier 1 — Relay Actuation	108
5.9.6	Tier 4 — Dashboard Update	108
5.10	Known Limitations and Technical Debt	108
5.11	Chapter Summary	109

III Closure 111

6	Conclusions, Limitations, and Future Work	113
6.1	Introduction to the Chapter	113
6.2	Achievement of Project Objectives	113
6.3	and does not affect the four core capabilities	114
6.4	Summary of Project Contribution	114
6.5	Limitations	115
6.5.1	Implementation Deviations	115
6.5.2	Project-Level Limitations	115
6.6	Future Work	116
6.7	Concluding Remarks	116
6.8	Chapter Summary	117

Appendix A Bill of Materials	119
---	------------

List of Figures

List of Tables

Part I

Foundation and Analysis

Chapter 1

Introduction

Introduction

1.1 Background and Motivation

Computer science faculty laboratories are energy-intensive academic spaces because they commonly contain desktop computers, networking switches and routers, projectors, overhead lighting circuits, cooling fans, and continuously occupied workstations. Unlike ordinary lecture rooms, laboratories are often used according to irregular schedules that include taught sessions, open laboratory periods, examinations, maintenance work, and temporary reassignment between instructors. In many faculties, these spaces remain illuminated and ventilated even when no occupants are present, because the responsible person may have left without switching off appliances, or because the schedule indicates that a session should be active even though the instructor has moved the class elsewhere. The cumulative effect of such waste across dozens of laboratories and hundreds of weekly sessions represents a substantial and avoidable electricity cost for the faculty and the university.

Recent smart-building research identifies occupancy information as a central input for energy-efficient control of lighting and heating, ventilation, and air-conditioning systems. Chaudhari et al. conducted a comprehensive survey of occupancy detection technologies for smart buildings using IoT sensors, demonstrating that occupancy-aware automation can meaningfully reduce energy consumption by ensuring that lighting and HVAC systems operate only when and where occupants are actually present [?]. Song et al. further examined the impact of occupancy behaviour on building energy efficiency, showing that occupant behaviour is a primary variable in academic building energy demand, and that detection and monitoring technologies are essential for closing the gap between designed and actual building performance [?]. Waghchoure et al. demonstrated a practical classroom automation system that uses PIR sensing and camera activation to trigger appliance control, confirming that low-cost IoT components can be deployed in educational settings to reduce

unnecessary electricity use [?].

GreenLab is proposed as an intelligent energy management system for computer science faculty laboratories. The system combines embedded sensing, automated appliance control, computer-vision occupancy estimation, backend decision logic, role-based monitoring, and manual override through a web dashboard. The hardware layer is centred on an ESP32 microcontroller connected to a light-dependent resistor for natural-light sensing, a DHT11 temperature and humidity sensor, an HC-SR04 ultrasonic distance sensor for entry and activity detection, and a 4-channel relay module for controlling lights and fans. The artificial-intelligence layer uses camera frames and a YOLO-based model to estimate the number of occupants in the laboratory. The backend layer receives sensor and occupancy data, applies operational rules, stores audit records, and exposes secure REST endpoints. The frontend layer presents real-time status, laboratory assignments, manual controls, and administrative actions through an Angular single-page application dashboard.

The project is motivated by the limitations of purely manual control and purely schedulebased automation. A schedule may indicate that a laboratory should be active, but it does not confirm whether occupants are present, whether natural light is sufficient, whether ventilation is required, or whether an instructor needs to override the automatic policy for pedagogical reasons. Similarly, passive sensing alone may detect motion without distinguishing sustained occupancy from transient passage, while camera-based detection may be affected by lighting conditions, occlusion, privacy restrictions, or processing availability. The reviewed literature therefore supports multimodal approaches that combine environmental sensors, occupancy estimation, and supervisory control for practical smart-building systems [?, ?]. GreenLab follows this direction by using embedded sensor readings for immediate environmental awareness and computer vision for improved occupancy interpretation, all governed by a backend rule engine and role-based dashboard.

1.2 Problem Statement

The problem addressed by GreenLab is the absence of an integrated, faculty-oriented system that can reduce unnecessary laboratory energy use while preserving instructor control, operational accountability, and safe equipment management. In many laboratories

2 GreenLab: Intelligent Laboratory Energy Management System

at Mansoura University's Faculty of Engineering, lights and fans are controlled manually through wall switches, and responsibility for switching appliances off is distributed informally among instructors, students, and laboratory staff. This arrangement is vulnerable to forgotten switches, delayed response after class changes, and inconsistent handling of temporary laboratory transfers. When a laboratory becomes unoccupied or only partially occupied, appliances may continue operating at the same level because no automated mechanism links occupancy, light level, temperature, schedule, and user authority.

The problem is not limited to electricity consumption. It also concerns laboratory usability and governance. Instructors require visibility of the laboratory state during sessions and need a controlled method to override automatic decisions when teaching conditions require it, such as dimming lights for a projector demonstration or activating ventilation before students arrive on a hot day. Administrators require logs, user-account management, laboratory scheduling, and approval workflows for assignment changes so that room transfers do not bypass access control or leave appliances in an unmanaged state.

Hardware faults, communication failures, sensor anomalies, and relay actuation issues must be detectable and auditable so that the system can enter a safe state and alert the appropriate personnel. Challa et al. demonstrated that combining gas sensing with visual detection improved HVAC response reliability in classrooms, highlighting the importance of multimodal sensing and governed control paths [?]. Chaudhari et al. confirmed that occupancy-aware automation reduces building energy waste, but noted that sensor placement, real-time constraints, and privacy considerations remain practical challenges [?].

The absence of a unified system means that each of these concerns is currently addressed in isolation, if at all. A standalone motion sensor may switch a light but cannot enforce role-based override or record audit trails. A web dashboard may display laboratory status but cannot actuate relays without a backend authority. A schedule may define when a laboratory should be active but cannot confirm that the assigned instructor is actually present. GreenLab is designed to bridge these gaps by providing an integrated cyber-physical system that connects embedded sensing, occupancy detection, rule-based automation, supervised override, and administrative governance in one coherent platform.

3

1.3 Project Aim

The aim of GreenLab is to design an intelligent energy management system that automatically controls lighting and ventilation in computer science faculty laboratories according to occupancy, environmental conditions, schedule context, and authorised user input. The system is intended to support energy-aware operation, reduce avoidable manual effort, preserve safety through controlled relay actuation, and provide administrators with clear monitoring and audit capabilities. This aim is pursued through a four-tier architecture that separates hardware sensing, AI processing, backend decision-making, and dashboard presentation, enabling each tier to evolve independently while maintaining well-defined interface contracts.

1.4 Project Objectives

The project objectives define the specific technical goals that GreenLab must achieve. Each objective is expressed as a full statement of intent with sufficient technical detail

to guide the subsequent requirements, design, and implementation chapters.

- Objective 1: Define a layered four-tier architecture with clear interface contracts. Establish a layered architecture that separates hardware sensing and actuation, AI processing, backend decision-making and persistence, and frontend presentation into four distinct tiers. Each tier communicates through well-defined interfaces so that changes in one tier, such as replacing the YOLO model or redesigning the dashboard, do not require modifications to the other tiers. The hardware tier uses the ESP32 microcontroller to read sensors and control relays, communicating with the backend through REST endpoints. The AI tier runs a Python YOLO service that receives camera frames and returns occupancy counts. The backend tier implements an ASP.NET Core REST API with authentication, business rules, and SQL Server persistence. The frontend tier provides an Angular single-page application for monitoring and control. This separation is essential for maintainability, testability, and independent evolution of each system component.
- Objective 2: Specify functional and non-functional requirements in formal IEEE 830

4 GreenLab: Intelligent Laboratory Energy Management System

style. Specify functional and non-functional requirements in a formal IEEE 830 style across nine functional areas: system initialisation, entry and exit detection, occupancy detection, lighting control, ventilation control, monitoring, manual override, fault handling, and authentication and access control. Each functional requirement must describe one specific, observable, verifiable behaviour of the system, with priority and source annotations. Non-functional requirements must address performance, reliability, security, usability, maintainability, and scalability with measurable targets where possible. This formal specification provides the contractual basis for design decisions and future verification, ensuring that every feature in the system traces to a documented requirement.

- Objective 3: Design occupancy detection using multimodal input. Design occupancy detection using multimodal input that combines ESP32 sensor readings, specifically the HC-SR04 ultrasonic distance sensor for motion and entry detection, the DHT11 temperature sensor for environmental context, and the LDR for light-level awareness, with YOLO-based visual counting from an IP camera stream. Multimodal detection is necessary because no single sensor provides reliable occupancy information under all conditions: ultrasonic sensors may produce false positives from non-human movement, cameras may be affected by lighting and occlusion, and environmental sensors alone cannot confirm human presence. The combination of embedded sensor activity and camera-derived person counts, filtered by confidence thresholds and temporal consistency rules, provides a more robust occupancy estimate than any single modality.
- Objective 4: Design lighting control with a decision matrix. Design lighting control with a decision matrix that uses LDR level, occupancy state, and override state

as inputs. The decision matrix must cover all relevant combinations of natural-light zones (dark, dim, bright), occupancy states (occupied, vacant, unknown), and over-ride conditions (no override, instructor override, administrator override) to produce the appropriate light relay command. This approach replaces ad-hoc threshold logic with a structured, auditable rule table that can be reviewed, configured, and extended without modifying firmware code. The decision matrix is evaluated by the backend rule engine, ensuring that lighting commands are authorised, logged, and consistent

5

with the current system state.

- Objective 5: Design ventilation control with a fan activation table. Design ventilation control with a fan activation table that uses temperature zones and occupancy classification as inputs. The table must define how many fan relay channels are activated and at what speed setting, based on the current temperature zone (cool, comfortable, warm, hot) and the occupancy classification (empty, low, moderate, high). This tabular approach ensures that ventilation decisions are explicit, consistent, and traceable, and that fan behaviour can be tuned per laboratory without rewriting embedded logic. The ventilation control must also respect override commands, fault states, and safety rules that take priority over automatic fan activation.
- Objective 6: Implement a secure ASP.NET Core REST API. Implement a secure ASP.NET Core REST API with authentication, role-based access control, input validation, and audit logging. The API must serve as the single authority for all state-changing actions, including sensor ingestion, occupancy updates, relay commands, manual overrides, transfer requests, user management, and fault reporting. Authentication must use JWT bearer tokens with configurable expiry. Role-based access must distinguish between instructor, administrator, and service roles, with each role permitted to access only its authorised endpoints. Input validation must reject malformed or out-of-range payloads before they reach business logic. Audit logging must record every state-changing request with actor, action, entity, timestamp, and outcome, creating a tamper-resistant operational record for administrative review.
- Objective 7: Implement an Angular dashboard with monitoring, override, scheduling, and administration. Implement an Angular single-page application dashboard that provides real-time monitoring of laboratory status, manual override controls for authorised instructors and administrators, scheduling views for laboratory sessions, transfer request forms for laboratory assignment changes, and administrative management screens for user accounts, logs, and fault review. The dashboard must present occupancy, sensor values, relay states, active overrides, and fault warnings in a clear, role-appropriate interface. Instructor views must be restricted to assigned laboratories, while administrator views must provide access to all laboratories within their authority. Real-time updates must be delivered

through polling or signal-based

6 GreenLab: Intelligent Laboratory Energy Management System
mechanisms so that dashboard users see current system state without manual refresh.

1.5 Project Timeline

The GreenLab project spans nine academic months, from October to June, organised into eight sequential phases. Table 1.1 summarises the phase focus and core deliverable for each month.

7

Table 1.1: GreenLab Monthly Project Timeline (October–June)

Month	Phase	Focus	Core Deliverable
October	Phase 1	Literature Review & Literature review docu-	Requirements Foun-
November	Phase 2	Use Cases & Feature Use case diagram; feature	Definition definition
December	Phase 3	Architecture & Four-tier architecture di-	Database Schema
January	Phase 4	ESP32 Firmware De-	Working firmware with
February	Phase 5	ASP.NET Core Back-	Functional REST API
March	Phase 6	Python YOLO Occu-	Working YOLO service
April	Phase 7	Angular Dashboard & Working	dashboard; end-
May	Phase 8a	Report Writing & Draft Chapters 3–6 of the Consolidation	LaTeX report
June	Phase 8b	Final Compilation & Compiled seven-chapter	Submission PDF and submission package

1.6 Scope

The project scope includes the design of the GreenLab system architecture, requirements specification, operational features, workflow modelling, diagrams, and implementation nar-

8 GreenLab: Intelligent Laboratory Energy Management System

rative. It encompasses automatic lighting control based on occupancy and natural-light conditions, automatic ventilation control based on occupancy and temperature context, continuous monitoring of laboratory state through a web dashboard, manual override for authorised instructors and administrators, authenticated dashboard access with role-based permissions, laboratory assignment change handling with approval workflows, fault detection and reporting, and audit logging of all state-changing actions. The scope also includes a literature review focused on IoT energy management, classroom and laboratory automation, YOLO-based occupancy detection, ESP32-based control systems, and web dashboard architectures for IoT systems.

The scope explicitly excludes a dedicated testing and evaluation chapter. No measured energy savings percentages, experimental accuracy values for YOLO detection on GreenLab laboratory footage, measured latency results, or user-satisfaction study

findings are reported because such measurements have not been supplied as verified project outputs. Where research papers report their own metrics, those metrics are attributed to the original sources and are not presented as GreenLab results [?, ?]. The scope also excludes physical installation of hardware in production laboratories, final deployment of the AI service on production servers, and any regulatory compliance assessment beyond the institutional IT policy of Mansoura University.

1.7 Project Organisation

This project is organised into seven chapters. Chapter 1 introduces the project background, problem statement, aim, objectives, timeline, and scope. Chapter 2 reviews relevant research papers and project references, providing an explicit analysis of each source's relevance and a comparative analysis that identifies the literature gap addressed by GreenLab. Chapter 3 presents the system analysis, including stakeholder analysis, user requirements, use case descriptions, functional requirements in IEEE 830 style across nine areas with a total of 135 requirements, non-functional requirements across six quality dimensions, performance budgets, technical constraints, and a requirements traceability matrix. Chapter 4 describes the twelve system features and their operational workflows, including sequence diagrams for four key workflows and a comparison of operating modes. Chapter 5 presents the detailed system design, covering the four-tier architecture, hardware design with sensor

9

specifications, database design with eleven tables, REST API design with eighteen endpoints, AI processing design, lighting control decision matrix, fan activation levels, frontend design, and security design. Chapter 6 provides a grounded implementation narrative with representative code snippets for the ESP32 firmware, ASP.NET Core backend, Python YOLO service, and Angular dashboard. Chapter 7 concludes the report, summarises the project contributions, acknowledges limitations, and identifies future work directions.

10

Chapter 2

Literature Review

Literature Review

2.1 Review Strategy

The literature review was restricted to sources that directly contribute to GreenLab’s core design concerns: IoT energy management, occupancy-driven smart-building control, YOLObased visual detection, ESP32 or Arduino-class device control, laboratory or classroom automation, and web dashboards for real-time IoT monitoring. Sources were not included merely because they used IoT terminology or addressed building energy in general. Each considered source is preceded by an explicit relevance analysis comment, and only relevant papers and projects are reviewed in detail. The review strategy follows an inclusion and exclusion methodology: a source is included if it addresses at least two of GreenLab’s design concerns and provides verifiable technical content that can be accurately summarised and attributed; a source is excluded if it addresses only tangentially related topics, if its full text is unavailable for verification, or if more recent and directly relevant sources supersede it.

Eight sources met this standard and are reviewed in Sections 2.2.1 through 2.2.8. Each review follows the same analytical structure: a description of the methodology and the hardware or software choices made by the original authors, the core findings reported in the source, an explicit breakdown of advantages and disadvantages relative to GreenLab’s design concerns, and a statement of what GreenLab adopts, adapts, or deliberately departs from.

Section 2.3 consolidates all eight sources into a single comparative matrix, and Section 2.4 synthesises the literature gap that GreenLab is designed to close.

2.2 Related Work

2.2.1 Challa et al. (2025) — Multimodal Classroom Occupancy for HVAC

Methodology and Technical Summary Challa et al. proposed a classroom occupancy detection method that combines real-time video analysis with gas-sensor data for HVAC optimisation [?]. The paper is directly relevant to GreenLab because both systems treat educational spaces as variable-occupancy environments and use visual occupancy information to support energy-related control decisions. The study integrates a webcam for visual capture, an MS1100 gas sensor for environmental monitoring, Arduino-class hardware for data collection, Python and OpenCV for image processing, and a YOLOv4 detection model for person counting. The authors report that the model was trained using more than 20,000 labelled human face images and that multimodal sensing was used to improve reliability under conditions where visual detection alone may be affected by lighting, occlusion, or camera blind spots. The gas sensor provides an independent signal that can corroborate or contradict the camera-derived occupancy estimate, reducing the risk that a single modality failure leads to an incorrect HVAC response [?].

The methodology is important for GreenLab because it demonstrates the value of combining environmental signals with computer vision. GreenLab similarly avoids reliance on one sensing method by using environmental sensors through the ESP32 and camera frames through the AI processing service. The gas-sensor approach used by Challa et al. is analogous to GreenLab’s use of the DHT11 temperature sensor and the HC-SR04 ultrasonic sensor as corroborating inputs alongside the YOLO-derived occupancy count. The key insight from this paper is that HVAC or ventilation decisions should not be based solely on camera detection, because camera accuracy can degrade under variable lighting, occlusion from furniture or equipment, and camera placement constraints that are common in laboratory environments.

Challa et al. also emphasise that occupancy information can be connected to dynamic HVAC control, which supports GreenLab’s decision to link occupancy and temperature context to fan or ventilation states. The results reported in the cited paper, including improved HVAC response times and reduced false-positive occupancy classifications, are not transferred to GreenLab as project results; rather, they justify the design principle that multimodal occupancy estimation can be more robust than isolated motion sensing. GreenLab extends the multimodal principle by adding natural-light sensing for lighting control, a

12 GreenLab: Intelligent Laboratory Energy Management System

backend rule engine for governance, and a role-based dashboard for human supervision, none of which are present in the Challa et al. system.

Advantages

- **Multimodal corroboration.** Combining gas sensing with visual detection reduces the risk that a single sensor failure (poor lighting, occlusion) produces an incorrect HVAC decision.

- Large training dataset. The YOLOv4 model was trained on more than 20,000 labelled images, giving the detector a substantial data foundation relative to many comparable classroom studies.
- Direct HVAC linkage. Occupancy output is connected explicitly to dynamic HVAC control rather than left as a standalone detection result, demonstrating a complete sense-to-actuate pipeline.

Disadvantages / Limitations

- Privacy-sensitive training data. Reliance on face-based image detection (rather than whole-person or silhouette detection) raises privacy considerations that are less acceptable in many educational deployment contexts.
- Gas sensor suitability. The MS1100 gas sensor may not be appropriate or necessary for all educational environments, adding hardware cost and calibration overhead without a clearly transferable benefit to a laboratory setting.
- No governance layer. The system has no web-based dashboard, no role-based access control, and no audit trail for override or administrative action — it is a detection- and-actuation pipeline, not an operationally governed platform.

Relevance to GreenLab

departs from Challa et al.’s specific sensor choices: person detection is used instead of face detection to reduce privacy sensitivity, and temperature and light sensors are used instead of a gas sensor, since they are less privacy-sensitive and more directly tied to GreenLab’s

13

lighting and ventilation decisions. Most significantly, GreenLab adds the complete backend and dashboard governance layer that this paper does not attempt.

2.2.2 Chaudhari et al. (2024) — Occupancy Detection Survey for Smart

Buildings

Methodology and Technical Summary Chaudhari, Xiao, Cheng, and Li presented a broad survey of occupancy detection technologies for smart buildings using IoT sensors [?]. The survey is relevant because GreenLab requires reliable occupancy information for lighting, ventilation, monitoring, and safety-related decisions, and the survey provides a comprehensive taxonomy of the available sensing methods, their strengths, and their limitations. The paper explains that occupancy information can reduce energy consumption by automating lighting and HVAC systems, improve indoor air quality by operating ventilation only where needed, and support security and emergency response by providing real-time occupancy data. The survey covers five principal sensing methods:

passive infrared sensors, carbon dioxide sensors, camera-based detection, Wi-Fi-based device counting, and Bluetooth Low Energy proximity sensing [?].

The survey identifies several practical limitations that are directly relevant to GreenLab’s design. Sensor placement affects detection accuracy because a PIR sensor positioned near an air conditioner vent may produce false positives from moving air, while a camera placed behind a projector screen may have an obstructed field of view. Privacy concerns limit the deployment of camera-based systems in some educational contexts, and real-time constraints require that occupancy decisions be made within the control-loop budget rather than after lengthy batch processing. The survey also highlights interference between sensing modalities, such as Wi-Fi counting that may overestimate occupancy when students carry multiple wireless devices.

The survey’s contribution to GreenLab is conceptual and architectural rather than implementational, since the paper does not describe a deployed system. It supports the decision to treat occupancy detection as a system-level capability rather than as a single sensor reading. If occupancy is a system-level capability, then the system must be designed to fuse evidence from multiple sensors, resolve contradictions, and present a consistent occupancy state to downstream control logic. It also reinforces the need for privacy-aware design,

14 GreenLab: Intelligent Laboratory Energy Management System

maintainability, and fault handling. GreenLab therefore uses a backend decision layer and dashboard audit trail so that sensor readings, vision-based counts, manual overrides, and faults can be interpreted in context rather than in isolation. The survey provides the empirical and theoretical foundation for GreenLab’s multimodal approach, while GreenLab provides the governance layer that the survey identifies as a practical gap in many existing systems.

Advantages

- Comprehensive taxonomy. Covers five distinct sensing modalities with a consistent comparative framework, giving a structured basis for evaluating sensor trade-offs rather than relying on a single technology’s marketing claims.
- Practical limitation analysis. Documents concrete, deployment-relevant failure modes (sensor placement, occlusion, multi-device overcounting) rather than only theoretical accuracy figures.
- System-level framing. Treats occupancy detection as a property of the system as a whole rather than of any one sensor, supporting fusion-based architectures like GreenLab’s.

Disadvantages / Limitations

- No deployed system. As a survey, the paper does not describe or validate an end-to-end automation platform, so its conclusions are necessarily general rather than specific to a laboratory deployment.

- No governance discussion. The survey focuses on detection accuracy and does not address authentication, role-based access, override management, or audit logging.
- No quantitative system comparison. Because the survey aggregates findings from many underlying studies, it does not provide a single, consistent benchmark across all five modalities under identical conditions.

15

Relevance to GreenLab

it is excluded from the system comparison table in Section 2.3 but is drawn on throughout this chapter to justify GreenLab’s multimodal sensing rationale and its emphasis on fusing rather than relying on a single sensing modality.

Figure 2.1: Occupancy-detection taxonomy and sensing modalities discussed by Chaudhari et al.

for smart-building applications.

2.2.3 Zhang et al. (2026) — Modified Lightweight YOLOv8 for Indoor

Occupancy

Methodology and Technical Summary Zhang et al. proposed a modified lightweight YOLOv8 model for fast and precise indoor occupancy detection [?]. The paper is relevant to GreenLab’s AI processing layer because GreenLab depends on person detection or occupancy estimation from laboratory camera frames. The authors identify a common problem in image-based indoor occupancy detection: small human targets and resource constraints can reduce real-time performance. In a typical laboratory or classroom, occupants may

16 GreenLab: Intelligent Laboratory Energy Management System

be seated at desks with only their upper bodies visible, they may be partially occluded by monitors and equipment, and the camera may be mounted at a height and angle that makes person detection more challenging than in open pedestrian scenes. These factors reduce the detection accuracy of standard YOLO models that are trained primarily on outdoor or full-body datasets [?].

The proposed architecture modifies YOLOv8 using feature-map improvements and convolutional adjustments, and the paper reports reduced parameters and computational complexity relative to a baseline YOLOv8 model while improving detection performance on the evaluated indoor dataset. The authors demonstrate that lightweight architectures can achieve competitive detection accuracy with fewer computational resources, which is important for deployment scenarios where a dedicated GPU may not be available or where multiple camera streams must be processed concurrently. The paper also discusses the trade-off between model complexity and inference speed, noting that real-time occupancy detection for building automation requires inference times compatible with the control-loop budget, typically a few seconds per frame.

For GreenLab, this paper supports the selection of a YOLO-family detector and informs future optimisation. The present report does not claim that GreenLab implements the modified architecture exactly. Instead, the paper is used to justify why lightweight detection is a meaningful concern if the AI layer is deployed on constrained local hardware or expected to process multiple laboratory streams. GreenLab’s design therefore separates the AI processing service from the ESP32 controller, allowing the model to be improved without changing the embedded relay-control firmware. This separation is consistent with the layered architecture principle described in the project objectives and with the broader literature on modular IoT systems. The paper also supports a future optimisation direction where GreenLab could adopt a lightweight YOLO variant to reduce inference latency and server resource consumption, particularly as the number of monitored laboratories increases.

Advantages

- Reduced computational cost. Reports lower parameter count and computational complexity relative to baseline YOLOv8 while improving detection performance on indoor data, directly relevant to constrained-hardware deployment.

17

- Indoor-specific optimisation. Explicitly targets the small-target and occlusion problems that arise in seated, desk-based occupancy scenarios — the same scenario class as a computer science laboratory.
- Compatibility with real-time control-loop budgets. Demonstrates inference times suitable for building-automation control loops rather than only offline batch analysis.

Disadvantages / Limitations

- Algorithmic paper, not a deployed system. The paper validates a detection model in isolation; it does not address sensor fusion, relay control, governance, or any end-to-end automation workflow.
- Single-dataset evaluation. Results are reported on the authors’ evaluated indoor dataset, and transferability to a specific laboratory’s camera placement, lighting, and occupant density is not independently verified.
- No discussion of multimodal fusion. The model is evaluated purely on visual input, without considering how its output should be reconciled with environmental sensor signals.

Relevance to GreenLab

complete automation platform. GreenLab does not claim to implement Zhang et al.’s exact modified architecture; rather, the paper justifies why model lightness is a legitimate design concern and supports a stated future-work direction (Chapter 7) in

which a lightweight YOLO variant could be adopted to reduce inference latency as the number of monitored laboratories scales.

18 GreenLab: Intelligent Laboratory Energy Management System

Figure 2.2: Modified lightweight YOLOv8 architecture used for indoor occupancy detection by

Zhang et al.

2.2.4 Waghchoure et al. (2025) — IoT Classroom Appliance Control

with PIR and Camera

Methodology and Technical Summary Waghchoure et al. described a classroom energysaving system using IoT, PIR motion sensing, camera-based detection, a microcontroller, and relay control for lights and fans [?]. The paper is relevant because it addresses the same educational-building problem that GreenLab targets: appliances may remain active in unoccupied rooms, increasing electricity bills and environmental impact. The proposed system detects human presence using a PIR sensor, checks whether devices are on or off, controls appliances based on detection, and continuously monitors entry and exit activity. The camera is activated when the PIR sensor detects motion, providing visual verification

19

that reduces false positives from non-human heat sources such as sunlight or heating vents [?].

The methodology contributes to GreenLab by showing a practical classroom automation pattern: low-cost sensing triggers device control through relays. The PIR-and-camera combination demonstrates that sensor-triggered camera activation can reduce continuous camera processing overhead while still providing visual verification when it is most needed. GreenLab adopts a similar approach in principle: the HC-SR04 ultrasonic sensor and other embedded sensors provide the initial activity signal, and the YOLO service performs detailed occupancy counting when triggered. However, GreenLab extends this pattern significantly by adding a backend rule engine for governance, role-based user access, laboratory assignment workflows, audit logging, and a web dashboard for instructors and administrators. The Waghchoure et al. system operates as a standalone microcontroller application without a backend server, without user accounts, and without operational auditability.

The paper also reveals several practical challenges that GreenLab must address. PIR sensors have limited detection range and may not detect stationary occupants, leading to false vacancy detection. Camera-based verification introduces latency that must be managed within the control-loop budget. Relay control must handle the physical characteristics of the controlled appliances, such as inrush current for fluorescent lighting and minimum on-off cycling intervals for fans. GreenLab addresses these challenges through grace periods before vacancy-based appliance shutdown, confidence thresholds for YOLO occupancy decisions, and configurable command intervals that prevent rapid

relay cycling. These design choices are directly informed by the limitations identified in the Waghchoure et al. work.

Advantages

- Low processing overhead. PIR-triggered camera activation avoids continuous video processing, conserving compute resources until visual verification is actually needed.
- Practical false-positive reduction. Camera verification after a PIR trigger filters out non-human heat sources (sunlight, heating vents) that would otherwise cause unnecessary appliance activation.

20 GreenLab: Intelligent Laboratory Energy Management System

- Demonstrated low-cost feasibility. Confirms that commodity microcontroller and relay hardware is sufficient for classroom-scale appliance automation.

Disadvantages / Limitations

- No backend server or persistence. The system operates entirely on the microcontroller, with no central data store, no historical record of occupancy or appliance state, and no way to review past behaviour.
- No user accounts or access control. Anyone with physical or network access to the system can interact with it; there is no concept of an instructor or administrator role.
- Stationary-occupant blind spot. Like all PIR-based systems, it cannot reliably detect a seated, motionless occupant, risking false vacancy classification during quiet study or exam periods.

Relevance to GreenLab

implemented through the HC-SR04 sensor initiating a YOLO-based check, but replaces the standalone-microcontroller model with a full backend and dashboard. The grace periods, confidence thresholds, and relay-cycling safeguards in GreenLab's design are direct, traceable responses to the limitations this paper documents.

2.2.5 Wibowo et al. (2023) — IoT Energy Management in a University

Laboratory

Methodology and Technical Summary Wibowo et al. presented an IoT-based energy management system designed specifically for university laboratory environments [?]. The paper is relevant to GreenLab because it targets the same environment, university laboratories, and the same goal, reducing energy waste through IoT-based monitoring

and automation. The authors describe a centralised IoT sensor network that monitors laboratory appliance status, environmental conditions, and energy consumption, and uses this data to automate appliance switching and provide monitoring dashboards for laboratory administrators. The system demonstrates that centralised monitoring reduces energy waste in

21

academic labs by ensuring that appliances are switched off when laboratories are not in use and that energy consumption patterns are visible to decision-makers [?].

The key contribution of this paper to GreenLab is the validation of the university laboratory context as a meaningful target for IoT energy management. While many IoT energy systems target commercial offices or residential buildings, university laboratories have distinct characteristics: irregular schedules, variable occupancy, shared equipment, and multiple responsible parties. Wibowo et al. show that a centralised monitoring approach can address these characteristics by providing real-time visibility and automated control.

However, the paper's system does not include computer-vision occupancy detection, role-based access control, manual override with audit trails, or laboratory transfer workflows. These governance features are essential for a real faculty deployment where instructors must retain pedagogical authority, administrators must manage schedules and approvals, and all actions must be auditable.

GreenLab extends the Wibowo et al. approach by adding YOLO-based occupancy detection that provides person counts rather than binary occupied or vacant signals, a rolebased dashboard that separates instructor and administrator functions, override management with time-limited expiry and audit logging, and transfer request workflows that prevent informal room changes from bypassing access control. The centralised monitoring principle is retained, but the governance layer is significantly enhanced to meet the operational requirements of a faculty environment where accountability and safety are as important as energy savings.

Advantages

- Direct context match. Targets the university laboratory environment specifically, rather than a generic office or residential setting, making its findings about irregular schedules and shared equipment directly transferable to GreenLab.
- Centralised visibility. Provides administrators with a single monitoring point for appliance status and energy consumption across the laboratory network, rather than per-room isolated control.
- Demonstrated energy-waste reduction. Confirms that centralised IoT monitoring,

22 GreenLab: Intelligent Laboratory Energy Management System

even without vision-based occupancy detection, can reduce energy waste in an academic laboratory setting.

Disadvantages / Limitations

- Binary occupancy signal only. Without computer-vision integration, the system can only report occupied/vacant states rather than person counts or occupancy classification levels.
- No role-based access control. All users of the monitoring dashboard appear to have equivalent access, without separation between instructor and administrator privileges.
- No override audit trail or transfer workflow. The paper does not describe a mechanism for authorised manual override with expiry, nor a workflow for handling laboratory transfer requests.

Relevance to GreenLab

sufficient, condition for a real faculty deployment — the governance features absent from this paper (role separation, override audit trails, transfer workflows) are precisely the features GreenLab adds on top of the validated centralised-monitoring pattern.

2.2.6 Ahmed et al. (2020) — IoT Classroom Automation with Sensor-

Driven Control

Methodology and Technical Summary Ahmed et al. described an IoT-based classroom automation system that uses motion detection, environmental sensors, a microcontroller, and a web dashboard for classroom appliance switching [?]. The paper is relevant because it demonstrates the feasibility of low-cost IoT classroom automation using off-the-shelf components. The system detects classroom occupancy using motion sensors, reads environmental conditions such as temperature and light level, and uses a microcontroller to switch lights and fans through relays. A web dashboard provides remote monitoring and manual control, allowing instructors to check classroom status and override automatic decisions from a web browser [?].

23

The paper highlights several challenges that are directly relevant to GreenLab's design. Sensor placement affects detection reliability, because a motion sensor mounted in a corner may not detect occupants near the door, and a temperature sensor placed near a window may be affected by solar gain. False positives from non-human motion sources, such as curtains moved by air conditioning, can trigger unnecessary appliance activation. Delayed response after class changes, when the motion sensor has not yet detected vacancy, can leave appliances running for extended periods after occupants have left. Ahmed et al. note that these challenges can be partially mitigated through sensor calibration, debouncing, and grace periods, but that a more robust solution would require additional sensing modalities and a centralised rule engine.

GreenLab addresses these challenges through multimodal sensing that combines ultrasonic, temperature, and light sensors with YOLO-based visual verification, grace

periods that delay vacancy-based shutdown until occupancy absence is confirmed, confidence thresholds that filter transient detections, and a backend rule engine that applies consistent, auditable control logic. The web dashboard in the Ahmed et al. system provides basic remote control but does not implement role-based access, time-limited override with automatic expiry, laboratory transfer approvals, or audit logging. GreenLab provides all of these governance features, ensuring that the convenience of remote control does not compromise operational accountability or safety.

Advantages

- Early validation of web-based remote control. Demonstrates, as early as 2020, that a web dashboard can provide useful remote monitoring and manual override for classroom automation, anticipating the dashboard-centred pattern GreenLab also follows.
- Transparent limitation analysis. The authors explicitly document sensor-placement and false-positive challenges rather than presenting an idealised result, providing concrete, citable design lessons.
- Low-cost, off-the-shelf feasibility. Confirms that classroom automation does not require specialised or expensive hardware to achieve basic occupancy-driven control.

24 GreenLab: Intelligent Laboratory Energy Management System

Disadvantages / Limitations

- No role-based access control. The web dashboard’s manual override is available without distinguishing between instructor and administrator authority.
- No time-limited override or audit logging. Manual overrides, once issued, have no documented expiry mechanism or persistent audit record.
- Single-modality occupancy detection. Relies on motion sensing alone, without a corroborating modality, leaving it exposed to the same false-positive and stationary-occupant problems documented elsewhere in this chapter.

Relevance to GreenLab

periods alone are insufficient mitigations for single-modality sensing; this is part of the justification for combining ultrasonic and YOLO-based detection rather than relying on motion sensing in isolation. The basic remote-override pattern in this paper is the conceptual precursor to GreenLab’s fully governed override mechanism, with time-limited expiry, role validation, and audit logging added on top.

2.2.7 Ingole et al. (2024) — ESP32 Classroom Automation

Methodology and Technical Summary The IRJMETs classroom automation project by Ingole et al. integrates ESP32 technology, LDR sensing, temperature sensing, relay-controlled lighting and fan systems, entry management, two operational modes, and a Wi-Fi manual control interface [?]. This project is highly relevant because its hardware stack closely matches the GreenLab hardware concept. The ESP32 serves as the central controller for sensor acquisition and actuator control, reading LDR values to determine natural-light conditions, reading temperature values to determine ventilation requirements, and activating relays to switch lights and fans. The system supports two operating modes: an automatic mode in which the ESP32 applies sensor-based control rules independently, and a manual mode in which the user can override automatic decisions through a Wi-Fi interface [?].

The two-mode design is a useful reference for GreenLab because GreenLab also needs to support both automatic and manual operation. However, the Ingole et al. implementation

25

treats manual control as a direct Wi-Fi command to the ESP32, without authentication, role-based access, time-limited expiry, or audit logging. Any device on the same Wi-Fi network can send a control command to the ESP32, which is acceptable for a prototype but not for a production faculty system where unauthorised control could create safety hazards or operational disruptions. GreenLab adopts the embedded-control philosophy of this project but places it inside a broader multi-tier architecture. In GreenLab, the ESP32 does not operate as an isolated classroom controller; it communicates with an ASP.NET backend, receives authorised control states, reports sensor data, and participates in audit-backed workflows. Manual override is issued through the authenticated dashboard, validated by the backend, recorded in the audit log, and subject to configurable expiry and cancellation policies.

The hardware similarity extends to the sensor types and relay configuration. Both systems use an LDR for light sensing, a temperature sensor for ventilation decisions, and a relay module for light and fan control. GreenLab adds the HC-SR04 ultrasonic sensor for entry and activity detection, an IP camera for YOLO occupancy verification, and additional relay channels for multi-fan control. These additions are motivated by the limitations of the Ingole et al. approach: without occupancy verification, the system cannot distinguish between a laboratory that is scheduled but empty and one that is actually occupied, and without visual verification, sensor-based occupancy decisions are vulnerable to false positives and false negatives.

Advantages

- Closest hardware match to GreenLab. Uses the same core controller (ESP32) and overlapping sensor types (LDR, temperature), making this the most directly transfer-able hardware reference in the literature reviewed.
- Working dual-mode design. Demonstrates a functional automatic/manual mode

switch, validating the basic premise that an embedded controller can support both autonomous and user-directed operation.

- Simple, low-latency override path. Direct Wi-Fi commands to the ESP32 achieve minimal latency for manual control, since no intermediate server is involved.

Disadvantages / Limitations

26 GreenLab: Intelligent Laboratory Energy Management System

- No authentication on manual control. Any device on the local Wi-Fi network can issue control commands, with no login, no role check, and no protection against unauthorised or accidental interference.
- No occupancy verification beyond environmental sensors. The system cannot distinguish a scheduled-but-empty laboratory from an actually occupied one, since it has no camera or vision-based confirmation.
- No backend, persistence, or audit trail. The ESP32 operates in isolation; there is no central record of historical sensor data, override events, or system state for administrative review.

Relevance to GreenLab

device itself to an authenticated backend, closing the most significant safety and governance gap this project leaves open. The sensor set is also extended with the HC-SR04 and the YOLO-based camera pipeline to address the occupancy-verification limitation identified above.

Figure 2.3: ESP32-based classroom automation hardware prototype reported by Ingole et al.

2.2.8 SmartHaus — Angular and REST API IoT Dashboard

Methodology and Technical Summary SmartHaus is an open-source home automation system for monitoring and controlling IoT devices through real-time dashboards and a RESTful API [?]. Its repository describes an Angular client, a Node.js server, REST

27

communication, microcontroller compatibility with Arduino, ESP8266, and ESP32, and API functions for reading and writing digital and analogue pins. The project is relevant to GreenLab because GreenLab also relies on a dashboard-centred architecture in which users monitor device state and issue authorised control commands. SmartHaus demonstrates that an Angular single-page application can serve as an effective IoT monitoring and control interface, with real-time device status updates and responsive command submission [?].

The main contribution of SmartHaus to GreenLab is architectural rather than domainspecific. It illustrates how a web dashboard can expose device-level operations through REST abstractions while keeping IoT devices on a local network. The REST

API decouples the dashboard from the device communication protocol, allowing the dashboard to be developed and deployed independently of the microcontroller firmware. This decoupling is essential for GreenLab because the Angular dashboard must work with the ASP.NET Core backend rather than directly with the ESP32, and the backend must enforce business rules, authentication, and audit logging that are not present in the SmartHaus architecture.

GreenLab follows a similar separation between dashboard controls and embedded devices but adapts the pattern for faculty laboratories. The SmartHaus system treats all users as equivalent controllers with access to all devices, which is appropriate for a home environment but not for a faculty laboratory where instructors should only control their assigned laboratories, administrators should manage schedules and approvals, and some actions should require explicit authorisation and audit logging. GreenLab adds role-based access that distinguishes instructor, administrator, and service roles; laboratory scheduling and transfer approvals that prevent unauthorised room changes; audit logs that record every state-changing action with actor, timestamp, and outcome; and YOLO occupancy integration that provides automated occupancy-aware control beyond simple on-off switching.

Advantages

- Validated Angular + REST pattern. Demonstrates a working, decoupled architecture in which an Angular client communicates with IoT devices entirely through a REST abstraction layer, directly informing GreenLab’s frontend-to-backend design.
- Broad microcontroller compatibility. Supports Arduino, ESP8266, and ESP32 devices, showing that the REST abstraction generalises across common low-cost mi-

28 GreenLab: Intelligent Laboratory Energy Management System
crocontroller platforms.

- Open-source transparency. As an open repository, the architecture and API design can be inspected directly, giving a verifiable, citable technical reference rather than a description relying solely on a paper’s claims.

Disadvantages / Limitations

- No role differentiation. All dashboard users have equivalent access to all connected devices, which is unsuitable for a multi-instructor, multi-administrator faculty environment with differentiated authority.
- No domain-specific automation logic. SmartHaus exposes raw pin read/write operations rather than higher-level automation concepts such as occupancy classification, lighting decision matrices, or fan activation tables.
- No audit logging or accountability mechanism. State-changing actions are not recorded in a persistent, reviewable audit trail.

Relevance to GreenLab

SmartHaus — keeping the dashboard’s REST contract independent of the embedded device protocol — but replaces SmartHaus’s flat, single-tier user model with role-based access control, laboratory-scoped permissions, transfer approval workflows, and comprehensive audit logging, none of which the SmartHaus architecture provides.

2.3 Comparative Analysis

The reviewed sources collectively show that existing works address individual parts of smart-building automation, including detection, control, or dashboards, but none combines all the governance features that a real faculty laboratory requires. Chaudhari et al. provide a survey of sensing technologies rather than a deployed system, and Zhang et al. contribute an improved YOLO algorithm rather than a complete automation platform; consistent with Sections 2.2.2 and 2.2.3, these two works are discussed in the text but excluded from the comparison table below. The remaining six reviewed works and GreenLab are compared

29

across the technical dimensions that most directly distinguish GreenLab from the existing body of work.

30 GreenLab: Intelligent Laboratory Energy Management System Table 2.1: Comprehensive Feature Comparison Matrix

Dimension	Challa et al. (2025)	Waghchoure et al. (2023)	Wibowo et al. (2020)	Ahmed et al. (2024)	Ingole et al. (2025)	SmartHaus	GreenLab
Target environment	Classroom	Classroom	University laboratory	Classroom	Classroom	Classroom	Classroom
Generic home/IoT	Faculty laboratory	Faculty laboratory	Sensing (multi-)	Camera + gas	PIR	+ camera	✓ IoT sensor net-
✓ LDR + DHT11 + modal)	sensor work	ronmental	ture	tal	sensing	HC-SR04	AI /
Computer vi-	✓ YOLOv4 (face-	Partial (camera	× × × ×	✓ YOLO-based	sion based)	verification, no person	counting count)
Automated light-	×	✓	✓	✓	✓	Partial (manual	pin ✓
Decision-matrix	ing	control	write)	driven	Automated	venti-	✓ HVAC-linked
✓ Fans	Partial	✓ Fans	✓ Fans	Partial (manual	pin ✓	Fan activation ta-	lation /
HVAC con-	write)	ble	trol	Web dashboard	× ×	✓ Monitoring only	✓ Monitoring +
Partial (Wi-Fi	✓ Angular dash-	✓ Angular	SPA	basic	override	command only, no	board
dashboard UI)	Role-based	access	× × × × × ×	✓ Instructor /	Ad-	control (Gover-	ministrator /
Service	nance)	roles	Manual	override	× ×	Basic, no	expiry
Basic, no	expiry	✓ Wi-Fi only,	×	✓ Time-limited,	with expiry	unauthenticated	authenticated,
au-	ditable	Audit logging /	ac-	× × × × × ×	✓ Tamper-resistant	countability	audit
log	Laboratory trans-	× × × × × ×	✓ Request and ap-	fer	workflow	proval	workflow
Backend persis-	Partial (local pro-	×	✓ Centralised	Partial	×	✓ Node.js server	✓
ASP.NET Core +	tence layer	cessing only)	monitoring	SQL	Server		

Sources: Challa et al. [?]; Waghchoure et al. [?]; Wibowo et al. [?]; Ahmed et al. [?]; Ingole et al. [?]; SmartHaus [?].

31

The comparison table confirms that no single reviewed source combines laboratory scheduling, role-based dashboard governance, ESP32 relay control, YOLO occupancy detection, manual override with audit trails, and accountability logging. Challa et al. provide strong computer-vision occupancy detection but no web-based governance. Wibowo et al. target university laboratories but lack YOLO integration and role-based access. Ahmed et al. demonstrate classroom automation but without computer vision or audit logging. Ingole et al. provide the closest hardware match but operate an isolated ESP32 controller without backend authority or governance. SmartHaus validates the dashboard-and-REST architectural pattern but provides no domain-specific automation logic and no role differentiation. GreenLab fills this gap by integrating sensing, AI-based detection, automated control, dashboard governance, and audit accountability within a single layered architecture designed specifically for faculty laboratory deployment.

2.4 Literature Gap Addressed by GreenLab

The literature review reveals a clear gap in the existing body of work: no single reviewed source combines laboratory scheduling, role-based governance, ESP32 relay control, YOLO occupancy detection, manual override with time-limited expiry and automatic policy restoration, and audit logging with tamper-resistant operational records. Each reviewed source addresses one or two aspects of smart-building automation, but the governance layer required for a real faculty deployment is consistently absent. Occupancy detection papers focus on algorithmic accuracy without addressing who may view or act on the detection results. Classroom automation projects focus on sensor-to-relay control without providing role-based access or audit trails. Dashboard projects focus on user interface and REST architecture without integrating occupancy-aware automation or laboratory scheduling.

This gap can be stated precisely along three structural lines, each grounded in the eight reviewed sources above:

- The detection-without-governance gap. Challa et al. [?], Chaudhari et al. [?], and Zhang et al. [?] demonstrate that occupancy can be detected accurately and efficiently through multimodal sensing and lightweight computer vision, but none of these sources extends detection into an accountable control system: there is no

32 GreenLab: Intelligent Laboratory Energy Management System

concept of who is authorised to act on a detection result, and no record of who did.

- The automation-without-accountability gap. Waghchoure et al. [?], Ahmed et al. [?], and Ingole et al. [?] demonstrate that sensor-to-relay automation is achievable with low-cost embedded hardware, but each treats manual override as an unauthenticated, unaudited convenience feature rather than a governed operation, leaving unauthorised or unaccountable control as an open risk in any production deployment.
- The dashboard-without-domain-logic gap. Wibowo et al. [?] and SmartHaus [?] demonstrate that centralised web dashboards can provide visibility and control

over distributed IoT devices, but neither integrates occupancy-aware automation logic, laboratory-specific scheduling, or the role separation that a multi-instructor, multi-administrator faculty environment requires.

GreenLab addresses this three-part gap by designing a system in which energy management is not merely a sensor-to-relay control problem but a governance problem that requires authentication, authorisation, override management, transfer approvals, fault handling, and audit logging. The four-tier architecture ensures that each concern is handled by the appropriate tier: the ESP32 handles local sensor reading and relay actuation, the Python YOLO service handles occupancy inference, the ASP.NET Core backend handles business rules and persistence, and the Angular dashboard handles presentation and user interaction. This integration of automation and governance is the distinguishing contribution of GreenLab relative to the reviewed literature, and it is motivated by the practical requirements of faculty laboratories where automation must coexist with teaching authority, administrative oversight, and safe equipment operation.

33

Chapter 3

System Analysis

GreenLab — Chapter 3: System Analysis

3.1 Introduction to the Chapter

This chapter is the analytical foundation on which the design and implementation of GreenLab, described in the subsequent chapters, are built. All technical values, requirement identifiers, database fields, and API paths quoted in this chapter are taken directly from the project’s TA Reference Document and are treated as authoritative.

Chapter 2 reviewed the literature on occupancy-driven building automation and identified a gap that no single reviewed system closes: the combination of multimodal embedded sensing, YOLObased visual occupancy counting, a rule-based backend automation engine, a role-based governance dashboard, immutable audit logging, and laboratory transfer management. This chapter turns that gap into a concrete, analysable system. It answers three questions in sequence. First, what is GreenLab, and what kind of system is it — Section 3.2 positions GreenLab as a four-tier cyber-physical system and explains the temporal and governance pressures that shape every requirement that follows, and Section 3.3 immediately translates that system context into a commercial case using the Business Model Canvas. Second, who needs GreenLab to do what, and how do they interact with it — Section 3.4 identifies the six stakeholder groups affected by the system, and Section 3.5 translates their needs into a formal use case model. Third, precisely what must the system do and how well must it do it — Sections 3.6 and 3.7 state the 36 functional requirements and the non-functional requirements, organised by area and quality dimension respectively, exactly as fixed in the TA Reference Document.

The chapter closes by validating the requirement set dynamically rather than only statically. A requirements table states what the system must do in isolation; it does not show how those requirements compose into an end-to-end behaviour that crosses all four tiers in a single operational scenario. Section 3.8 supplies that missing dimension by walking through the four sequence diagrams produced for GreenLab — the occupancy-triggered automation workflow, the live camera streaming workflow, the manual override workflow, and the laboratory transfer request workflow — explaining each lifeline, each

conditional branch, and each loop in terms of the functional requirements and database tables it realises. Section 3.9 then summarises the chapter and states explicitly how its outputs feed into the design chapter that follows. 1 GreenLab — Chapter 3: System Analysis

Two conventions are used consistently throughout this chapter and the rest of the project book. First, the four tiers are always referred to by their fixed names — the hardware layer, the Python AI service, the ASP.NET Core backend, and the Angular dashboard — rather than informal synonyms such as “the server” or “the frontend”, in line with the project’s terminology standard. Second, every functional and non-functional requirement is cited by its fixed identifier (for example FRLC-03 or NFR-SEC-01) so that it can be traced unambiguously into the design, implementation, and testing chapters.

2 GreenLab — Chapter 3: System Analysis

3.2 System Context

GreenLab is analysed as a cyber-physical system that bridges the physical environment of a university IT laboratory with software services that automate energy management and provide governance tools for authorised users. The system is organised into four tiers, each owning a distinct responsibility and exposing a well-defined interface to its neighbours, as summarised in Table 3.1.

Table 3.1: GreenLab four-tier architecture and tier responsibilities.

Tier Exact Term Technology Responsibility

Tier 1 the hardware layer ESP32 + Sensors + Relay Sensing, relay control, entry/exit counting

Tier 2 the Python AI service Python + YOLOv11n Visual occupancy detection, person counting

Tier 3 the ASP.NET Core backend ASP.NET Core 8 + SQL Automation engine, API, Server RBAC, audit logging

Tier 4 the Angular dashboard Angular SPA Real-time monitoring, override, admin governance

Figure 3.1: GreenLab four-tier system architecture and data flow.

3 GreenLab — Chapter 3: System Analysis

The hardware layer, built around the ESP32 microcontroller, observes laboratory conditions through three connected sensors — a DHT11 temperature and humidity sensor, an LDR lightdependent resistor, and an HC-SR04 ultrasonic distance sensor — and acts on the physical environment through a four-channel relay module that switches two lighting circuits and two ventilation fans. The Python AI service runs YOLOv8n object detection on frames pulled from an IP camera, providing a person count that complements the hardware tier’s entry/exit counting. The ASP.NET Core backend applies the automation rules captured in the decision matrix and fan activation table, persists every state change and event to SQL Server, and exposes the 21 authenticated REST endpoints listed in Section 3.6 to both the frontend and the hardware devices.

The Angular dashboard presents real-time laboratory monitoring, manual override controls, scheduling, transfer-request forms, and administrative tools to instructors and administrators.

This separation of concerns is intentional and is exploited throughout the project. The hardware layer can adopt a new sensor without the backend’s rule engine changing; the Python AI service could be replaced with a different detection model without the Angular dashboard ever noticing; the ASP.NET Core backend can add new endpoints without any firmware change; and the Angular dashboard can be redesigned without touching server-side code. Feature dependencies in Table 3.2 make this layering explicit: every frontend feature depends on authentication (BE-03); the automation engine (BE-01) depends on live sensor data (HW-01) and on both occupancy signals (AI-01, AI-02); relay-based appliance control (HW-02) is only ever driven by the automation engine or by an override (BE-01, BE-02); and manual override itself (BE-02) depends on authentication and produces audit log entries (BE-03, BE-04).

Table 3.2: Feature dependency chain across the four-tier architecture.

Feature(s) Depends On Reason

FE-01, FE-02, FE-03, FE-04 BE-03 All frontend features require authentication (JWT)

BE-01 HW-01, AI-01, AI-02 Automation engine needs live sensor + occupancy data

HW-02 BE-01, BE-02 Relay control driven by automation engine or override

4 GreenLab — Chapter 3: System Analysis

Feature(s) Depends On Reason

BE-02 BE-03, BE-04 Override requires auth and generates audit log entries

3.2.1 Temporal Dimension

Unlike residential or commercial buildings, where occupancy patterns are comparatively stable, a university IT laboratory experiences highly variable occupancy that changes by semester, by session, and sometimes within a single day: a taught session from 8:00 to 10:00, two empty hours, an open-lab period from 12:00 to 14:00, and an examination from 14:00 to 16:00 are all plausible within one room on one day. Each usage type implies different lighting and ventilation needs — a projector-led taught session benefits from dimmed lighting, while an open-lab period with active workstations needs full lighting — so the system must combine real-time, sensor-driven automation with schedule-aware context rather than a single static rule set.

3.2.2 Governance Dimension

GreenLab is deliberately not a simple sensor-to-relay automation system. University laboratory operations are governed by institutional policy that defines who may control which appliances, how a laboratory transfer request is reviewed, what must be auditable, and how a fault is escalated. These governance requirements are why the functional requirement set in Section 3.6 includes a dedicated authentication and access control area (FR-AC), a manual override area with explicit precedence rules between instructor

and administrator (FR-MO), and an INSERT-only AuditLog table that makes every state-changing action attributable and tamper-evident.

3.2.3 Resilience and Safe Defaults

Because GreenLab is cyber-physical, it must keep behaving sensibly when the network is unreliable, when sensor readings are noisy, or when the Python AI service is temporarily unreachable. The design therefore commits to a safe default at every tier: the ESP32 holds its last known relay state without resetting when it loses contact with the backend (NFR-REL-01); the backend continues serving cached sensor data and honouring active overrides if the AI service is offline; and when any fault is active, the automation engine holds Relay 1 (lighting) ON and leaves Relays 2–4 at their last commanded state rather than guessing (FR-FH-02). Grace periods — the 3second HC-SR04 debounce (FR-EE-02) and the 15/30-second sensor and camera timeout

5 GreenLab — Chapter 3: System Analysis

thresholds (FR-FH-01) — prevent transient sensor noise or brief connectivity gaps from being mistaken for genuine occupancy changes or hardware failure.

Read together, the system context is a set of four directional data flows governed by a single authorisation point at the backend: sensor data flows upward from the hardware layer; occupancy data flows upward from the Python AI service; control commands flow downward from the Angular dashboard, through the backend, to the hardware layer; and audit records flow from every tier into SQL Server. Sections 3.6 to 3.8 formalise this context first as testable requirements and then as concrete sequence diagrams.

6 GreenLab — Chapter 3: System Analysis

3.3 Business Model

Section 3.2 established GreenLab as a technical system: its four-tier architecture and the temporal and governance pressures that shape it. A graduation project intended for genuine deployment beyond the university, however, must also be positioned as a viable proposition rather than a working prototype alone. This section supplies that commercial dimension using the Business Model Canvas (BMC), a strategic framework that maps how an organisation creates, delivers, and captures value across nine interlocking blocks. Figure 3.2 presents the completed canvas for GreenLab; the discussion that follows explains each block in turn and traces it back to the four-tier architecture of Section 3.2 and to the stakeholder, requirement, and sequence-diagram analysis presented later in this chapter, so that the business case is grounded in the same requirement set as the rest of the chapter rather than treated as a separate exercise.

Figure 3.2: GreenLab Business Model Canvas.

3.3.1 Value Proposition and Customer Focus

The value propositions in the canvas restate, in commercial language, exactly what the requirement areas fixed later in this chapter (Section 3.6) express in technical language. Reduced energy consumption, lower operating costs, and the reduction of electricity waste are the direct commercial expression of the Lighting Control and Ventilation Control areas (FR-LC, FR-VC), which switch relays off whenever occupancy is Empty rather than leaving appliances running by

7 GreenLab — Chapter 3: System Analysis

default. Automating lighting and ventilation and monitoring laboratory occupancy in real time map directly onto the automation engine and the Monitoring and Dashboard area (FR-MD): a laboratory operator is not asked to trust an opaque automation engine, but can see its occupancy and relay state at any moment, consistent with the refresh behaviour fixed by FR-MD-01. Taken together, these value propositions are framed in the canvas as supporting a single overarching goal — sustainable campus operations — which positions GreenLab as a campus-wide energy-efficiency initiative rather than only a per-laboratory monitoring tool.

The customer segments named in the canvas are a refinement of the six stakeholder groups identified in Section 3.4, now split explicitly into a primary and a secondary segment. Universities and educational institutions form the primary segment: the buying organisations that would authorise a GreenLab deployment and its budget, corresponding to the Faculty Management row of the stakeholder table. IT departments, laboratory staff, facilities and maintenance teams, and university administrators form the secondary segment: the operational users who interact with the system day to day or who depend on its outputs, corresponding respectively to the IT Department, Laboratory Staff, and Administrator rows of the same table. Separating the two makes explicit that GreenLab is sold to an institution as a single purchasing decision, even though several distinct roles within that institution go on to operate it. The customer relationship block completes this picture on the service side: technical support, installation service, scheduled maintenance, software updates, and training sessions are the ongoing obligations a vendor of GreenLab would owe its customers, and the training-session commitment in particular is what keeps the low-friction override path of NFR-USA-01 credible for an instructor who is a subject-matter expert rather than an IT specialist.

3.3.2 Key Partners, Activities, and Resources

The key partners block identifies the organisations a real GreenLab deployment depends on outside the development team itself: the university administration and its facilities and maintenance department as the institutional sponsor and physical site owner, the IT department as the network and security gatekeeper referenced throughout Section 3.2.2's governance discussion, IoT hardware vendors supplying the ESP32 boards, sensors, and relay modules, and a cloud or server hosting provider for the ASP.NET Core backend and SQL Server database. Rather than assuming a dedicated camera infrastructure

provider, the canvas treats the IP cameras already installed across university laboratories as an existing asset that the Python AI service reuses, which lowers the barrier to a pilot deployment considerably. Each of these partners corresponds to a tier or a cross-

8 GreenLab — Chapter 3: System Analysis

cutting concern already fixed in Table 3.1: IoT hardware vendors supply Tier 1, the existing university camera estate supplies the visual input consumed by Tier 2, and the hosting provider supplies the infrastructure Tier 3 runs on.

The key activities — energy management, occupancy counting, lighting and ventilation automation, and real-time system monitoring — are the commercial names for the automatic use cases detailed later in this chapter (UC-09 through UC-13): they are the activities the system performs continuously regardless of who is watching, and they are what a customer is ultimately paying to have run reliably. The key resources block separates the assets that make those activities possible into three categories that mirror the architecture directly: physical resources (cameras and IoT devices) correspond to Tier 1 and the AI-processing camera input; technology resources (the YOLO model and the SQL Server database) correspond to Tier 2 and the persistence layer described in Section 3.2; and human resources (software developers) are the team responsible for maintaining the backend’s decision matrix, fan activation table, and Angular dashboard as the system evolves beyond the pilot deployment analysed in this chapter.

3.3.3 Channels

The channels block sets out how GreenLab would reach and be adopted by the customer segments identified above: direct sales to faculty management, university partnerships that allow a pilot deployment in one laboratory to expand across a faculty’s full estate (the scenario NFR-SCAL-01 is designed to support), participation in technology competitions that provide visibility among IoT solution providers, and relationships with existing IoT solution providers who could integrate GreenLab’s automation engine alongside their own hardware offerings. These channels are consistent with the project’s current status as a graduation project rather than a commercial product: the university partnership and competition channels are the most immediately actionable routes, while direct sales and third-party integration represent the longer-term path once the fourtier architecture has been validated at the scale Section 3.7.6 targets.

3.3.4 Cost Structure and Revenue Streams

The cost structure groups GreenLab’s expenses into three areas that again align with the architecture: hardware costs (the ESP32 boards, sensors, relays, and cameras that make up Tier 1 and the camera input to Tier 2), software development costs (backend development, dashboard development, and AI maintenance and integration, spanning Tier 2 through Tier 4), and cloud and

9 GreenLab — Chapter 3: System Analysis

infrastructure costs (database hosting for the SQL Server instance described in Section 3.2). Set against these costs, the revenue streams propose five ways GreenLab

could recover its costs and generate a margin: one-time installation fees for initial system setup; annual maintenance contracts that fund ongoing support; a recurring software subscription that covers access to the monitoring dashboard and future updates; paid custom solutions for client-specific requirements; and integration services for connecting GreenLab to a customer's existing cameras and IoT devices rather than requiring a full hardware replacement. The subscription and maintenance-contract streams in particular depend on the reliability targets of Section 3.7.2 — a backend that is unavailable during operating hours, or an ESP32 that resets its relay state on every network interruption, would undermine the recurring-revenue case as directly as it would undermine the technical case for the system.

Table 3.3 closes this section by tracing each canvas block back to the chapter section or requirement area it draws on, preserving the same traceability discipline applied to the use cases and sequence diagrams later in this chapter.

Table 3.3: Business Model Canvas blocks traced to their chapter basis.

BMC Block Chapter Basis

Value Propositions FR-LC, FR-VC, FR-OD, FR-MO, FR-MD, NFR-SCAL

Customer Segments Table 3.4 (Stakeholder Analysis); Faculty Management (primary), IT Department / Laboratory Staff / Administrator (secondary)

Customer Relationships NFR-USA-01, NFR-MAIN-01

Key Partners Table 3.1 (four-tier architecture); Section 3.2.2

Key Activities UC-09–UC-13 (Section 3.5.3)

Key Resources Section 3.2 (tier responsibilities)

Channels Section 3.7.6 (Scalability)

Cost Structure Table 3.1 tier technologies

Revenue Streams Section 3.7.2 (Reliability)

10 GreenLab — Chapter 3: System Analysis

Read alongside the rest of this chapter, the canvas in Figure 3.2 shows that GreenLab's requirement set was not defined in a commercial vacuum: the same automation behaviour, governance controls, and reliability targets that satisfy the functional and non-functional requirements set out later in this chapter are also the features and guarantees a paying customer would evaluate before adopting the system beyond a single pilot laboratory.

11 GreenLab — Chapter 3: System Analysis

3.4 Stakeholder Analysis

Six stakeholder groups have a direct or indirect interest in GreenLab's operation. Identifying them precisely is what keeps the requirements in Sections 3.6 and 3.7 grounded in real operational need rather than hypothetical scenarios.

Table 3.4: Stakeholder analysis summary.

Stakeholder Role Primary Concern System Features

Instructor Conducts sessions, monitors Automatic control must not Real-time dashboard, assigned labs, overrides interfere with teaching manual override with timed

automation when teaching expiry, transfer request form needs it, requests transfers

Administrator Manages users, schedules, Operational oversight, User management, schedule transfers, and configuration; policy compliance, management, transfer reviews audit logs and faults accountability approval, audit log, fault acknowledgement

Laboratory Staff Monitors hardware integrity, Hardware faults detected Fault alert dashboard, sensor responds to fault alerts, and resolved before they status display, relay maintains equipment compromise safety feedback

Student Occupies the laboratory; an A comfortable environment Indirect benefit through indirect beneficiary without needing to interact occupancy-driven lighting with the system and ventilation

IT Department Maintains network, server, Reliable infrastructure, HTTPS, JWT, service and security infrastructure security compliance, accounts, firmware update updateable firmware support, network configuration

Faculty Management Reviews energy trends and Energy cost reduction, Energy reports, audit log policy compliance; environmental review, configuration authorises budget responsibility, auditability oversight

The instructor and administrator are the two roles that interact directly with the Angular dashboard and therefore drive most of the use case model in Section 3.5; laboratory staff, students, the IT department, and faculty management are served indirectly through the features that the instructor and administrator operate on their behalf — fault visibility for laboratory staff, ambient comfort for

12 GreenLab — Chapter 3: System Analysis

students, HTTPS/JWT compliance for the IT department, and audit/reporting access for faculty management.

13 GreenLab — Chapter 3: System Analysis

3.5 Use Case Analysis

The use case diagram for the GreenLab Intelligent Energy Management System (Figure 3.3) captures every interaction identified in the stakeholder analysis above and groups them around four actors: the two human roles, Instructor and Administrator, and two automated actors, the Hardware System (Automatic) and the AI Processing System. Separating human-triggered use cases from automatically-triggered ones makes explicit which behaviours require a logged-in user and which run continuously regardless of who is watching — a distinction that matters because, as Section

3.5.1 explained, GreenLab’s governance model hinges on knowing exactly which actions are

attributable to a person.

Figure 3.3: GreenLab use case diagram showing actors and system interactions.

3.5.2 Actors

- Instructor — a human actor who logs in to monitor their assigned laboratory, issue manual overrides, and submit transfer requests.
- Administrator — a human actor with faculty-wide visibility who manages user accounts, laboratory schedules, transfer approvals, and system logs.
- Hardware System (Automatic) — the ESP32 and its sensors and relay module, acting without a human in the loop to apply occupancy-based lighting and temperature-based ventilation control. 14 GreenLab — Chapter 3: System Analysis
- AI Processing System — the Python YOLO-based occupancy detection service, acting as an automated actor that supplies person-count data into the same automatic control loop.

3.5.3 Human-Triggered Use Cases

Both human actors share “Login to Dashboard” as a common entry point. The diagram models this with «include» relationships: “Manual Override Control Lights and Fans”, “Submit Laboratory Transfer Request”, and “Manage Laboratory Schedule” each include “Login to Dashboard”, meaning none of them can be performed without a valid, authenticated session — the direct use case expression of FR-AC-02 (every endpoint except login requires a valid JWT).

- Login to Dashboard — validates credentials and returns a role-bearing JWT (realised by FR-AC-01).
- View Real-Time Monitoring Data — the instructor or administrator views live LDR, temperature, humidity, occupancy, and relay state for a laboratory (FR-MD-01, FR-MD-03).
- Manual Override Control Lights and Fans — issues a timed override for a specific relay (FR-MO-01, FR-MO-02); this is the use case realised in detail by the sequence diagram in Section 3.8.3.
- Submit Laboratory Transfer Request — an instructor requests that a session move to a different laboratory, creating a row in TransferRequests via POST /transfer/request.
- Manage Laboratory Schedule — an administrator updates a laboratory’s session schedule via PUT /labs/id/schedule.
- Approve/Reject Transfer Requests — an administrator reviews a pending TransferRequests row and resolves it via PUT /transfer/id/approve or PUT /transfer/id/reject; the diagram models the approval decision as an «extend» of the base review action, since rejection is an alternative outcome of the same review rather than a separate trigger.

- Manage User Accounts — an administrator creates or updates Users rows via POST /users and PUT /users/id.

15 GreenLab — Chapter 3: System Analysis

- View System Logs and Reports — an administrator inspects the INSERT-only AuditLog table and active FaultEvents for oversight and incident investigation.

3.5.4 Automatic Use Cases

The lower half of the diagram belongs to the two automated actors and is connected back into “Login to Dashboard” by dashed «include» arrows. This is not a contradiction of the “no human in the loop” description above — it represents the fact that the automatic control loop’s output (current relay state, current occupancy classification) is what the dashboard displays once a user does log in, so the automatic use cases feed the same dashboard rather than triggering it.

- Automatic Occupancy-Based Lighting Control — the decision procedure behind FR-LC- 01 and FR-LC-03, evaluated on every 5-second sensor cycle by the Hardware System actor in conjunction with the backend’s decision matrix.
- Automatic Temperature-Based Ventilation Control — the decision procedure behind FR-VC-01 and FR-VC-02, giving the Hot temperature zone priority over occupancy when deciding fan state.
- Continuous Monitoring — the unending 5-second sensor polling cycle (FR-SI of Section

3.5.5 onward) that keeps SensorReadings current and is the precondition for every other

automatic use case.

- End of Session Reset — the behaviour invoked when occupancy returns to Empty with no active override, returning both light relays and, where the temperature zone permits, both fan relays to OFF (FR-LC-03, FR-VC-03).
- Perform Occupancy Detection (YOLO-based) — owned by the AI Processing System actor; this is FR-OD-01 through FR-OD-04, and it is the use case explained step by step in the sequence diagram of Section 3.8.1.

16 GreenLab — Chapter 3: System Analysis

3.5.6 Use Case Summary Table

Table 3.5 and Table 3.6 together trace every use case identified above to the functional requirements it realises, split across the two tables below for readability.

Table 3.5: Use case descriptions and traceability to functional requirements (UC-01 to UC-06).

Traced ID Use Case Actor(s) Description Requirements

UC-01 Login to Dashboard Instructor, Authenticate and FR-AC-01, FR-AC- Administrator receive a role-scoped 02 JWT.

UC-02 View Real-Time Instructor, Display live sensor, FR-MD-01–FR-MD- Monitoring Data Administrator occupancy, and relay 04 state for one or all labs.

UC-03 Manual Override Instructor, Issue a timed override FR-MO-01–FR-MO- Control Lights and Administrator on a specific relay, 04, FR-LC-02, FR- Fans superseding VC-04 automation.

UC-04 Submit Laboratory Instructor Request a session TransferRequests Transfer Request move to a different table; Endpoint 13 laboratory.

UC-05 Manage Laboratory Administrator Create or update a Endpoint 12 Schedule laboratory’s session schedule.

UC-06 Approve/Reject Administrator Review and resolve a Endpoints 14–16 Transfer Requests pending transfer request.

17 GreenLab — Chapter 3: System Analysis

Table 3.6: Use case descriptions and traceability to functional requirements (UC-07 to UC-13).

Traced ID Use Case Actor(s) Description Requirements

UC-07 Manage User Administrator Create or update user Endpoints 17–19, FR- Accounts accounts and role AC-04 assignment.

UC-08 View System Logs Administrator Inspect the audit trail FR-FH-03, FR-FH- and Reports and active fault 04, AuditLog events.

UC-09 Automatic Hardware System Apply the lighting FR-LC-01, FR-LC- Occupancy-Based (Automatic) decision matrix on 03, FR-LC-04 Lighting Control every sensor cycle.

UC-10 Automatic Hardware System Apply the fan FR-VC-01–FR-VC- Temperature-Based (Automatic) activation table on 04 Ventilation Control every sensor cycle.

UC-11 Continuous Hardware System Poll all three sensors FR-SI-01–FR-SI-04, Monitoring (Automatic) every 5 seconds. FR-EE-01–FR-EE-04

UC-12 End of Session Reset Hardware System Return relays to OFF FR-LC-03, FR-VC-03 (Automatic) once occupancy is Empty and no override is active.

UC-13 Perform Occupancy AI Processing System Run YOLOv8n FR-OD-01–FR-OD- Detection (YOLO- inference and report a 04 based) classified person count.

18 GreenLab — Chapter 3: System Analysis

3.6 Functional Requirements

The functional requirements are organised into the nine areas defined in the TA Reference Document, with four requirements per area, for a total of thirty-six. Requirement identifiers follow the pattern FR-[AREA]-[NUMBER]; priority is marked P1 (must have) or P2 (should have). These nine areas correspond directly to the use cases of

Section 3.5 and to the sequence diagrams analysed in Section 3.8: System Initialisation feeds Continuous Monitoring (UC-11); Entry/Exit Detection and Occupancy Detection together feed the occupancy-triggered automation workflow of Section 3.8.1; Lighting and Ventilation Control are the two automatic use cases UC-09 and UC-10; Monitoring and Dashboard underlies UC-02 and the camera streaming workflow of Section 3.8.2; Manual Override is UC-03 and the workflow of Section 3.8.3; Fault Handling and Safety underlies UC-08; and Authentication and Access Control underlies UC-01 and gates every other use case in the model.

3.6.1 Area: System Initialisation and Startup

These requirements govern the ESP32 firmware’s behaviour during power-up, before the device enters its normal sensing-and-control loop. They establish the conditions under which the hardware tier is considered ready to participate in the wider GreenLab data flow, and they define the fallback behaviour when network connectivity cannot be established immediately.

Table 3.7: Area: System Initialisation and Startup.

FR-ID Requirement Priority

FR-SI-01 The ESP32 shall verify all three sensor P1 interfaces on power-up before entering the normal polling loop; if any sensor fails, log fault and proceed.

FR-SI-02 The ESP32 shall attempt Wi-Fi P1 connection with up to 3 retries on startup; if all fail, enter Degraded Mode with all relays holding last state.

19 GreenLab — Chapter 3: System Analysis

FR-ID Requirement Priority

FR-SI-03 On first successful Wi-Fi connection P1 after startup, the ESP32 shall retrieve the current active relay command from GET /relay/state/labId.

FR-SI-04 The backend shall reject any sensor P1 POST from an ESP32 presenting an invalid or expired Service-role JWT, returning HTTP 401.

3.6.2 Area: Entry and Exit Detection

Entry and exit detection is the hardware tier’s first line of occupancy awareness, implemented through the HC-SR04 ultrasonic sensor. These requirements define how a physical presence event at the laboratory doorway is converted into a debounced, countable signal that feeds the occupancy counter referenced throughout the automation engine.

Table 3.8: Area: Entry and Exit Detection.

FR-ID Requirement Priority

FR-EE-01 The ESP32 shall classify an entry P1 event when an HC-SR04 reading drops below 200 cm and increment the local occupancy counter.

FR-EE-02 The ESP32 shall enforce a minimum P1 3-second debounce interval between consecutive HC-SR04 entry or exit events.

FR-EE-03 The ESP32 shall include the current P2 occupancy counter value in every sensor POST payload sent to /sensor/readings.

20 GreenLab — Chapter 3: System Analysis

FR-ID Requirement Priority

FR-EE-04 If the HC-SR04 returns no echo within P2 30 ms (out of effective range), the ESP32 shall record the reading as null in SensorReadings.

3.6.3 Area: Occupancy Detection and Classification

Where entry/exit counting gives a coarse occupancy signal, the Python AI service provides a richer, vision-based estimate using YOLOv8n person detection. These requirements define the inference cycle, the classification thresholds that translate a raw person count into an operational category, and the reconciliation rule applied when the two occupancy signals disagree.

Table 3.9: Area: Occupancy Detection and Classification.

FR-ID Requirement Priority

FR-OD-01 The Python AI service shall run P1 YOLOv8n inference every 5 seconds per lab, filter detections to class 0 with confidence ≥ 0.6 .

FR-OD-02 The ASP.NET Core backend shall map P1 the received person count to exactly one of four classifications: Empty (0), Low (1–5), Moderate (6–15), High (>15).

FR-OD-03 If the IP camera stream is unreachable, P1 the AI service shall POST a `CAMERA_UNAVAILABLEfaultandceasesendingoccupancyupdates`.

FR-OD-04 If YOLO and HC-SR04 classifications P2 disagree by more than one tier, the backend shall use the higher classification as the active value.

21 GreenLab — Chapter 3: System Analysis

3.6.4 Area: Lighting Control

Lighting control is the first of the two automation outputs driven by the decision matrix in the ASP.NET Core backend. These requirements specify when a lighting relay command is issued, how an active manual override suppresses automatic lighting decisions, and how every lighting state change is captured for traceability.

Table 3.10: Area: Lighting Control.

FR-ID Requirement Priority

FR-LC-01 The automation engine shall evaluate P1 the lighting decision matrix on every sensor cycle and issue a relay command only when state changes.

FR-LC-02 When an active `OverrideCommand` P1 exists for Relay 1 or Relay 2, the automation engine shall not issue any lighting relay commands.

FR-LC-03 When occupancy classification is P1 Empty and no active override exists for light relays, the automation engine shall command both light relays OFF.

FR-LC-04 Every relay state change shall produce P1 one row in `RelayStates` (`CommandSource` = Automation or Override) and one row in `AuditLog`.

3.6.5 Area: Ventilation Control

Ventilation control mirrors lighting control but is driven primarily by the DHT11 temperature reading rather than occupancy alone, reflecting the fact that thermal comfort and equipment cooling needs can apply even in an empty room. These requirements define the fan activation rule, the priority given to the Hot temperature zone, and the same override-suppression and audit principles applied to lighting.

22 GreenLab — Chapter 3: System Analysis

Table 3.11: Area: Ventilation Control.

FR-ID Requirement Priority

FR-VC-01 The automation engine shall evaluate P1 the fan activation table on every sensor cycle and command Relay 3 and Relay 4 accordingly.

FR-VC-02 When temperature zone is Hot (above P1 30°C), the automation engine shall command Relay 3 (Fan 1) to ON regardless of occupancy.

FR-VC-03 When occupancy is Empty and P1 temperature zone is Cool or Comfortable, both Relay 3 and Relay 4 shall be commanded OFF.

FR-VC-04 When an active OverrideCommand P1 exists for Relay 3 or Relay 4, the automation engine shall not issue any ventilation relay commands.

3.6.6 Area: Monitoring and Dashboard

These requirements define what the Angular dashboard must show, how often it refreshes, and how visibility is scoped by role — an instructor sees only their assigned laboratory, while an administrator sees the full estate. They also define how fault conditions surface visually so that they cannot be missed.

Table 3.12: Area: Monitoring and Dashboard.

FR-ID Requirement Priority

FR-MD-01 The Angular dashboard shall refresh P1 each lab state every 10 seconds via GET /labs/id, showing LDR value/zone, temperature, humidity, occupancy, relay states.

23 GreenLab — Chapter 3: System Analysis

FR-ID Requirement Priority

FR-MD-02 An instructor shall only view data for P1 the LabId in their JWT AssignedLabId claim; a mismatched GET /labs/id request returns HTTP 403.

FR-MD-03 An administrator shall view P2 LabStatusCards for all active laboratories from one view, with fault banners visible without scrolling.

FR-MD-04 Active fault events shall appear as a P1 styled banner showing fault type, timestamp, and a Resolve button visible only to administrators.

3.6.7 Area: Manual Override

Manual override is the mechanism through which the governance dimension of GreenLab asserts itself over pure automation: an authorised user may temporarily take direct control of a relay. These requirements define who may override what, how an override

expires automatically, and how administrator overrides take precedence over instructor overrides — directly reflected in the Manual Override sequence diagram analysed in Section 3.8.3.

Table 3.13: Area: Manual Override.

FR-ID Requirement Priority

FR-MO-01 An instructor shall issue a manual P1 override for any relay in their assigned lab only, specifying relay target, desired state, duration, and reason.

FR-MO-02 An administrator shall issue a manual P1 override for any relay in any lab; an administrator override supersedes any existing instructor override.

24 GreenLab — Chapter 3: System Analysis

FR-ID Requirement Priority

FR-MO-03 When an override ExpiresAt is P1 reached, the backend shall set IsActive = 0 and resume automation engine control of the affected relay.

FR-MO-04 Every override creation, automatic P1 expiry, and manual cancellation shall be recorded in AuditLog with ActorId, ActorRole, and Outcome.

3.6.8 Area: Fault Handling and Safety

Because GreenLab directly switches physical appliances, the system must fail safe rather than fail silent. These requirements define the timeout thresholds that distinguish a healthy data flow from a stalled one, the safe state the automation engine holds during a fault, and the administrator workflow for acknowledging and clearing a fault once resolved.

Table 3.14: Area: Fault Handling and Safety.

FR-ID Requirement Priority

FR-FH-01 The backend shall generate P1 `SENSOR_TIMEOUT` if no `SensorReadings` row arrives within

FR-FH-02 When any fault is active, the P1 automation engine shall hold safe state: Relay 1 ON (lights), Relays 2–4 hold last known state.

FR-FH-03 Active faults appear as fault banners P1 on the relevant LabStatusCard; administrators see all labs; instructors see their assigned lab only.

25 GreenLab — Chapter 3: System Analysis

FR-ID Requirement Priority

FR-FH-04 An administrator resolves a fault via P1 PUT `/faults/id/resolve`; the backend sets `IsResolved` = 1, `ResolvedAt` = UTC now, `ResolvedByAdminId`.

3.6.9 Area: Authentication and Access Control

Every other functional area depends on this one: nothing in GreenLab is reachable without a valid identity and a verified role. These requirements define how credentials are validated, how the resulting JSON Web Token is structured and enforced, and the non-negotiable rule that plaintext passwords are never stored or transmitted in clear form — the same JWT life cycle that underlies the authentication step of the Manual Override workflow in Section 3.8.3.

Table 3.15: Area: Authentication and Access Control.

FR-ID Requirement Priority

FR-AC-01 POST /auth/login shall validate P1 username and verify password against bcrypt PasswordHash; on success return a JWT signed with HMAC- SHA256, expiring in 60 minutes.

FR-AC-02 Every endpoint except POST P1 /auth/login requires a valid non- expired JWT bearer token; missing/invalid token returns HTTP 401.

FR-AC-03 The backend shall enforce that an P1 instructor's JWT AssignedLabId claim matches the labId in every request; any mismatch returns HTTP 403.

FR-AC-04 All passwords shall be stored P1 exclusively as bcrypt hashes; plaintext passwords shall never be stored, logged, or returned.

26 GreenLab — Chapter 3: System Analysis

3.7 Non-Functional Requirements

Where Section 3.6 states what the system must do, this section states how well it must do it, across six quality dimensions: performance, reliability, security, usability, maintainability, and scalability. Every design target below is a design target, not a measured result — verification against these targets belongs to the testing chapter, not this one.

3.7.1 Performance

Performance requirements bound how quickly the system must react, since a 10-second dashboard refresh and a 5-second sensor cycle are only meaningful if every tier underneath them keeps pace.

Table 3.16: Non-functional requirements — Performance.

NFR-ID Requirement Design Target Rationale

NFR-PERF-01 Backend responds to all API < 500 ms Dashboard 10 s polling must requests under normal load feel instantaneous. (≤ 20 labs, ≤ 50 concurrent users).

NFR-PERF-02 End-to-end sensor-reading ≤ 2 seconds One 5 s polling cycle must $\rightarrow$ relay-command latency complete within it. under normal load.

NFR-PERF-03 Python AI service completes ≤ 5 seconds Keeps occupancy one full YOLOv8n classification fresh every inference cycle per lab. polling cycle.

3.7.2 Reliability

Reliability requirements address what happens when connectivity is imperfect — the normal condition for IoT deployments inside a university building rather than the exception.

27 GreenLab — Chapter 3: System Analysis

Table 3.17: Non-functional requirements — Reliability.

NFR-ID Requirement Design Target Rationale

NFR-REL-01 ESP32 in Degraded Mode No time limit Prevents relay flapping on maintains last known relay intermittent network. state without reset until connectivity returns.

NFR-REL-02 Backend uptime during lab ≥ 99 operating hours (08:00– on continuous availability. 22:00, Mon–Sat).

NFR-REL-03 On ESP32 reconnection, Within 1 poll cycle Prevents stale relay states device retrieves current after reconnection. relay command before resuming automation.

3.7.3 Security

Security requirements protect the confidentiality of credentials and sensor data in transit and guarantee that the audit trail cannot be altered after the fact.

Table 3.18: Non-functional requirements — Security.

NFR-ID Requirement Design Target Rationale

NFR-SEC-01 All tier communication uses TLS 1.2+ Protects JWT tokens and HTTPS; backend redirects sensor data in transit. any HTTP request to HTTPS.

NFR-SEC-02 Role enforcement Middleware-level Single enforcement point; implemented at middleware no gap from missing checks. level, not inside individual controller methods.

NFR-SEC-03 AuditLog has no UPDATE INSERT-only Records must be tamper- or DELETE permissions for evident and legally any role; only INSERT by defensible. audit middleware.

28 GreenLab — Chapter 3: System Analysis

3.7.4 Usability

Usability requirements recognise that instructors are domain experts in teaching, not in building automation software, so the override path in particular must be self-evident.

Table 3.19: Non-functional requirements — Usability.

NFR-ID Requirement Design Target Rationale

NFR-USA-01 A first-time instructor can ≤ 3 clicks Instructors are not IT locate and issue a manual specialists. override without reading documentation.

NFR-USA-02 Fault banners visible on 1920×1080, no scroll Faults must be immediately LabStatusCard without visible. scrolling, up to 6 cards on screen.

3.7.5 Maintainability

Maintainability requirements keep tuning and evolution out of the source code, consistent with the independently-deployable four-tier philosophy described in Section 3.2.

Table 3.20: Non-functional requirements — Maintainability.

NFR-ID Requirement Design Target Rationale

NFR-MAIN-01 All control thresholds (LDR 0 hardcoded thresholds Enables tuning without code zones, YOLO confidence, changes or redeployment. polling intervals, JWT expiry, fan levels) stored in SystemConfig.

NFR-MAIN-02 Each of the four tiers is Zero cross-tier coupling Supports iterative independently deployable; improvement of individual updating one tier requires no tiers. changes to any other tier, provided the API contract is stable.

29 GreenLab — Chapter 3: System Analysis

3.7.6 Scalability

Scalability requirements ensure that GreenLab can grow from a pilot deployment to the full set of IT laboratories without a redesign.

Table 3.21: Non-functional requirements — Scalability.

NFR-ID Requirement Design Target Rationale

NFR-SCAL-01 System supports ≥ 20 labs Faculty has >10 IT labs; simultaneously active future growth must be laboratories, each posting supported. sensor data every 5 s, without architecture or schema changes.

NFR-SCAL-02 Adding a new laboratory 0 schema changes per new Reduces IT Department requires only: one lab effort for lab additions. Laboratories row, one user account, one ESP32 configured, one camera configured.

30 GreenLab — Chapter 3: System Analysis

3.8 Sequence Diagram Analysis

A requirement table establishes correctness criteria for one behaviour in isolation. A sequence diagram shows how several requirements, several actors, and all four tiers cooperate to deliver one complete, time-ordered scenario. The four sequence diagrams produced for GreenLab and analysed below were chosen because together they exercise every tier of the architecture at least once: the occupancy workflow exercises the hardware layer, the Python AI service, and the backend; the camera streaming workflow exercises the backend’s media-handling path and the Angular dashboard; the manual override workflow exercises authentication, the backend’s automation engine, the hardware layer, and the audit subsystem in a single pass; and the transfer request workflow exercises the same authorisation layer applied to an organisational rather than a hardware-level decision.

3.8.1 Occupancy-Triggered Automation Workflow

This diagram has six lifelines — Motion Sensor, ESP32, Controller, API, Model YOLO, and Camera — and models the complete path from a physical presence event to an appliance being switched, including the visual confirmation step performed by YOLO. It is the dynamic counterpart of the Entry/Exit Detection (FR-EE), Occupancy Detection and Classification (FROD), Lighting Control (FR-LC), and Ventilation Control (FR-VC) requirement areas, and it realises use cases UC-09, UC-10, UC-12, and UC-13.

31 GreenLab — Chapter 3: System Analysis

Figure 3.4: Sequence diagram — occupancy-triggered automation workflow.

The scenario begins when the Motion Sensor detects activity (step 1) and signals the ESP32, which sends a trigger to the Controller (step 2) — the practical instantiation of the HC-SR04 entry classification in FR-EE-01, subject to the 3-second debounce of FR-EE-02. The Controller issues an HTTP request to analyse the scene (step 3), which the API forwards to the Model YOLO component as a request for frame analysis (step 4). This request-response pair is wrapped in an opt [Camera Active] guard: if the camera is reachable, YOLO captures a frame from the Camera (step

- and receives it back (step 6); if the camera is not active, this branch is skipped entirely, which is the sequence-diagram expression of the `CAMERA_UNAVAILABLE` fault path definition — the diagram shows the happy path, while the fault path is specified formally by that requirement.

32 GreenLab — Chapter 3: System Analysis

Once a frame is available, YOLO processes the image and counts students (step 7, a self-call representing the YOLOv8n inference and the confidence-0.6 / class-0 filter of FR-OD-01), then returns the count to the API (step 8), which relays it to the Controller as the response students count (step 9). The Controller's next action is wrapped in an alt block with two mutually exclusive branches that correspond directly to FR-LC-03 and FR-VC-03 on one side and FR-LC-01/FR-VC01 on the other: when [students greater than 0], the Controller tells the ESP32 to turn ON devices (step 10), and the ESP32 activates the lights and AC/fan (step 11); when [students equal 0 for a period], the Controller instead instructs the ESP32 to turn OFF devices (step 12), and the ESP32 deactivates the relevant components (step 13) — this is the End of Session Reset use case (UC-12) executing in practice. The diagram closes with a loop [Continuous Monitoring] (step 14), which is the visual representation of the fact that this entire sequence repeats indefinitely on the 5-second polling cycle defined across the System Initialisation and Occupancy Detection requirement areas, rather than running once.

Two design decisions in this diagram are worth calling out explicitly because they recur in the requirements. First, occupancy is confirmed by a vision-based count rather than acted on directly from the motion trigger alone — the motion event only initiates the check; the relay decision waits for YOLO's answer. This is what allows FR-OD-04 to exist at all: a disagreement between the HCSR04-based entry/exit signal and the YOLO-based count is only meaningful because both signals are computed and compared, not because one pre-empts the other. Second, the alt branch's OFF path is not triggered the instant the count reaches zero but only when it is zero "for a period", which is the sequence-diagram phrasing of the safe-default philosophy described in Section 3.2.3: a single empty reading should not instantly switch off lighting that a class is merely stepping out of for a moment.

3.8.2 Live Camera Streaming Workflow

This diagram has four lifelines — Admin, System, CameraService, and Camera — and models the path an administrator follows to view a laboratory's live video feed from the Angular dashboard, distinct from the YOLO inference path of Section 3.8.1 even though

both ultimately read from the same IP camera. It realises the visual-monitoring half of UC-02 and is underpinned, on the implementation side, by the camera-source resolution logic described in the `camera_sources.py` and `androidip.py` modules referenced later in the design chapter.

33 GreenLab — Chapter 3: System Analysis

Figure 3.5: Sequence diagram — live camera streaming workflow.

The Admin clicks “View Camera for Lab” (step 1), and the System requests a camera stream for that LabId from the CameraService (step 2). This request is followed by an opt [Connection Needed] guard: a connection to the physical Camera is established (step 3) and confirmed (step 4) only if one is not already open, which mirrors the connection-pooling behaviour implemented by the `create_capture/ensure_camera_started` logic in the `streaming_service` — `—already-registered camera is reused rather than reconnected on every request. Once connected, the camera continuously sends video data to the CameraService (step 5), which forwards the stream to the S` diagram view of the long-running per-camera worker thread that keeps reading frames for as long as the

The outcome is resolved by an alt block with two branches. On [Success], the System displays live video to the Admin (step 7) — the direct realisation of FR-MD-01’s requirement that the dashboard show live state. On [Failure], the System instead shows an error message (step 8). This 34 GreenLab — Chapter 3: System Analysis

failure branch is the user-facing counterpart of the same condition that drives FR-OD-03 in the occupancy workflow: an unreachable camera should produce a clear, immediate signal to the user rather than a frozen or blank video panel, consistent with the fault-visibility requirements of FRMD-04 and FR-FH-03.

This diagram complements the occupancy workflow of Section 3.8.1 rather than duplicating it: that diagram shows the camera being used as a sensor input to an automated decision, while this diagram shows the same camera being used as a direct visual aid for a human administrator. Both depend on the camera being reachable, which is why camera connectivity is treated as a fault condition (`CAMERA_UNAVAILABLE/CAMERA_TIMEOUT`) in FR-OD-03 and FR-FH-01 regardless of which consumer — `—YOLO or a human — — is currently using the stream.`

3.8.3 Manual Override (Relay Control) Workflow

This diagram has five lifelines — Instructor, Angular Dashboard, ASP.NET Backend, ESP32, and Relay Module — and is the most complete single-diagram demonstration of GreenLab’s governance dimension: in one continuous scenario it exercises authentication, role validation, relay actuation, dashboard feedback, and audit logging. It is the dynamic realisation of use case UC-03 and of the Manual Override (FR-MO) and Authentication and Access Control (FR-AC) requirement areas together, and it shows concretely how FR-LC-02 and FR-VC-04 (override suppresses automatic relay commands) are enforced at the point an override is created rather than left to be checked later.

35 GreenLab — Chapter 3: System Analysis

Figure 3.6: Sequence diagram — manual override (relay control) workflow.

The scenario opens with the Instructor opening the dashboard and logging in (step 1). This is wrapped in an opt [Authentication] guard, since a returning user with an unexpired JWT can skip straight past it: the Angular Dashboard posts the username

and password to `/api/auth/login` (step 2), and the ASP.NET Backend returns 200 OK with a JWT (step 3) — the direct execution of FR-AC01, which specifies bcrypt password verification and an HMAC-SHA256-signed token expiring in 60 minutes. The dashboard then loads (step 4).

With a session established, the Instructor sets lighting to 100 and the Angular Dashboard posts this to `/api/control/manual-override` with the JWT attached as a Bearer token (step 6). The backend's first action is to validate the JWT and confirm the caller's role (step 7, a self-call) — this is FR-AC-02 and FR-AC-03 being enforced at the middleware level as required by NFR-SEC-02, before any relay logic runs, and it is also the point at which FR-MO01's constraint that an instructor may only override a relay in their own assigned lab would be checked. Only once that check passes does a loop [Device Command] begin: the backend sends the payload (lighting 100, mode MANUAL) to the ESP32 (step 8), the ESP32 performs the relay 36 GreenLab —

actuation — channel ON, intensity 100 (step 9) — against the physical Relay Module, receives an acknowledgement (step 10), and forwards an ACK that the command was executed back to the backend (step 11). This loop framing reflects that a real override may legitimately need to retry or re-issue the device command, not that the user repeats their request.

Once the device confirms execution, the backend updates its dashboard data store (step 12, a selfcall corresponding to writing a new RelayStates row with CommandSource = Override, satisfying FR-LC-04) and returns 200 OK with state MANUAL and lighting 100 to the Angular Dashboard (step 13). The outcome is then resolved by an alt block: on [Success], the dashboard displays a MANUAL badge and updates its indicators (step 14), giving the instructor immediate visual confirmation that automation has been suspended for that relay, exactly as FR-LC-02 requires; on [Failure], the dashboard shows an error message instead (step 15), consistent with the same failvisibly principle used in the camera streaming workflow of Section 3.8.2. Independently of which branch was taken, the backend logs the action to the audit table with timestamp, user ID, action, and resulting state (step 16, a self-call) — the direct execution of FR-MO-04 and FR-LC-04, and the reason the AuditLog table is defined as INSERT-only under NFR-SEC-03: this final step is not optional or conditional on success, because both a successful and a failed override attempt are governance-relevant events that must be attributable.

Taken together with the other diagrams, this workflow demonstrates the precedence rule of FRMO-02 implicitly: because the override-creation path runs through the same role-validation selfcall at step 7, an administrator's request would be authorised to target any laboratory's relay, while an instructor's request is constrained to their own AssignedLabId, and a later administrator override on the same relay would simply repeat steps 6–16 and overwrite the active OverrideCommand row, which is why FR-MO-02 states that an administrator override supersedes an existing instructor override rather than being rejected by it.

3.8.4 Laboratory Transfer Request Workflow

This diagram has five lifelines — Instructor, Angular Dashboard, ASP.NET Backend, Administrator, and Notification Service — and models the governance workflow through which an instructor formally requests that their session be moved to a different laboratory, and an administrator reviews and resolves that request. It is the dynamic realisation of use case UC-06 (Approve/Reject Transfer Requests) together with UC-04 (Submit Laboratory Transfer Request),

37 GreenLab — Chapter 3: System Analysis

and it exercises the TransferRequests table, the role-based authorisation layer (FR-AC), and the audit logging requirement (FR-MO-04) in a single end-to-end scenario.

Figure 3.7: Sequence diagram — laboratory transfer request workflow.

The scenario opens with the Instructor logging in and opening the dashboard (step 1), protected by the same opt [Authentication] guard used in Section 3.8.3: the Angular Dashboard posts credentials to /api/auth/login (step 2), the ASP.NET Backend returns 200 OK with a JWT (step 3), and the dashboard loads (step 4). This authentication preamble is identical across all instructor-initiated workflows because FR-AC-01 applies uniformly — no action is reachable without a verified identity and a role-scoped token.

With a session established, the Instructor submits a transfer request through the dashboard form (step 5), and the Angular Dashboard posts it to /api/transfer/request with the JWT as a Bearer token (step 6). The backend validates the JWT and confirms the instructor role (step 7, a self-call) — enforcing FR-AC-02 and FR-AC-03 at the middleware level before any business logic runs. Once authorised, the backend creates a new TransferRequests row with status PENDING (step 8, a selfcall), and returns 201 Created to the Angular Dashboard (step 9), which confirms the submission to 38 GreenLab — Chapter 3: System Analysis

the Instructor (step 10). This creation step is the direct execution of the Submit Laboratory Transfer Request use case (UC-04).

The workflow then shifts to the administrator’s side, modelled here as a separate lifeline that picks up asynchronously. A loop [Pending Requests] reflects that an administrator may review one or many pending requests in a single session. The Notification Service (step 11) alerts the Administrator that a new transfer request is awaiting review, and the Administrator retrieves the request details from the backend via GET /api/transfer/id (step 12), receiving the full TransferRequests record in response (step 13). The administrator then makes a resolution decision, captured by an alt block: on [Approve], the Angular Dashboard puts /api/transfer/id/approve (step 14), the backend updates the TransferRequests row to APPROVED (step 15, a self-call), and the Notification Service sends an approval notification to the Instructor (step 16); on [Reject], the Angular Dashboard puts /api/transfer/id/reject (step 17), the backend updates the row to REJECTED (step 18, a self-call), and the Notification Service sends a rejection notification instead (step 19). This branching realises the Approve/Reject Transfer Requests use case (UC-06) and its «extend» relationship: approval and rejection are both outcomes of the same review action rather than separate triggers.

Independently of the resolution branch, the backend logs the entire transaction

— request creation, review, and outcome — to the AuditLog table with timestamp, userID, action, and resulting state (step 20, a self-call), satisfying FR-MO-04 and the INSERT-only constraint of NFR-SEC-03. As in the manual override workflow, this logging step is unconditional: a rejected transfer request is as governance-relevant as an approved one, because the audit trail must be able to show not only what changes were made to laboratory assignments, but also which requests were considered and declined.

Together, the four sequence diagrams of Section 3.8 demonstrate that GreenLab’s governance model — role-scoped tokens, explicit decision branches, mandatory audit records — applies uniformly across automated, administrative, and instructor-initiated workflows rather than being confined to any single scenario.

39 GreenLab — Chapter 3: System Analysis

3.9 Chapter Summary

This chapter established the analytical basis for the GreenLab system. Section 3.2 positioned GreenLab as a four-tier cyber-physical system shaped by two forces beyond simple sensor-to-relay automation: a temporal dimension driven by irregular academic scheduling, and a governance dimension driven by institutional accountability requirements — and showed how both are answered by safe defaults at every tier rather than by assuming reliable connectivity. Section 3.3 then positioned that same architecture commercially, using the Business Model Canvas to show that GreenLab’s automation behaviour, governance controls, and reliability targets double as the value propositions, cost drivers, and revenue streams of a viable product rather than a prototype alone.

Section 3.4 identified the six stakeholder groups — Instructor, Administrator, Laboratory Staff, Student, IT Department, and Faculty Management — whose concerns motivate every feature in the system. Section 3.5 converted those concerns into a formal use case model of thirteen use cases across two human and two automated actors, distinguishing governance actions that require an authenticated session from automation actions that run continuously.

Sections 3.6 and 3.7 then fixed the testable requirement baseline for the rest of the project: thirty-six functional requirements across the nine areas of System Initialisation and Startup, Entry and Exit Detection, Occupancy Detection and Classification, Lighting Control, Ventilation Control, Monitoring and Dashboard, Manual Override, Fault Handling and Safety, and Authentication and Access Control; and the non-functional requirements spanning performance, reliability, security, usability, maintainability, and scalability. Every requirement identifier, threshold, and design target stated in these two sections is taken verbatim from the TA Reference Document and is treated in this project as the single source of truth, taking precedence over any earlier draft figures that may appear elsewhere.

Finally, Section 3.8 validated that requirement baseline dynamically by walking through the four sequence diagrams produced for GreenLab. The occupancy-triggered automation workflow showed how a motion event, a YOLO-confirmed person count, and a debounce-aware decision combine to switch lighting and ventilation automatically.

The live camera streaming workflow showed how the same camera infrastructure also serves a human administrator directly, with an explicit failure path rather than a silent one. The manual override workflow tied authentication, role 40 GreenLab — Chapter 3: System Analysis

validation, relay actuation, dashboard feedback, and audit logging into a single traceable scenario, demonstrating concretely how an override suppresses automation and how administrator precedence over instructor overrides is structurally guaranteed rather than merely documented. The laboratory transfer request workflow completed the picture by modelling the institutional governance process through which an instructor requests a room change and an administrator approves or rejects it, demonstrating that the same role-scoped authorisation, conditional branching, and unconditional audit logging that govern hardware-level actions apply equally to organisational workflows.

Together, the business model, the stakeholder and use case analysis, the requirement tables, and the sequence diagram analysis in this chapter form the contract that the design chapter must satisfy: every component introduced in Chapter 4 — the database schema, the REST API, the automation engine, the AI service, and the Angular dashboard — exists because a specific requirement, use case, or sequence-diagram step in this chapter demands it.

Part II

Design and Implementation

Chapter 4

System Design

System Design

This chapter translates the functional and non-functional requirements established in Chapter 3 into a complete, concrete design for GreenLab. The design is organised across eight sections that follow the system’s four-tier structure: hardware layer (Section 4.2), Python AI service (Section 4.3), ASP.NET Core backend including the database, REST API, automation engine, and security design (Sections 4.4–4.7), and Angular dashboard frontend (Section 4.8). Each section specifies the component topology, interface contracts, data flows, and decision logic with sufficient precision to guide a direct implementation.

Two conventions hold throughout this chapter. First, all four tiers are referred to by their fixed names — the hardware layer, the Python AI service, the ASP.NET Core backend, and the Angular dashboard — consistent with the terminology established in Chapter 1 and formalised in Chapter 3. Second, every design decision that implements a specific requirement cites the requirement’s identifier (FR-XX-NN or NFR-XX-NN) so that the design can be traced backward to the requirement it satisfies and forward to the implementation described in Chapter 5.

4.1 Four-Tier Architecture Overview

GreenLab is structured as a four-tier cyber-physical system. Each tier owns a single, wellbounded responsibility and communicates with its neighbours through a defined interface, so that a change within one tier — replacing the YOLO model, redesigning the dashboard, or extending the database schema — does not require a corresponding change in any other tier.

Table 4.1: GreenLab four-tier architecture — tiers, responsibilities, and technologies.

Tier	Name	Primary Responsibility	Core Technology	Sense
1	Physical Laboratory	environment every polling cycle;	ESP32 microcontroller,	actuate relay-controlled appli-
2	Hardware	on command received from the channel relay module,	Layer backend;	maintain JWT- Arduino C++ firmware, authenticated HTTP communication;
3	Python AI service			WiFi HTTP client fall back to local logic on network loss.

Tier Name Primary Responsibility Core Technology Receive IP camera stream frames; run YOLO-based person detection Python 3, FastAPI, Python AI using ByteTrack persistent tracking; Ultralytics YOLO Service maintain per-camera state and (yolo11n.pt), OpenCV,

```
1 return a stable, de-duplicated person    ByteTrack tracker,
    uvicorn
```

count on demand.

Receive sensor telemetry from the hardware layer; query the AI service for occupancy count; evaluate the lighting decision matrix and fan ASP.NET Core 8, Entity ASP.NET activation table; Framework Core, SQL Core Server, JWT Bearer Backend persist all state changes and issue authentication, REST API authenticated relay commands; (Swagger/OpenAPI) enforce role-based access control and maintain an immutable audit log.

Present real-time laboratory status to authenticated instructors and administrators; issue timed manual Angular 17, TypeScript, Angular override commands; submit HTTP Client, reactive forms, Dashboard laboratory transfer requests; expose REST API consumer administrative controls for labs, devices, users, schedules, and audit logs.

The communication paths between tiers are: the hardware layer POSTs sensor events to the backend via HTTP and receives relay instructions in the JSON response body; the backend GETs the current person count from the Python AI service's /current-count endpoint on demand; and the Angular dashboard communicates with the backend exclusively through the authenticated REST API. There is no direct channel between the hardware layer and the dashboard, or between the hardware layer and the AI service — all routing passes through the backend.

4.1.1 Data Flow Diagrams

To complement the tier-level architecture in Table 4.1, this section presents two complementary data flow diagrams (DFDs) that trace how information moves through GreenLab at increasing levels of detail. The context-level diagram (Figure 4.1) treats the system as a single process and defines its boundary with the four external actors, while the Level-1 diagram (Figure 4.2) decomposes that single process into the five functional processes and three persistent data stores that realise the tier responsibilities summarised above.

Figure 4.1: GreenLab context-level data flow diagram (DFD Level 0) — the system as a single process bounded by four external actors and three data stores.

The context diagram fixes the system boundary. Two human actors — the Instructor and the Administrator — and two hardware actors — the ESP32 hardware and the IP camera — exchange data exclusively with the GreenLab System process. The instructor issues manual override commands and laboratory transfer requests and receives lab status and override confirmations in return; the administrator issues user, device, and schedule management commands and transfer decisions, and receives lab status, audit log entries, and alerts. The ESP32 hardware reports sensor events — motion, distance,

LDR, temperature, and humidity readings — and receives relay commands for light and fan control, while the IP camera contributes a continuous video frame stream. Internally, the system persists data to three stores: the Sensor/Device Data Store, the Schedule & User Management DB, and the Audit Log Archive.

Figure 4.2: GreenLab Level-1 data flow diagram — decomposition of the GreenLab System process into five functional processes and three data stores.

The Level-1 diagram decomposes the single context process into the five processes that map directly onto the tier responsibilities in Table 4.1. Process 1.0 (Authenticate User) validates instructor and administrator credentials against D2 — User & Schedule Data before routing role-scoped commands onward. Process 2.0 (Process Sensor Event) receives sensor readings from the ESP32 hardware and the video frame stream from the IP camera, persists them to D1 — Sensor/Device Data, and forwards occupancy-relevant events onward. Process 3.0 (Manage Devices) issues relay commands — light and fan control — back to the ESP32 hardware and handles instructor override requests validated by Process 1.0. Process 4.0 (Manage Lab Schedule) and Process 5.0 (Generate Logs & Alerts) close the loop by managing transfer requests and writing every state change to D3 — Audit Log, from which lab status, alerts, and audit records are returned to both the instructor and the administrator.

4.2 Hardware Layer Design

4.2.1 Component Topology

The hardware layer is built around a single ESP32 microcontroller that reads four physical sensors and drives two relay channels. The following pin assignments are taken directly from the implemented firmware and govern both the circuit layout and the software configuration.

Table 4.2: ESP32 pin assignments and sensor specifications.

Component GPIO Signal Function Threshold / Range Pin(s) Type Entry and proximity TRIG: detection — triggers a Trigger if distance HC-SR04 GPIO 4 / Digital sensor event when an ≤ 50 cm; timeout at Ultrasonic ECHO: PWM object is within 30 000 μ s GPIO 2 threshold distance

Environmental monitoring — Temperature 0–50 DHT11 Single- temperature and °C; Humidity 20– Temperature & Humidity GPIO 5 wire digital humidity readings 90included in every sensor every loop iteration event payload

Natural-light level Dark < 1000; Dim LDR (Light- GPIO 34 Analogue monitoring — 1000–2500; Bright > Dependent Resistor) (ADC) 0–4095 determines whether 2500 (12-bit ADC, artificial lighting is needed

4.3 V reference)

LOW = ON (activeRelay Channel GPIO 26 Digital Controls laboratory low relay); HIGH = 1 (Light) output lighting circuit 1 OFF; default HIGH (safe-off) at startup

LOW = ON (activeRelay Channel Digital Controls laboratory low relay); HIGH = GPIO 27 2 (Fan) output ventilation fan circuit 1 OFF; default HIGH (safe-off) at startup

The following figure shows the complete wiring diagram for the hardware layer prototype.

Figure 4.3: ESP32 hardware layer wiring diagram — HC-SR04 ultrasonic sensor (top), DHT11 temperature/humidity sensor (left), LDR light sensor module (right), and two relay modules (bottom left and right) connected to the central ESP32 microcontroller.

4.3.1 Firmware Architecture

The firmware is structured around three concurrent concerns: JWT lifecycle management, sensor polling and HTTP event reporting, and a local WebServer that receives direct relay commands from the backend when the backend needs to override the automatic state. These three concerns interact through a set of shared state variables that are read in the main loop and written by either the event-response handler or the WebServer route handlers.

JWT Authentication Lifecycle

The firmware maintains a JWT token obtained from POST /api/devices/token with a payload containing labId, deviceId, and a device password. The token is cached in a global string and its expiry is tracked by recording millis() + 50 minutes at the time of issue, which approximates the server-side expiry without requiring the firmware to parse ISO 8601 timestamps. On every outbound HTTP request, ensureTokenValid() checks whether the cached token has expired; if so, it calls refreshToken() before proceeding. If a live request receives a 401 Unauthorized response, the firmware performs a single in-flight token refresh and retries the request with the new token before treating the failure as terminal. This design satisfies NFR-SEC-01 (authenticated device communication) without requiring a real-time clock module.

Sensor Polling and Event Reporting

The main loop runs at approximately 100 ms intervals (governed by the terminal

```
1 delay(100) call). On each iteration, the loop reads the HC-SR04
   distance, the LDR analogue
```

value, and the DHT11 temperature and humidity. A proximity event is triggered when distance $\leq$ 50 cm AND the previous reading was not near (rising-edge detection via the wasNear flag) AND at least 2 000 ms have elapsed since the last event (debounce via COOLDOWN). This edge-detection pattern prevents continuous event flooding while ensuring the backend is notified of every genuine entry event, satisfying FR-EE-01 and FR-EE-02.

When a proximity event fires, the firmware calls sendMotionRequest() which POSTs a JSON payload containing motionDetected, distanceCm, ldrValue, temperatureCelsius, and humidity to POST /api/devices/labId/event. The backend processes this event, evaluates

the automation engine, and returns a JSON response containing: lightOn, fanOn, relay3On, relay4On, occupancyCount, and motionIgnored. The firmware parses this response and actuates the relays accordingly. If occupancyCount is 0, both relays are forced OFF regardless of any other flags, implementing the empty-room safe-off rule.

Local WebServer for Direct Override

In parallel with the main loop, the firmware runs a WebServer on port 80 that exposes four endpoints: GET /light/on, GET /light/off, GET /fan/on, GET /fan/off. These endpoints allow the backend to issue direct relay commands to the ESP32 when a manual override is activated through the dashboard (FR-MO-01). The WebServer is serviced by calling server.handleClient() on every main loop iteration, so it shares the same thread as the sensor loop; the design relies on the low duty cycle of sensor reads and the short HTTP response time of the relay handlers to avoid contention.

Network Resilience and Fallback

If `WiFi.status()` is not `WL_CONNECTED` at the top of the main loop, the firmware attempts to reconnect. It compares the current `LD_R` reading against `LIGHT_THRESHOLD` (2000) and directly controls `Relay1` `RES - 01` (continued basic operation during network loss).

4.4 Python AI Service Design

4.4.1 Service Architecture

The Python AI service is a FastAPI application that runs on port 8000 and exposes two endpoints: GET /current-count and GET /health. It is designed as a stateful streaming service: a dedicated per-camera background thread continuously reads frames from a camera stream, runs YOLO ByteTrack inference, and updates a shared in-memory count that the endpoint returns synchronously. This design avoids per-request camera initialisation overhead and provides sub-second response times to the backend's polling requests.

4.4.2 Model and Inference Pipeline

The service loads a single YOLO model instance (yolo11n.pt — the nano variant) at startup using a thread-safe double-checked locking pattern, ensuring the model is loaded exactly once regardless of concurrent request traffic. The model is placed on the CPU

(`MODEL_DEVICE = 'cpu'`) to maintain compatibility with deployment environments that do not have

Table 4.3: Python AI service inference configuration parameters.

Parameter Value Rationale Nano variant — lowest parameter count; runs Model yolo11n.pt within CPU budget for real-time building automation

ByteTrack persistent tracking assigns stable IDs by `bytetrack.y` across frames, preventing the ID-fragmentation Tracker aml overcounting problem documented by Kim et al. (Section 2.6)

Reduced input size — halves inference time `imgsz` relative to 640 with acceptable accuracy loss for occupancy counting

Minimum detection confidence; filters spurious conf (`CONFIDENCE`)

4.5 low-confidence boxes while retaining seated

$E_T HRESHOLD$) *occupants partially occluded by desk equipment*

Person class only — eliminates false positives from classes [0] chairs, bags, and other indoor objects

Inference runs every 3rd frame, reducing CPU load $FRAME_S KIP$ by 67% frequency

Enables ByteTrack cross-frame ID persistence — persist true the key to stable, non-inflated counts

4.5.1 Camera Source Abstraction

The service decouples camera URL resolution from the inference loop through the `camera_source` module. The `resolve_source()` function accepts any of the following source formats and norms

USB device: `usb:0`, `usb:1` → integer device index

Network stream (passed through unchanged): `http://`, `https://`, `rtsp://`

Video file by extension: `.mp4`, `.avi`, `.mov`, `.mkv`, `.flv`, `.ts`

Bare IPv4 address (DroidCam backward compatibility): `192.168.x.x` → `http://192.168.x.x:4747/video`

This abstraction means the backend can register any IP camera by simply passing its IP address to the `/current-count?ip=` endpoint, without the service requiring per-camera configuration at deployment time.

4.5.2 Camera Lifecycle Management

Each distinct stream URL is registered in an in-memory `camera` dictionary keyed by the resolved URL string. The `camera` class holds the current count, connection status, a threading event that signals

4.5.3 Backend Integration Contract

The backend integrates with the AI service through a single polling call:

GET `http://AI_SERVICE_HOST:8000/current-count?ip=CAMERA_IP&port=PORT`

The response JSON is:

```
1 { "camera": "<resolved_url>", "people_count": <int>, "
  connected": <bool> }
```

The backend maps the `people_count` integer to an occupancy classification (`Empty` = 0, `Low` = 1–5, `Moderate` = 6–15, `High` > 15) and passes this classification into the lighting decision

4.6 Database Design

4.6.1 Schema Overview

The GreenLab database contains eleven tables managed by Entity Framework Core with a SQL Server provider. The schema is structured around three functional domains:

laboratory management (Labs, LabSensors, LabDevices, StaffLabAssignment), operational control (LabControlOverrides, ManualControlCommands, LabSectionSchedules, LabChangeRequests), and audit and identity (Users, AuditLogs, ScheduleImageImports). The *EFMigrationsHistory* table is managed automatically by EF Core and is not part of the application database.

The following figure shows the complete entity-relationship diagram for the GreenLab database.

Figure 4.4: GreenLab database entity-relationship diagram — 11 application tables spanning laboratory management, operational control, and audit/identity domains.

4.6.2 Table Specifications

Table 4.4: GreenLab database table definitions and key columns.

Table Purpose Key Columns Notable Constraints Master record for each Id (PK), Name, LightThreshold and Labs Directory, TempFanOnThreshold physical laboratory LightThreshold, d store per-lab

Table Purpose Key Columns Notable Constraints TempFanOnThreshold calibration values; Id, ClubCode, LabOccupancyCount CameraResult, is updated on every LabOccupancyCount sensor event

Role is an enum Authenticated system Id (PK), Email, (Admin, Staff, users FullName, Device); Users (instructors, administrators PasswordHash, Role, PasswordHash ors, service accounts) CreatedAt stores BCrypt-hashed credential; Email is unique

SensorType constrains the Id (PK), LabDeviceId Registered sensor reading (FK), LabId (FK), LabSensors hardware within each interpretation; Name, SensorType, laboratory IsActive allows soft- Unit, IsActive disable without deletion

DevicePasswordHas Id (PK), LabId (FK), ESP32 or other IoT h enables the device- Name, controller nodes level JWT token LabDevices DevicePasswordHas registered per endpoint; LastSeen h, DeviceType, laboratory updated on every IsActive, LastSeen event receipt

ValidFrom/ValidUntil Id (PK), UserId (FK), support time- LabId (FK), bounded Tracks which instructor DayOfWeek, StaffLabAssign assignments; is currently assigned to StartTime, EndTime, n ment AssignmentSource which laboratory ValidFrom, distinguishes manual ValidUntil, vs schedule-import AssignmentSource assignments

Table Purpose Key Columns Notable Constraints LabId (FK), MotionSensorDisable One row per lab; ed, ForceLightOn, SetByUserId Records the current ForceRelayOn, LabControlOv provides the actor manual override state ForceRelay2On, er rides link to AuditLogs; for each laboratory ForceRelay3On, UpdatedAt enables ForceRelay4On, expiry evaluation SetByUserId, UpdatedAt

Target identifies Id (PK), LabId (FK), which relay; RequestedByUserId Command-Status Log of every manual (FK), Target, tracks ManualControl relay command issued RequestedValue, pending/executed/fai IC ommands through the dashboard Command-Status, le d lifecycle; RequestedAt, AcknowledgedAt AcknowledgedAt records ESP32 confirmation

Imported from Id (PK), LabId (FK), ScheduleImageImpo Timetable sessions Day-Of-Week, rts pipeline; used by LabSectionSch imported or manually StartTime, End-

Time, the backend to e dules created for each CourseName, determine laboratory AcademicTerm scheduled-active periods

Id (PK), Status lifecycle: RequestedByUserId Pending → Instructor requests to (FK), FromLabId Approved/Rejected; LabChangeRe (FK), ToLabId (FK), transfer a laboratory ReviewedByUserId q uests SectionStart, assignment records the SectionEnd, administrator who RequestedAt, Status, actioned it ReviewedByUserId,

Table Purpose Key Columns Notable Constraints ReviewedAt

Id (PK), FileName, Supports the StoredPath, schedule-import Metadata for schedule ScheduleImag ContentType, workflow (POST images uploaded for AI- eI mports FileSizeBytes, /api/admin/schedules assisted parsing UploadedByUserId, /i mport-image → UploadedAt, Notes analyze → apply)

INSERT-only by design; no UPDATE Id (PK), Action, Immutable record of or DELETE Details, Entity, AuditLogs every state-changing permissions granted UserId (FK), action in the system to the application CreatedAt user; CreatedAt is server-assigned

4.7 Automation Engine Design

4.7.1 Occupancy Classification

The backend maps the raw *people_countintegerreturnedbytheAI servicetoafourleveloccupancyclassification* *controllogicfromtheprecisecountvalue, makingthedecisionrulesstableundersmallframe-* *to – framecountfluctuations.*

Table 4.5: Occupancy classification thresholds.

people_countRelayBehaviourDefaultClassificationRangeAllrelaysforcedOFF – – hardcodedsafe – of fruleEmptythatcannotbeoverriddenbylighting/fanmatrices

Low 1–5 Lighting decision matrix evaluated with LDR; Fan

people_countRelayBehaviourDefaultClassificationRangeactivationtableevaluatedwithtemperature

As Low — same matrix evaluation; fan may Moderate 6–15 activate at a lower temperature threshold

As Low/Moderate — fan activation priority High > 15 elevated; lighting assumed required regardless of LDR

4.7.2 Lighting Decision Matrix

The lighting decision matrix combines three inputs — LDR zone, occupancy classification, and the current manual override state — to determine the value of lightOn in the event response. LDR zones are derived from the per-lab LightThreshold column in the Labs table, which allows each laboratory to be calibrated independently.

Table 4.6: Lighting decision matrix (LDR Zone × Occupancy Classification → lightOn).

LDR Threshold Empty Low Moderate High Zone ADC < OFF Dark LightThreshold
× (safe- ON ON ON

4.8 off)

LightThreshold $\times$ OFF Dim 0.5 – (safe- ON ON ON LightThreshold off)

ON (high OFF OFF (natural OFF (natural Adequat LightThreshold – density (safe-light light e LightThreshold overrides off) sufficient) sufficient) natural light)

$\times$ 1.5 Bright > LightThreshold $\times$ 1.5 OFF (safe-off) OFF OFF OFF

Manual override (LabControlOverrides.ForceLightOn = true) short-circuits the matrix entirely and sets lightOn = true regardless of LDR or occupancy. The backend writes an AuditLog entry for every override activation and deactivation, recording the actor, the previous state, and the timestamp.

4.8.1 Fan Activation Table

The fan activation table maps temperature zone (derived from the per-lab TempFanOnThreshold column) and occupancy classification to the value of fanOn in the event response.

Table 4.7: Fan activation table (Temperature Zone $\times$ Occupancy Classification $\rightarrow$ fanOn).

Temperature Range Empty Low Moderate High Zone Cool < TempFanOnThreshold – 5°C OFF OFF OFF OFF

TempFanOnThreshold – 5°C to Comfortable OFF OFF OFF ON TempFanOnThreshold

TempFanOnThreshold to Warm OFF ON ON ON TempFanOnThreshold + 5°C Hot > TempFanOnThreshold + 5°C OFF ON ON ON

As with lighting, ForceRelay2On = true in LabControlOverrides overrides the fan table and forces fanOn = true. The Empty safe-off rule applies to fans as it does to lights: occupancyCount = 0 forces all relays OFF before the fan table is evaluated.

4.9 REST API Design

4.9.1 API Structure

The GreenLab REST API is implemented in ASP.NET Core 8 and documented through Swagger/OpenAPI. Endpoints are organised into five groups, each with its own controller and role-based authorization policy: Auth, Devices, Schedules (admin), Staff, and AdminLabs. All endpoints require HTTPS in production. The lock icon visible in the API

documentation confirms that every endpoint requires a valid JWT Bearer token; the sole exception is POST /api/Auth/login, which issues the token.

4.9.2 Complete Endpoint Catalogue

Table 4.8: Full GreenLab REST API endpoint catalogue.

Group Method Path Authorised Role(s) Description Authenticate a user with email and POS None Auth /api/Auth/login password; returns T (public) JWT Bearer token and expiry

Receive sensor event from ESP32; evaluate automation engine;

1 **return relay**

POS Device Devices /api/devices/labId/event command response T (JWT) (lightOn, fanOn, relay3On, relay4On, occupancyCount, motionIgnored)

Issue a JWT token None to an authenticated POS Devices /api/devices/token (device ESP32 device using T password) labId, deviceId, and device password

POS Register a new IoT Devices /api/devices Admin T device in the

Group Method Path Authorised Role(s) Description system

Retrieve current Admin / Devices GET /api/devices/light light state for all Staff registered devices

Upload a timetable Schedule POS /api/admin/schedules/import- image for AI-Admin s T image assisted schedule extraction

Manually submit Schedule POS /api/admin/schedules/import- parsed schedule Admin s T data data for a laboratory

Schedule List all schedule GET /api/admin/schedules/imports Admin s import records

Trigger AI analysis Schedule POS /api/admin/schedules/analyze- of the most Admin s T image recently uploaded schedule image

Trigger AI analysis Schedule POS /api/admin/schedules/analyze- of a specific Admin s T import/importId uploaded schedule image by import ID

Apply the results of an analysed Schedule POS /api/admin/schedules/apply- schedule import to Admin s T analysis the LabSectionSchedules table

Staff GET /api/staff/me/current-labs Staff Return the list of

Group Method Path Authorised Role(s) Description laboratories currently assigned to the authenticated instructor

Issue a manual override command for the specified POS Staff /api/staff/labs/labId/override Staff laboratory T (activates ForceLightOn or ForceRelay2On)

Submit a POS laboratory transfer Staff /api/staff/lab-change-requests Staff T request (Pending status)

List all transfer requests submitted /api/staff/lab-change-requests/ Staff GET Staff by the mine authenticated instructor

AdminLa List all registered GET /api/admin/labs Admin bs laboratories

AdminLa POS Create a new /api/admin/labs Admin bs T laboratory record

AdminLa Retrieve a single GET /api/admin/labs/id Admin bs laboratory by ID

AdminLa Update laboratory PUT /api/admin/labs/id Admin bs configuration

Group Method Path Authorised Role(s) Description (name, thresholds, club code)

Delete a laboratory AdminLa DELETE /api/admin/labs/id Admin record (soft or hard bs TE delete per policy)

Add a timetable AdminLa POS /api/admin/labs/id/sections Admin section directly to a bs T laboratory

List all devices AdminLa GET /api/admin/labs/labId/devices Admin registered to a bs laboratory

Register a new AdminLa POS /api/admin/labs/labId/devices Admin device for a bs T laboratory

AdminLa /api/admin/labs/labId/devices/ Retrieve a specific GET Admin bs deviceId device record

AdminLa /api/admin/labs/labId/devices/ Update device PUT Admin bs deviceId configuration

AdminLa DELETE /api/admin/labs/labId/devices/ Remove a device Admin bs TE deviceId from a laboratory

Regenerate the AdminLa POS /api/admin/labs/labId/devices/ device password / Admin bs T deviceId/token JWT credential for a specific device

Force lights ON AdminLa POS across all /api/admin/labs/light/on Admin bs T laboratories under administrator

Group Method Path Authorised Role(s) Description authority

Force lights OFF AdminLa POS /api/admin/labs/light/off Admin across all bs T laboratories

Force fans ON AdminLa POS /api/admin/labs/fan/on Admin across all bs T laboratories

Force fans OFF AdminLa POS /api/admin/labs/fan/off Admin across all bs T laboratories

4.9.3 Sensor Event Request and Response Contract

The most critical endpoint — POST /api/devices/labId/event — defines the primary communication contract between the hardware layer and the backend. The request body is:

POST /api/devices/labId/event

Authorization: Bearer <device_JWT>

Content-Type: application/json

```
1 {
    "motionDetected": true, "distanceCm": 35, "ldrValue": 1450, "temperatureCelsius":
    28.5, "humidity": 62.0
  }
```

The response body is:

```
1 {
    "lightOn": true, "fanOn": false, "relay3On": false, "relay4On": false, "occupancy-
    Count": 4, "motionIgnored": false
  }
```

The `motionIgnored` field is set to `true` when the backend's automation engine determines that the event should not trigger a relay state change — for example when a `LabControlOverride` with `MotionSensorDisabled = true` is active for this laboratory. The firmware treats `motionIgnored = true` as a terminal condition and does not apply the `lightOn` or `fanOn` values.

4.10 Security Design

4.10.1 Authentication Architecture

GreenLab uses a dual-credential authentication architecture. Human users (instructors and administrators) authenticate against the `Users` table through `POST /api/Auth/login` using email and BCrypt-hashed password. IoT devices authenticate through `POST /api/devices/token` using a device-specific password stored in `LabDevices.DevicePasswordHash`. Both paths return a JWT Bearer token with a configurable expiry (50 minutes in the current firmware, matching the server-side token lifetime).

4.10.2 Role-Based Access Control

The `Users.Role` column carries one of three values that map directly to ASP.NET Core authorization policies:

Table 4.9: Role definitions and permitted endpoint groups.

Role	Holder	Permitted Endpoint Groups	Key Restrictions
Faculty	IT	Full system access; can All	— Auth, Devices, Admin
administrators	override any lab, Schedules, Staff	(read-only n and lab approve/reject transfer view), AdminLabs coordinators requests, manage users	

Scoped to assigned labs Instructors only; cannot access Staff assigned to one or Auth, Staff endpoints only AdminLabs endpoints; more laboratories override is time-limited and logged

Cannot access any `POST /api/devices/token`, Device ESP32 firmware human-facing endpoints; `POST e` (service account) identity is per-device, not `/api/devices/labId/event` only per-user

4.10.3 Audit Logging Design

Every state-changing action — automated relay command, manual override activation, override cancellation, transfer request submission, administrator approval, user creation,

device registration, schedule import — results in an INSERT into the AuditLogs table. The AuditLogs table carries no UPDATE or DELETE grant for the application database user, making every record effectively immutable from the application layer. Each record contains: Action (the operation type), Details (a JSON-serialisable description of the change), Entity (the affected entity type and ID), UserId (the authenticated actor), and CreatedAt (server-assigned timestamp). This design satisfies NFR-SEC-02 (tamperresistant audit trail) and supports the administrative audit-log review view described in Section 4.8.

4.11 Angular Dashboard Design

4.11.1 View Architecture

The Angular dashboard is a single-page application structured around a role-gated routing module. On login, the JWT response's role claim determines which navigation group is activated: Staff users see only their assigned laboratory views; Admin users see

both the administrative panel and can access all laboratory monitoring views. Unauthenticated routes redirect to the login page. The HTTP interceptor attaches the stored JWT as a Bearer token to every outbound API call and redirects to login on 401 responses.

4.11.2 View Inventory

Table 4.10: Angular dashboard views, their authorised roles, and primary data sources.

View	Route	Role	Primary API Key	User Actions (indicative)	Endpoints	Logi	Publi
POST Enter	/login	n	c	/api/Auth/login	receive JWT; redirect to role-		

appropriate home view	View live sensor readings (temperature, humidity, LDR, Real-Time Monitor labs states, occupancy count, and active overrides for each assigned laboratory	My Labs — GET					
-----------------------	--	---------------	--	--	--	--	--

Toggle light or fan relay state; set Override Staff view current override holder and expiry	Manual POST override duration; /staff/labs/:id/override	Staff	/api/staff/labs/labId/o	cancel active override	Panel v	erride	override;
---	---	-------	-------------------------	------------------------	---------	--------	-----------

Transfer /staff/change-requests/new	change-requests ToLab, session time	POST	/api/staff/lab-	Select FromLab, Staff Request re-			
-------------------------------------	-------------------------------------	------	-----------------	-----------------------------------	--	--	--

appropriate home view window; submit request for Form administrator review							
--	--	--	--	--	--	--	--

View status (Pending/Approved/ and Transfer Staff requests transfer requests	My /staff/change-requests/mine	admin	notes for all Requests submitted	GET	/api/staff/lab-	Re jected)	
--	--------------------------------	-------	----------------------------------	-----	-----------------	------------	--

Monitor all laboratories simultaneously; see occupancy, relay sensor All Labs overrides at a glance; quick-access light/fan master controls	Admin — Admi states, All Labs /admin/labs	GET	/api/admin/labs n	readings, and active Dashboard			
---	---	-----	-------------------	--------------------------------	--	--	--

Edit lab name, LightThreshold, Admin — TempFanOnThresho Lab Admi GET/PUT /admin/labs/:id ld , ClubCode; Configurati n /api/admin/labs/id manage lab-specific on sensor and device records

Admin — GET/POST/PUT/ Register, update, or Device /admin/labs/:id/ DELETE deregister ESP32 Admi Manageme devices /api/admin/labs/labId devices; regenerate n nt / devices/... device credentials; view LastSeen

appropriate home view timestamp

Upload timetable image; trigger AI Admin — /admin/ analysis; review Admi POST/GET Schedule schedules/ parsed result; apply n /api/admin/schedules/... Import import to LabSectionSchedule s

Admin — POST Force all lights or all /admin/labs/ Admi fans ON/OFF across Master Lab /api/admin/labs/light/o controls n the entire Controls n| off, fan/on|off laboratory

estate in one action; each action is audit- logged Filter by date range, actor, entity type, or Admin — Derived from Admi action; view full detail Audit Log /admin/audit AuditLogs via admin n of each log entry; Viewer endpoint export as CSV for compliance reporting

Review pending Admin — transfer requests; Transfer /admin/change- Admi Derived from approve or reject with Request requests n LabChangeRequests notes; view history of Review all requests by status and date

4.11.3 Real-Time Update Strategy

The dashboard uses a polling strategy for real-time status updates: the laboratory monitoring views poll their respective API endpoints every 5 seconds using Angular’s interval-based observable pattern. This interval is configurable and was chosen to balance freshness (the sensor-event loop fires approximately every 100 ms on the firmware side, but the backend aggregates state between polls) against unnecessary HTTP load. A future enhancement identified in Chapter 7 is to replace polling with SignalR WebSocket push notifications, which would reduce latency for time-sensitive override feedback.

4.12 Chapter Summary

This chapter presented the complete system design for GreenLab across all four tiers.

Section 4.2 specified the hardware layer’s component topology (five GPIO-connected components on a single ESP32), pin assignments, firmware architecture including JWT lifecycle management, event-triggered reporting, local WebServer override reception, and network fallback logic. Section 4.3 specified the Python AI service’s streaming inference pipeline (yolo11n.pt + ByteTrack, $FRAME_{S KIP} = 3$, $conf = 0.5$, $imgsz = 320$), *camerasourceabstraction*, and *thebackendintegrationcontract*. Section 4.4 described *allelevendatab* — — *thelightingdecisionmatrix* and *thefanactivationtable* — — — *bothanchoredtooper* —

4.13. inventoried the twelve Angular dashboard views with their routes, roles, API sources,

labcalibrationvaluesstoredinthedatabase. Section 4.6 catalogued all 31 REST API endpoints across five role RBAC model, and the INSERT – only audit log design. Section

4.13 inventoried the twelve Angular dashboard views with their routes, roles, API sources,

and user actions, and described the polling-based real-time update strategy.

Chapter 5 takes these design specifications and describes their direct implementation: representative ESP32 firmware excerpts, ASP.NET Core controller and service class structures, Python FastAPI route handlers, and Angular component architecture. Every implementation decision in Chapter 5 traces back to a design decision made in this chapter.

Chapter 5

Implementation

Implementation

5.1 Introduction

This chapter presents the fully as-built GreenLab system: every line of source code that runs on each tier of the architecture defined in Chapter 3, paired with annotated screenshots of the running administration and monitoring interface. Where Chapter 3 fixed what the system must do and Chapter 4 fixed how it would be structured, Chapter 5 is the proof that the structure was realised in working software. Every code listing that follows is taken verbatim from the project’s committed source files; every screenshot is from the live deployed dashboard, not a mock-up or wireframe.

GreenLab is a four-tier IoT system for smart energy management in university computerscience laboratories. The four tiers are: a hardware layer running on the ESP32 microcontroller, a Python AI processing service performing real-time person detection and tracking, a .NET Core REST API that hosts the business logic and database access layer, and an Angular web dashboard through which administrators and instructors monitor and configure the system. This chapter covers the first two tiers in full source-code depth — firmware (Section 5.2) and the Python AI service (Section 5.3) — and covers the dashboard interface (Sections 5.4 and 5.5) through annotated screenshots tied to functional requirements.

The chapter is structured as follows:

- Section 5.2 covers the ESP32 firmware in six subsections, each presenting a complete function or logical block with line-by-line commentary, design rationale, and requirement tracebacks.
- Section 5.3 covers the Python AI processing service in two stages: first the single-camera prototype (`cout_code.txt`), then the production multi-camera tracking API (`droidip.py` and `camera_sources.py`).
- Section 5.4 covers every screen of the Angular administration interface with annotated screenshots.

- Section 5.5 covers the monitoring dashboard.
- Section 5.6 provides a complete requirement traceability matrix.
- Section 5.7 walks one complete end-to-end motion event through all four tiers with real JSON payloads.
- Section 5.8 documents all known implementation gaps and technical debt.
- Section 5.9 summarises the chapter.

Two editorial decisions are stated up front. First, Wi-Fi credentials and device passwords that appear as plain string literals in the firmware are redacted in every listing in this chapter; only the surrounding logic that consumes them is shown. Second, this chapter is honest about the small number of places where the as-built code diverges from the Chapter 3 design — for example, the firmware’s API paths use `/api/devices/` rather than `/sensor/readings`, and relay channels three and four are parsed but not yet physically wired. These deviations are reported explicitly, not smoothed over.

5.2 Hardware Layer Implementation — ESP32 Firmware

The hardware layer is a single Arduino sketch, `arduino_code.txt`, that runs on an ESP32 microcontroller permanently installed in each smart laboratory. The sketch is 487 lines long and implements six coherent responsibilities: including the required libraries and declaring all hardware constants (Section 5.2.1); reading the three sensors (Section 5.2.2); responding to direct HTTP commands from the dashboard (Section 5.2.3); authenticating with the backend using JSON Web Tokens (Section 5.2.4); packaging sensor data and communicating it to the backend on motion events (Section 5.2.5); and running the main event loop that ties all of the above together (Section 5.2.6). Each section below presents the relevant code in full, followed by a detailed explanation.

5.2.1 Library Includes, Hardware Pin Definitions, and Global Constants

The top of the sketch begins with five library includes and a `WebServer` instance, followed by all `#define` macros for sensor and relay pins, all tunable constants, and all global variables used across functions.

```
1 #include <WiFi.h> 2 #include <HTTPClient.h>
3 #include <DHT.h> 4 #include <ArduinoJson.h> 5 #include <WebServer.h>
```

```
7 WebServer server(80);
```

Listing 5.1: Library includes and local web server declaration (`arduino_code.txt`, lines 4–10).

Each library serves a specific architectural role. `WiFi.h` provides the ESP32’s Wi-Fi stack, exposing the `WiFi.begin()` and `WiFi.status()` primitives used in `setup()` and in the loop’s connectivity guard. `HTTPClient.h` provides the high-level HTTP client used for both the token-exchange POST and the motion-event POST; it handles TCP connection management, keep-alive, and response body buffering internally, which would otherwise

require hundreds of lines of socket code. DHT.h is the Adafruit DHT sensor library that abstracts the one-wire protocol used by the DHT11; without it, the firmware would need to implement precise microsecond-level bit-banging for temperature and humidity acquisition. ArduinoJson.h is Benoît Blanchon’s widely-used embedded JSON library; it provides a document-oriented parser and serialiser that avoids dynamic heap allocation by working

```
1 from a statically-sized DynamicJsonDocument or
   StaticJsonDocument. WebServer.h
```

wraps the ESP32’s built-in TCP server and provides the `server.on()` route-registration and `server.handleClient()` polling API used for the local light and fan control endpoints described in Section 5.2.3.

Design note. WebServer runs on port 80 in addition to the main backend communication. This gives the Angular dashboard a direct HTTP path to toggle the light and fan without sending a round-trip through the .NET Core backend — useful for manual override during demos or when the backend is un restarting.

```
1 #define TRIG_PIN 4 // HC-SR04 ultrasonic trigger output 2 #define ECHO_PIN
2 // HC-SR04 ultrasonic echo input 3 #define RELAY_PIN 26 // Relay channel 1 –
controls laboratory lights 4 #define RELAY_PIN2 27 // Relay channel 2 – controls
laboratory fan 5 #define LDR_PIN 34 // Analogue input for LDR (light-dependent
resistor) 6 #define DHT_PIN 5 // Digital data pin for DHT11 temperature/humidity
sensor 7 #define DHT_TYPE DHT11
```

Listing 5.2: GPIO pin assignments for all sensors and relay channels (arduino_code.txt, lines 11–

17).

Pin selection is not arbitrary. GPIO 34 is chosen for the LDR because it is one of the ESP32’s input-only ADC pins — it cannot be accidentally configured as an output. GPIO 4 for TRIG_PIN and GPIO 2 for ECHO_PIN are general-purpose I/O pins; GPIO 2 is also connected to the ESP32’s built-in LED on most development boards, which gives a visual flicker on every ultrasonic pulse during debugging. GPIO 26 and 27 are DAC-capable pins on the ESP32, but they are used here purely as digital outputs to drive the relay module’s control inputs; no analogue output is needed. GPIO 5 for DHT_PIN is chosen because it is one of the few pins that does not have a boot-mode conflict on the ESP32. 1 int labId = 1; 2 int deviceId = 1;

```
4 const char* Devicepassword = "[REDACTED]"; 5 const char* ssid = "[REDACTED]";
6 const char* password = "[REDACTED]";
```

```
8 const char* serverUrl = "http://192.168.1.7:5076/api/devices/1/event"; 9 const
char* lightApi = "http://192.168.1.7:5076/api/devices/light"; 10 const char* TokenGen
= "http://192.168.1.7:5076/api/devices/token";
```

```
12 String jwtToken = ""; 13 unsigned long tokenExpireMillis = 0;
```

```
15 static bool lastLightState = false; 16 static bool lastFanState = false;
```

```
18 const int DISTANCE_THRESHOLD = 50; // centimetres 19 const unsigned
long COOLDOWN = 2000; // milliseconds
```

```
20 const int LIGHT_THRESHOLD = 2000; // LDR ADC value
21 const unsigned long LIGHT_INTERVAL = 5000; // ms (reserved for future periodic checks)
23 bool wasNear = false;
24 unsigned long lastToggleTime = 0;
26 DHT dht(DHT_PIN, DHT_TYPE);
```

Listing 5.3: Device identity, network endpoints, JWT state, relay state trackers, and tuning constants

(arduino_code.txt, lines 19–46).

labId and deviceId are the identifiers that the backend uses to look up which laboratory this ESP32 belongs to and which device record to authenticate against. They are compiled-in integer literals in the current build, which means reflashing is required to de- ploy the same firmware binary to a second laboratory with a different ID — a maintainabil- ity gap noted in Section 5.8 and tracked against NFR-MAIN-01.

The three API endpoint strings embed the local network IP address 192.168.1.7 and port 5076, which are the host:port of the .NET Core backend process on the development network. serverUrl points to the motion-event endpoint; lightApi points to an endpoint used for direct light-state queries (reserved for future use); TokenGen points to the device authentication endpoint.

jwtToken and tokenExpireMillis implement the client side of the JWT caching strategy: after a successful token exchange, jwtToken holds the bearer token string and tokenExpireMillis holds the millis() value at which the firmware will proactively refresh it. lastLightState and lastFanState are the differential-update guards that prevent unnecessary relay switching chatter — a relay is only toggled when the commanded state differs from the last applied state.

DISTANCE_THRESHOLD (50 cm) defines the boundary within which the HC-SR04 con- sidered a person to be “near” the sensor. COOLDOWN (2000 ms) is the minimum gap between consecutive motion-event POSTs, preventing the firmware from flooding the backend if a person lingers near the sensor. LIGHT_THRESHOLD (2000) is the ADC reading below which the local fallback logic considers the room too dark to leave the light off. LIGHT_INTERVAL is declared but currently unused — it was intended for a periodic light-level check indepen-

dent of motion events, which remains as planned future work.

Table 5.1: Firmware constants and their alignment with the Add Lab form defaults.

Constant	Value	Purpose	Configurable via Add Lab?
DISTANCE_THRESHOLD	50 cm	Defines “presence” zone for	No — hardcoded (see HC-SR04 §5.8)
COOLDOWN	2000 ms	Minimum gap between mo-	No — hardcoded tion reports
LIGHT_THRESHOLD	2000	LDR threshold for local	No — must match Add (ADC) fallback
LIGHT_INTERVAL	5000 ms	Reserved for periodic light	N/A — unused in
		checks	rent build

The table above confirms the alignment between firmware constants and the Add Lab form defaults (Section 5.4.1): the values were set consistently by hand on both sides during integration, not by the firmware reading them from the backend API.

5.2.2 Sensor Reading Functions

Three sensor-reading functions are defined before any of the communication or authentication logic. Keeping them short and side-effect-free makes the main loop's intent clear: sensors are read, a decision is made, and then — only if necessary — a network request is sent.

```
1 // HC-SR04 Ultrasonic Distance Measurement
2 long readDistanceCM()
3 // Step 1: ensure trigger is low for a clean pulse
4 digitalWrite(TRIG_PIN, LOW);
5 delayMicroseconds(2);
```

```
7 // Step 2: fire a 10-microsecond HIGH pulse on the trigger pin
8 digitalWrite(TRIG_PIN, HIGH);
9 delayMicroseconds(10);
10 digitalWrite(TRIG_PIN, LOW);
```

```
12 // Step 3: measure the duration of the echo pulse
13 // timeout of 30000 us corresponds to 5 m – beyond lab sensor range
14 long duration = pulseIn(ECHO_PIN, HIGH, 30000);
```

```
16 // Step 4: guard against timeout (no echo returned)
17 if (duration == 0) return -1;
```

```
19 // Step 5: convert travel time to distance
20 // sound travels 0.034 cm/us; divide by 2 for one-way trip
21 return duration * 0.034 / 2;
22
```

Listing 5.4: readDistanceCM(): HC-SR04 ultrasonic distance measurement with timeout guard

(arduino_code.txt, lines 48–59).

The HC-SR04 works by emitting eight 40 kHz ultrasonic pulses when the trigger pin is held HIGH for at least 10 microseconds. The sensor then raises the echo pin HIGH for a duration proportional to the round-trip travel time of the first reflected pulse. `pulseIn(ECHO_PIN, HIGH, 30000)` blocks until the echo pin goes HIGH and then measures how long it stays HIGH, with a 30,000-microsecond timeout. The 30 ms timeout is chosen to correspond to a maximum measurable distance of approximately 510 cm — well beyond the 50 cm threshold the firmware cares about, ensuring the timeout only fires when there is genuinely no reflecting object in range. The -1 sentinel return on timeout allows the caller to distinguish “no target” from a legitimately short distance of zero centimetres.

The conversion formula — $\text{duration} * 0.034 / 2$ — derives from the speed of sound at room temperature being approximately $340 \text{ m/s} = 0.034 \text{ cm}/\mu\text{s}$. Dividing by two accounts for the signal travelling to the target and back.

Physics note. At 26°C (the temperature logged in the Section 5.7 worked example), the speed of sound is approximately 346 m/s rather than the assumed 340 m/s , introducing a $\sim 1.8\%$ presence detection but worth noting for any future use case that requires precise ranging.

```
1 // LDR Ambient Light Reading
2 int readLight()
3 return analogRead(LDR_PIN);
4
```

Listing 5.5: readLight(): single-line LDR analogue read returning 0–4095 on the ESP32's 12-bit

ADC (arduino_code.txt, line 62).

`readLight()` is intentionally a single-expression function. The ESP32's ADC is 12-bit, returning values from 0 (no voltage) to 4095 (3.3 V). A lower LDR reading means

more light is present — the LDR’s resistance falls as light intensity increases, which reduces the voltage at the analogue pin when the LDR is wired in a voltage-divider configuration with a fixed resistor. `LIGHT_THRESHOLD = 2000` therefore corresponds to a mid-scale brightness level; readings below 2000 indicate a well-lit room and readings above 2000 indicate a darker room. The firmware’s local fallback turns the light on when the reading is above this threshold (i.e., the room is dark), matching the expected behaviour.

A known limitation of the ESP32’s ADC is significant non-linearity near 0 V and

5.3 V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore

treated as a relative brightness indicator rather than a calibrated lux measurement, which is sufficient for the binary “room is dark / room is bright” decision the firmware needs to make.

```
1 // DHT11 temperature and humidity (called in loop()) 2 float temp = dht.readTemperature();
3 float humidity = dht.readHumidity();
```

```
5 if (isnan(temp) || isnan(humidity)) 6 Serial.println("Sensor error"); 7 return; 8
```

Listing 5.6: DHT11 read and NaN guard in loop() (arduino_code.txt, lines 321–328).

The DHT11 sensor is read inline in `loop()` rather than in a dedicated function because both `readTemperature()` and `readHumidity()` must be called in the same loop iteration to remain synchronised. The DHT library internally caches the last successful reading and can return NaN (Not a Number) when the one-wire protocol communication fails — a common occurrence if the sensor is read more frequently than once per two seconds, or if there is electrical noise on the data line. The `isnan()` guard causes `loop()` to return immediately without sending any network request, avoiding a sensor-error value of NaN being serialised into the JSON body and causing a deserialization failure on the backend.

The DHT11’s rated accuracy is $\pm 2^{\circ}\text{C}$ for temperature and ± 5 with a measurement range of $0\text{--}50^{\circ}\text{C}$ and $20\text{--}90$

fan on if temperature $> 28^{\circ}\text{C}$), this accuracy is adequate — the 2°C uncertainty means the fan may switch on slightly early or slightly late, but it will not fail to respond to a genuinely hot room.

5.3.1 Local HTTP Server — Direct Relay Control Endpoints

In addition to communicating with the .NET Core backend, the ESP32 runs a lightweight HTTP server on port 80. This server accepts four routes that allow the Angular dashboard — or any HTTP client on the local network — to directly command the light and fan relays without going through the backend. This is the manual-override mechanism used during demonstrations and when the backend is temporarily unavailable. 1 void handleLightOff() 2 digitalWrite(RELAY_PIN, HIGH); // HIGH = relay open =

5.3. *V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore*

```
light OFF 3 Serial.println("Light OFF from API Button"); 4 server.send(200, "application/json", 5 "status:ok;light:off"); 6 lastLightState = false; // keep state tracker in sync 7
9 void handleLightOn() 10 digitalWrite(RELAY_PIN, LOW); // LOW = relay closed = light ON 11 Serial.println("Light ON from API Button"); 12 server.send(200, "application/json", 13 "status:ok;light:on"); 14 lastLightState = true; 15
17 void handleFanOn() 18 digitalWrite(RELAY_PIN2, LOW); // LOW = relay closed = fan ON 19 Serial.println("Fan ON from API"); 20 server.send(200, "application/json", "fan:on"); 21 lastFanState = true; 22
24 void handleFanOff() 25 digitalWrite(RELAY_PIN2, HIGH); // HIGH = relay open = fan OFF 26 Serial.println("Fan OFF from API"); 27 server.send(200, "application/json", "fan:off"); 28 lastFanState = false; 29
```

Listing 5.7: Four direct relay-control HTTP handlers: /light/on, /light/off, /fan/on, /fan/off
(arduino_code.txt, lines 64–102).

The relay logic convention used throughout the firmware deserves an explicit note: HIGH on a relay control pin opens the relay (disconnects the load), and LOW closes it (connects the load). This is the active-low convention used by the opto-isolated relay module connected to the ESP32. Writing HIGH on RELAY_PIN therefore turns the light off; writing LOW turns it on. This is counter-intuitive when reading the code for the first time but is correct for the hardware configuration in use.

Each handler updates the lastLightState or lastFanState tracker immediately after the GPIO write. This is critical: if the dashboard sends a direct /light/on command and then, moments later, the main loop processes a new backend response that contains lightOn: false, the differential update guard (lightOn != lastLightState) will correctly detect the discrepancy and toggle the relay back off. Without updating the tracker in the handler, the guard would think the light was already off and do nothing, leaving the relay in a state the backend did not command.

Table 5.2: Local HTTP server routes for direct relay control.

Route	Method	GPIO write	Response body	State tracker updated
/light/on	GET	RELAY_PIN → LOW	"status":"ok","light":"on"	lastLightState = true
/light/off	GET	RELAY_PIN → HIGH	"status":"ok","light":"off"	lastLightState = false
/fan/on	GET	RELAY_PIN2 → LOW	"fan":"on"	lastFanState = true
/fan/off	GET	RELAY_PIN2 → HIGH	"fan":"off"	lastFanState = false

These four routes are registered in setup() and polled on every loop() iteration via server.handleClient(). Because handleClient() is non-blocking — it processes at most one pending request per call — it does not interfere with the motion-detection and sensor-reading logic that runs in the same loop iteration. The server responds to requests

synchronously within the handler function, so no asynchronous callback or interrupt is needed.

5.3.2 Device Authentication — JWT Token Lifecycle

Every HTTP request the ESP32 makes to the .NET Core backend must carry a valid JSON Web Token issued specifically to that device. The firmware manages this token lifecycle through three cooperating functions: `handleTokenResponse()` parses a token-exchange response and stores the result; `refreshToken()` performs the actual credential exchange with the backend; and `ensureTokenValid()` is a lightweight guard called before every outgoing request to ensure the stored token is still usable.

```
1 bool handleTokenResponse(String response)
2 DynamicJsonDocument doc(512);
3 DeserializationError error = deserializeJson(doc, response);
4 if (error)
5 Serial.println("Token Parse Failed");
6 return false;
7
```

```
9 jwtToken = doc["token"].as<String>();
10 String expiresAt = doc["expiresAt"].as<String>();
12 Serial.println("Token: " + jwtToken);
13 Serial.println("ExpiresAt: " + expiresAt);
15 // Conservative assumed expiry: 50 minutes from now
16 // Real expiry is in expiresAt but we avoid ISO-8601 parsing on ESP32
17 tokenExpireMillis = millis() + (50UL * 60UL * 1000UL);
19 return jwtToken != "";
20
```

Listing 5.8: `handleTokenResponse()`: JWT extraction and conservative 50-minute assumed expiry

(`arduino_code.txt`, lines 104–124).

`handleTokenResponse()` is called by `refreshToken()` after a successful HTTP response. It allocates a 512-byte JSON document on the stack, parses the response body

into it, and extracts two fields: `token` (the JWT string to be used as a bearer credential) and `expiresAt` (an ISO-8601 timestamp string representing when the server considers the token expired).

The `expiresAt` string is read and printed to the serial monitor for diagnostic purposes but is not parsed into a comparable timestamp value.

The reason for not parsing `expiresAt` is pragmatic: the ESP32 runs FreeRTOS and has access to `millis()` (milliseconds since boot) but does not have a calibrated real-time clock. Converting an ISO-8601 string like "2026-06-04T09:00:00Z" into a `millis()` value would require knowing the current Unix timestamp, which would need an NTP synchronisation step that is not in the current build. Instead, `tokenExpireMillis` is set to `millis() + 3,000,000` (50 minutes in milliseconds), using unsigned long arithmetic with the `UL` suffix to prevent 32-bit overflow on the multiplication. This is deliberately shorter than the server's configured token lifetime, so the device refreshes early and never risks sending a request with a token that expired moments ago.

Security note. Although the device password appears in the firmware source as a plain string constant, it is transmitted over the local network in a JSON POST body — not sent as a query parameter or in a URL. The `/api/devices/token` endpoint should be restricted to the local subnet in a production deployment to prevent credential interception from outside the laboratory network.

```
1 bool ensureTokenValid()
2 // Refresh if token is missing (first boot or parse failure)
3 // or if millis() has passed the assumed expiry timestamp
4 if (jwtToken == "" || millis() > tokenExpireMillis)
5 Serial.println("Token expired or missing ->
```

5.3. *V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore*

Refreshing..."); 6 return refreshToken(); 7 8 return true; // token is present and within assumed lifetime 9

Listing 5.9: ensureTokenValid(): lazy refresh guard — zero network calls when token is fresh

(arduino_code.txt, lines 126–133).

ensureTokenValid() is called at the top of sendMotionRequest() before any HTTP activity. If the token is present and millis() has not yet passed tokenExpireMillis, the function returns immediately with no network activity — no HTTP connection is opened, no socket is allocated. This is the normal path on the vast majority of loop() iterations. Only when the token is missing or presumed expired does the function call refreshToken(), which takes hundreds of milliseconds due to the network round-trip.

The millis() overflow edge case deserves mention. millis() returns an unsigned

```
1 long on the ESP32, which overflows back to zero after  
   approximately 49.7 days of continu-
```

ous uptime. If tokenExpireMillis was set close to the overflow boundary and millis() subsequently overflows, the condition millis() > tokenExpireMillis will be false when it should be true — causing the firmware to use a possibly-expired token. For a laboratory that is powered off overnight (fewer than 49.7 continuous days), this edge case is irrelevant. For a permanently-powered deployment, this should be addressed by using elapsed-time arithmetic rather than an absolute timestamp.

```
1 bool refreshToken() 2 if (WiFi.status() != WL_CONNECTED) return false;  
4 HTTPClient http; 5 http.begin(TokenGen); 6 http.addHeader("Content-Type",  
"application/json");  
8 DynamicJsonDocument doc(128); 9 doc["labId"] = labId; 10 doc["deviceId"] =  
deviceId; 11 doc["password"] = Devicepassword; // credential sent in body, not URL  
13 String json; 14 serializeJson(doc, json);  
16 Serial.println("Token Request:"); 17 Serial.println(json);  
19 int code = http.POST(json); 20 if (code <= 0) 21 Serial.println("HTTP failed");  
22 http.end(); 23 return false;  
24  
26 String response = http.getString(); 27 Serial.println("Token response:"); 28  
Serial.println(response);  
30 http.end(); 31 return handleTokenResponse(response); 32
```

Listing 5.10: refreshToken(): full device-credential exchange with the /api/devices/token endpoint

(arduino_code.txt, lines 157–191).

refreshToken() opens a new HTTPClient connection to TokenGen, adds the Content-Type header, builds a 128-byte JSON document containing labId, deviceId, and the device password, serialises it to a String, and POSTs it. A return code of 0 or negative from http.POST() indicates a transport-level failure (TCP connection refused, DNS failure, timeout) and causes the function to return false without attempting to parse an empty response. A positive return code — including 401 Unauthorized or 500 Internal Server Error — indicates the server responded; the response body is retrieved and

passed to `handleTokenResponse()`. If `handleTokenResponse()` fails to find a non-empty token string, it returns false, and `refreshToken()` propagates that failure to its caller.

The code `<= 0` guard is important: on the ESP32, `HTTPClient` returns negative values for library-level errors (e.g., `HTTPC_ERROR_CONNECTION_REFUSED` = -1, `HTTPC_ERROR_SEND_HEAD` = -2). Without this guard, attempting to call `getString()` on a failed request would return an empty string and `handleTokenResponse()` would fail to parse it, achieving the same result but generating confusing debug output. The guard makes the failure path explicit.

5.3.3 Motion-Triggered Sensor Reporting — `sendMotionRequest()`

`sendMotionRequest()` is the firmware's primary communication function. It is called once per detected motion event and is responsible for packaging all five sensor readings into a JSON body, attaching the JWT, POSTing to the backend, handling a 401 response by refreshing the token and retrying, and returning the backend's response body for the relay-actuation logic in Section 5.2.6.

```
1 String sendMotionRequest(bool motion, long distance, int ldr, float temp, float
hum) 2 // Guard 1: ensure a valid JWT is cached before any HTTP activity 3 if
(!ensureTokenValid()) 4 Serial.println("Token invalid"); 5 return "";
```

```
6 8 // Guard 2: confirm Wi-Fi is still connected 9 if (WiFi.status() != WL_CONNECTED)
10 Serial.println("WiFi not connected"); 11 return ""; 12
```

```
14 HTTPClient http; 15 http.begin(serverUrl); 16 http.setTimeout(10000); //
10-second timeout for this request 17 http.addHeader("Authorization", "Bearer " +
String(jwtToken)); 18 http.addHeader("Content-Type", "application/json");
```

```
20 // Build JSON payload manually (avoids another 512-byte DynamicJsonDocu-
ment) 21 String json = ""; 22 json += "motionDetected:" + String(motion ? "true" :
"false") + ","; 23 json += "distanceCm:" + String(distance) + ","; 24 json += "ldrValue:"
+ String(ldr) + ","; 25 json += "temperatureCelsius:" + String(temp, 1) + ","; 26 json
+= "humidity:" + String(hum, 1); 27 json += "";
```

```
29 Serial.println("Sending JSON:"); 30 Serial.println(json);
```

```
32 int code = http.POST(json); 33 String response = "";
```

Listing 5.11: `sendMotionRequest()` Part A: guards, header setup, JSON body construction, and

initial POST (arduino_code.txt, lines 195–236).

The function begins with two pre-flight guards. The first calls `ensureTokenValid()`, which may itself call `refreshToken()` and incur a network round-trip if the token has expired. If token validation fails for any reason, the function returns an empty string immediately without attempting the motion POST — a fast-fail that prevents a request that is guaranteed to be rejected. The second guard checks `WiFi.status()` again even though `ensureTokenValid()` also checks it internally, because a race condition is theoretically possible: the Wi-Fi connection could drop between the token validation check and the HTTP POST attempt. In practice, on a stable local network this race is extremely unlikely, but the second guard makes the failure path explicit for diagnostics.

5.3. *V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore*

The JSON body is built using manual string concatenation rather than a `DynamicJsonDocument` because the structure is fixed and known at compile time: five named fields, no nesting, no arrays. Manual construction avoids allocating another JSON document on the heap and produces smaller, faster code for this specific case. The `String(temp, 1)` call serialises the float temperature to one decimal place, which is the precision the DHT11 can reliably provide. 1 // 401 Retry: if the backend rejects the token, refresh once and retry 2 if (code == 401) 3 `Serial.println("401 -> Refresh token");` 4 if (`refreshToken()`) 5 `http.end();` 6 `http.begin(serverUrl);` 7 `http.addHeader("Authorization", "Bearer " + String(jwtToken));` 8 `http.addHeader("Content-Type", "application/json");` 9 `code = http.POST(json);` 10 11

13 if (code > 0) 14 `response = http.getString();` 15 `Serial.println("Response: " + response);` 16 else 17 `Serial.println("HTTP Request Failed");` 18

20 `http.end();` 21 return response; 22

Listing 5.12: sendMotionRequest() Part B: 401 retry logic, response collection, and cleanup

(`arduino_code.txt`, lines 237–264).

The 401-retry block is the firmware’s implementation of self-healing authentication. If the backend returns HTTP 401, the firmware calls `refreshToken()` to obtain a new JWT, ends the current `HTTPClient` session (freeing its socket), opens a new session to the same URL with the new token in the Authorization header, and retries the POST with the same JSON body. The JSON body string is intentionally declared before the retry block so that it is available without reconstruction. This retry happens at most once per motion event — if the refreshed token is also rejected, the code falls through to the `code > 0` check, which will be false, and the function returns an empty string. A second retry is not attempted, to avoid an infinite authentication loop in the event of a backend misconfiguration.

The `http.end()` call in the retry block is critical: the ESP32’s HTTP client internally holds a TCP socket open after a request. Without calling `end()` before calling `begin()` again, a second connection to the same host would either reuse the existing socket (which is in an undefined state after a 401 response) or open a second socket and leak the first. Either outcome would eventually exhaust the ESP32’s limited socket pool.

5.3.4 Main Firmware Logic — `setup()` and `loop()`

`setup()` runs once on boot and performs the following initialisation steps in order: pin mode configuration, relay initialisation to safe-off state, serial monitor at 115200 baud, DHT sensor initialisation, Wi-Fi connection, initial token acquisition, HTTP route registration, and local web server start. `loop()` is called continuously by the Arduino runtime and handles Wi-Fi reconnection, local server request polling, sensor reading, motion-edge detection, and relay actuation. 1 void `setup()` 2 // Configure sensor and relay pins 3 `pinMode(TRIG_PIN, OUTPUT);` 4 `pinMode(ECHO_PIN, INPUT);` 5 `pinMode(RELAY_PIN, OUTPUT);` 6 `pinMode(RELAY_PIN2, OUTPUT);`

8 // Safe initial state: both relays open (light and fan OFF) 9 // HIGH = relay open on active-low relay module 10 `digitalWrite(RELAY_PIN, HIGH);` 11 `digital-`

```
Write(RELAY_PIN2, HIGH);
13 Serial.begin(115200); 14 dht.begin();
16 // Block until Wi-Fi connects (dot-print progress indicator) 17 WiFi.begin(ssid,
password); 18 while (WiFi.status() != WL_CONNECTED) 19 delay(500); 20 Serial.print("."); 21 22 Serial.println(""); 23 Serial.println(WiFi.localIP());
25 // Acquire initial token; log failure but do not halt 26 if (!refreshToken()) 27 Serial.println("Failed to get token at startup"); 28
30 // Register direct relay-control routes 31 server.on("/light/on", handleLightOn);
32 server.on("/light/off", handleLightOff); 33 server.on("/fan/on", handleFanOn); 34 server.on("/fan/off", handleFanOff); 35 server.begin();
37 Serial.println("ESP32 Server Started"); 38
```

Listing 5.13: setup(): full initialisation sequence from pin configuration to server start

(arduino_code.txt, lines 267–310).

Setting both relay pins HIGH immediately after pinMode() is the safe-state initialisation: it ensures that even if setup() takes several seconds (due to the Wi-Fi connection loop), the light and fan are off during that time and do not switch on transiently. The alternative — not writing HIGH until after Wi-Fi connects — would leave the relay pins in an

indeterminate state during the connection phase, which on some relay modules could cause a brief activation glitch.

The blocking while (WiFi.status() != WL_CONNECTED) loop is acceptable in setup() because the ESP32 has no useful work to do until it has network connectivity — all of its functionality depends on being able to reach the backend. In loop(), by contrast, the Wi-Fi reconnection logic is non-blocking (see below) to avoid stalling the local HTTP server.

Calling refreshToken() at the end of setup() pre-populates jwtToken so that the first motion event after boot does not need to wait for a token exchange before it can POST sensor data. If the token exchange fails at startup (e.g., the backend is not yet running), a warning is printed but setup() completes normally — the next call to ensureTokenValid() in sendMotionRequest() will retry the exchange.

```
1 void loop()
2 // — Sensor reads (every iteration) — 3 long distance = readDistanceCM(); 4 int
lightValue = readLight(); 5 float temp = dht.readTemperature(); 6 float humidity =
dht.readHumidity();
```

```
8 // DHT guard: abort iteration on sensor error 9 if (isnan(temp) || isnan(humidity))
10 Serial.println("Sensor error"); 11 return; 12
```

```
14 bool isNear = (distance > 0 && distance <= DISTANCE_THRESHOLD);
16 // — Wi-Fi watchdog (non-blocking) — 17 static unsigned long lastWifiTry =
0; 18 if (WiFi.status() != WL_CONNECTED) 19 if (millis() - lastWifiTry > 5000)
20 Serial.println("Reconnecting Wi-Fi..."); 21 WiFi.disconnect(); 22 WiFi.begin(ssid,
password); 23 lastWifiTry = millis(); 24
```

```
25 return; // skip network operations until Wi-Fi is back 26
```

```
28 // — Service local HTTP requests — 29 server.handleClient();
```

5.3. *V, and sensitivity to crosstalk from Wi-Fi radio activity. The ADC readings are therefore*

```
31 // — Motion edge detection with cooldown — 32 if (isNear && !wasNear &&
millis() - lastToggleTime > COOLDOWN) 33 Serial.println("Motion detected ->
Sending to API"); 34 String response = sendMotionRequest(isNear, distance, lightValue,
temp, humidity);
```

Listing 5.14: loop() Part A: sensor reads, NaN guard, Wi-Fi watchdog, server polling, motion edge

detection (arduino_code.txt, lines 312–357).

loop() begins with sensor reads on every iteration, even if no motion event will be reported. This ensures the sensor values are always fresh when needed. The bool isNear expression combines two conditions: distance > 0 guards against the -1 sentinel

```
1 returned by readDistanceCM() on timeout (which would
incorrectly satisfy distance <=
```

DISTANCE_THRESHOLD), and distance <= DISTANCE_THRESHOLD implements the actual 50 cm threshold. A value of -1 is not ≤ 50 , but because distance is declared as long (signed), -1 would satisfy $-1 \leq 50$ — hence the explicit > 0 guard.

The Wi-Fi watchdog uses a static local variable lastWifiTry, which is initialised once on the first loop() call and retains its value across iterations. Checking millis() - lastWifiTry > 5000 before calling WiFi.begin() again prevents the firmware from hammering the Wi-Fi stack with connection attempts every 100 ms (the delay at the bottom of loop()). Calling WiFi.disconnect() before WiFi.begin() clears any stale connection state, which can prevent the ESP32's internal Wi-Fi stack from getting stuck in an intermediate state after a drop. The return immediately after the watchdog block ensures that no network operations (sendMotionRequest, server.handleClient) are attempted

```
1 while the device is o f i n e .
```

```
1 // — Local fallback if backend unreachable — 2 bool apiFailed = (response ==
""); 3 if (apiFailed) 4 Serial.println("Fallback to local logic"); 5 static bool lastState =
false;
```

```
6 bool newState = (lightValue < LIGHT_THRESHOLD); 7 if (newState != last-
State) 8 digitalWrite(RELAY_PIN, newState ? LOW : HIGH); 9 lastState = newState;
10 11 return; 12
```

```
14 // — Parse backend JSON response — 15 DynamicJsonDocument doc(512);
16 DeserializationError error = deserializeJson(doc, response); 17 if (error) 18 Se-
rial.println("JSON Parse Failed"); 19 return; 20
```

```
22 bool lightOn = doc["lightOn"]; 23 bool fanOn = doc["fanOn"]; 24 bool relay3On
= doc["relay3On"]; 25 bool relay4On = doc["relay4On"]; 26 bool motionIgnored =
doc["motionIgnored"]; 27 int occupancy = doc["occupancyCount"];
```

Listing 5.15: loop() Part B: local fallback logic and backend JSON response parsing (arduino_code.txt, lines 358–396).

The fallback branch is triggered when sendMotionRequest() returns an empty string, meaning the POST failed at transport level (no Wi-Fi, TCP timeout, or token refresh failure). In this case, the firmware applies a single local rule: if lightValue

`< LIGHT_THRESHOLD`, the room is currently well-lit, so the light stays off; if `lightValue >= LIGHT_THRESHOLD`, the room is dark, so the light is turned on. A static `lastState` variable tracks the last locally-applied light state so the relay is only toggled when the LDR reading crosses the threshold, not on every loop iteration — the same differential-update pattern used for the backend-driven relay logic.

The fallback uses only the light relay (`RELAY_PIN`), not the fan relay. This is a deliberate conservative choice: the local LDR reading can justify a lighting decision (dark room $\rightarrow$ light on), but the firmware has no local logic for fan control without the backend's

temperature rule evaluation. Leaving the fan in its last known state (which was initialised to `HIGH`/off in `setup()`) is the safe-default behaviour for ventilation.

```
1 // — Room-empty override: force all relays off — 2 if (occupancy == 0) 3
Serial.println("Room Empty -> FORCE OFF"); 4 digitalWrite(RELAY_PIN, HIGH);
// light off 5 digitalWrite(RELAY_PIN2, HIGH); // fan off 6 lastFanState = false; 7
lastLightState = false; 8 lastToggleTime = millis(); 9 return; // skip API-driven relay
logic below 10
```

```
12 // — Relay actuation from API response — 13 if (lightOn != lastLightState) 14
digitalWrite(RELAY_PIN, lightOn ? LOW : HIGH); 15 lastLightState = lightOn; 16
Serial.println(lightOn ? "Light ON" : "Light OFF"); 17
```

```
19 if (fanOn != lastFanState) 20 digitalWrite(RELAY_PIN2, fanOn ? LOW :
HIGH); 21 lastFanState = fanOn; 22 Serial.println(fanOn ? "Fan ON" : "Fan OFF");
23
```

```
25 // Relay 3 and 4: parsed and logged – not yet wired to GPIO 26 if (relay3On)
Serial.println("Relay3 ON"); 27 else Serial.println("Relay3 OFF");
```

```
29 if (relay4On) Serial.println("Relay4 ON"); 30 else Serial.println("Relay4 OFF");
```

```
32 // motionIgnored: backend chose not to act on this event 33 if (motionIgnored)
34 Serial.println("Motion Ignored by Server"); 35 lastToggleTime = millis(); 36 return;
```

```
37 38 // end if (isNear && !wasNear && cooldown)
```

```
40 wasNear = isNear; 41 delay(100); 42 // end loop()
```

Listing 5.16: loop() Part C: room-empty override, differential relay actuation, motionIgnored

handling, and loop bookkeeping (`arduino_code.txt`, lines 397–445).

The room-empty override is evaluated before the per-relay differential logic. If `occupancyCount` is 0, the backend is asserting that the AI service has confirmed no people are present in the room. In this case, both relays are forced off unconditionally — even if `lightOn` or `fanOn` in the same response happen to be true (which should not occur from a correctly behaving backend, but the firmware does not assume correct backend behaviour). Both state trackers are reset to false and `lastToggleTime` is updated to enforce the cooldown before the next motion event can be processed. The return prevents the differential relay logic below

```
1 from being evaluated.
```

The differential relay actuation pattern — if (`lightOn != lastLightState`) — is the firmware's most important mechanism for relay longevity. Relay contacts have a finite

switching lifetime, typically rated for 100,000 operations for a 10A resistive load. If the firmware wrote to the relay pin on every motion event regardless of whether the state had changed, a busy laboratory could generate thousands of unnecessary relay operations per day. By comparing the commanded state to the last applied state and only toggling on a genuine change, the firmware reduces relay operations to the minimum required.

The `motionIgnored` flag allows the backend to acknowledge a sensor POST without commanding any relay change — for example, because the motion event occurred outside a scheduled session and the administrator has configured the system to ignore after-hours motion. When `motionIgnored` is true, `lastToggleTime` is updated to suppress further motion events during the cooldown period, and the function returns without logging any relay state.

Finally, `wasNear` is updated at the bottom of every `loop()` iteration (outside the motion-event block) to track the previous presence state for the next rising-edge detection.

The `delay(100)` at the very end paces the loop at approximately 10 Hz, giving the DHT11 sensor sufficient time between reads to avoid NaN responses while keeping the ultrasonic presence detection responsive enough for human entry events.

5.4 Python AI Processing Service Implementation

The Python AI processing service is the second tier of the GreenLab architecture. It is responsible for consuming a live video stream from a camera installed in each laboratory, running the YOLO v11 object detection model on that stream, applying ByteTrack identity tracking so that the same person is not counted multiple times across frames, and exposing the resulting occupancy count via a FastAPI HTTP endpoint that the .NET Core backend can poll.

The service was implemented in two stages. The first stage — a working prototype in `cout_code.txt` — established the core architecture: a single shared YOLO model, one background thread per camera, a `threading.Event` for readiness signalling, and a FastAPI endpoint. The second stage — the production service in `droidip.py`, supported by the `camera_sources.py` abstraction — added multi-format camera input, ByteTrack tracking, a typed `CameraState` dataclass, and a background cleanup worker for idle cameras. This section presents all three files in their entirety, function by function.

5.4.1 Stage 1: Single-Camera Prototype (`cout_code.txt`)

`cout_code.txt` is 116 lines long and contains four functional components: import declarations, a shared YOLO model loader, a per-camera detection loop, a thread-start helper, and the FastAPI endpoint. Each is presented and explained in full below.

5.3.1.1 Import Declarations and Application Bootstrap

```
1 import threading
2 import time
3 from ultralytics import YOLO
4 import cv2
5 from fastapi import FastAPI, HTTPException
```

```
6 import uvicorn
8 app = FastAPI()
```

Listing 5.17: Module-level imports and FastAPI application object (cout_code.txt, lines 1–8).

Six imports cover the full dependency surface of the prototype. `threading` provides `Thread`, `Lock`, and `Event` — the three primitives used to make the model and camera state safely shareable across concurrent requests. `time` provides `time.sleep()`, used in the reconnection backoff. `ultralytics` provides the YOLO class, which encapsulates model loading, preprocessing, inference, and post-processing in a single object. `cv2` (OpenCV) provides `VideoCapture` for camera access and `resize` for frame downscaling. `fastapi` and `HTTPException` provide the REST endpoint infrastructure. `uvicorn` is the ASGI server that hosts the FastAPI application when the script is run directly.

5.3.1.2 Shared YOLO Model — Lazy Singleton with Double-Checked Locking

```
1 # Shared model – loaded ONCE
2 _model = None
3 _model_lock = threading.Lock()

5 def get_model():
6     global _model
7     if _model is None:
8         # First check (no lock – fast path)
9         with _model_lock:
10             # Acquire lock only when needed
11             if _model is None:
12                 # Second check (inside lock – safe)
13                 _model = YOLO("yolo11n.pt")
14             return _model
```

Listing 5.18: get_model(): double-checked locking singleton for YOLO model (cout_code.txt, lines 11–20).

The double-checked locking pattern is used here for a specific reason: loading a YOLO model from disk takes 2–5 seconds and allocates several hundred megabytes of memory. If two camera threads start simultaneously (e.g., two requests arrive at the `/current-count` endpoint before either thread has loaded the model), the naïve solution of locking and

checking only once inside the lock would still be correct — both threads would acquire the lock sequentially, one loads the model, the other finds `_model` already set. The double-checked pattern adds the outer `if _model is None` check before acquiring the lock, so that all subsequent calls after the first return immediately without locking — a significant performance gain for a lock-heavy concurrent server.

In Python, the GIL (Global Interpreter Lock) provides an additional safety guarantee: the assignment `_model = YOLO(...)` is atomic from the GIL’s perspective, so the outer check cannot observe a partially-initialised object. The double-checked pattern is therefore both correct and efficient in CPython.

`_model.to("cpu")` explicitly places the model on the CPU rather than a GPU. This is appropriate for a deployment on a laptop or server without a CUDA-capable GPU. If the deployment hardware has a GPU, changing `"cpu"` to `"cuda"` would accelerate inference by an order of magnitude, but `yolo11n` (the nano variant) is already fast enough on CPU for the sampling cadence this service uses.

5.3.1.3 Camera State Registry

```
1 camera_states = {}
2 camera_lock = threading.Lock()
```

Listing 5.19: Global camera state dictionary and its protecting lock (cout_code.txt, lines 22–23).

camera_states is a plain Python dictionary keyed by integer device index. Each value is itself a dictionary with keys "count", "ready" (a threading.Event), and "thread" (the running Thread object). camera_lock is a threading.Lock that must be held when- ever camera_states is read or written from any thread, to prevent concurrent dictionary modifications from corrupting its internal state — Python dictionaries are not thread-safe for concurrent reads and writes even though individual read operations are GIL-protected.

5.3.1.4 Per-Camera Detection Loop — camera_loop()

```
1 def camera_loop(device_index: int, ready_event: threading.Event): 2 model =
get_model() # shared reference – no lock needed here
```

```
4 while True: # outer reconnection loop
```

```
5 # Primary open attempt: no backend hint 6 cap = cv2.VideoCapture(device_index)
7 if not cap.isOpened(): 8 cap.release() 9 # Fallback: DirectShow backend (Windows
virtual cameras) 10 cap = cv2.VideoCapture(device_index, cv2.CAP_DSHOW)
```

```
12 # Tune buffer to reduce latency for virtual cameras 13 cap.set(cv2.CAP_PROP_BUFFERSIZE,
2) 14 cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640) 15 cap.set(cv2.CAP_PROP_FRAME_HEIG
480)
```

```
17 if not cap.isOpened(): 18 print(f"[ERROR] Cannot open camera index de-
vice_index, retrying in 3s ...") 19 time.sleep(3) 20 continue # retry outer loop
```

```
22 print(f"[INFO] Connected to camera index: device_index")
```

Listing 5.20: camera_loop() Part A: reconnection loop, two-attempt open strategy, and buffer

configuration (cout_code.txt, lines 26–50).

camera_loop() calls get_model() at the top, which returns the already-loaded shared model on all subsequent calls after the first. The returned model object is used in the inner loop for every inference call without any additional locking, which is safe because YOLO model inference is a read-only operation on the model's weights — the model object is never modified during inference.

The two-attempt camera open strategy addresses a specific quirk of virtual camera software on Windows: DroidCam (which turns an Android phone into a USB webcam) and its alternatives sometimes only enumerate correctly with the DirectShow backend (cv2.CAP_DSHOW) rather than OpenCV's default backend. The primary attempt without a backend hint works on Linux and macOS; the CAP_DSHOW fallback is the Windows safety net. Calling cap.release() between attempts is essential to free the file descriptor before attempting a new open.

Setting CAP_PROP_BUFFERSIZE to 2 reduces the number of frames OpenCV buffers internally from the default (typically 5–10 for USB cameras) to 2. A smaller buffer means

cap.read() returns a more recent frame — important for occupancy detection where we want the current count, not the count from three seconds ago. At 640×480 and 30 fps, two buffered frames represent approximately 67 ms of latency, which is acceptable.

```
1 while True: # inner frame-reading loop 2 ret, frame = cap.read() 3 if not ret: 4
```

```

print(f"[WARN] Lost connection on cam device_index, reconnecting ...")
5 break # break inner reconnect outer ->
7 # Downsample to 320x320 for faster inference
8 small_frame = cv2.resize(frame, (320, 320))
9 results = model(small_frame, verbose=False, imgsz=320)
11 # Count class-0 (person) detections above confidence threshold
12 person_count = sum(
13     14 for r in results
15     15 if r.boxes is not None
16     16 for box in r.boxes
17     17 if int(box.cls[0]) == 0 and float(box.conf[0]) > 0.5
18 )
20 with camera_lock:
21     camera_states[device_index]["count"] = person_count
23 # Signal ready after first successful detection pass
24 if not ready_event.is_set():
25     ready_event.set()
27 cap.release()
28 time.sleep(1) # brief pause before reconnect

```

Listing 5.21: camera_loop() Part B: frame reading, YOLO inference, person counting, and state

update (cout_code.txt, lines 51–76).

The inner frame loop reads frames continuously and runs inference on every frame. `cv2.resize(frame, (320, 320))` downsamples the 640×480 frame (or whatever the

camera delivers) to 320×320 pixels before inference. This is a deliberate accuracy/speed trade-off: `yolo11n` was trained on 640×640 images and performs best at its native resolution, but at 320×320 it still reliably detects persons at distances of 1–5 metres (typical for a laboratory doorway sensor) while running approximately four times faster than at 640×640 on CPU.

The person counting generator expression is a nested comprehension that iterates over all Results objects returned by `model()` (one per image, so effectively one object here), then over all detected bounding boxes in that result, and yields 1 for each box where the

```

1 class index (box.cls[0]) is 0 (the COCO class index for ‘‘
    person’’ ) and the c o n d e n c e

```

score (`box.conf[0]`) exceeds 0.5. The `sum()` of those 1s is the frame-level person count.

The `ready_event.set()` call signals the FastAPI endpoint that at least one frame has been successfully processed. The endpoint blocks on `ready_event.wait()` until this signal is received, preventing it from returning a stale zero count on the very first request to a freshly started camera. The `is_set()` check before calling `set()` avoids a redundant syscall on every frame after the first.

5.3.1.5 Thread-Start Helper — `_start_camera()`

```

1 def _start_camera(cam_id: int):
2     """Start camera thread only if not already
3     running. Must be called with camera_lock held."""
4     if cam_id not in camera_states:
5         ready_event = threading.Event()
6         camera_states[cam_id] = {"count": 0, "ready":
7         ready_event, "thread": None}
8         t = threading.Thread(
9             10 target=camera_loop,
11             args=(cam_id, ready_event),
12             daemon=True
13         )
14         t.start()
15         camera_states[cam_id]["thread"] = t

```

Listing 5.22: _start_camera(): guarded thread creation, must be called with camera_lock held

(`cout_code.txt`, lines 79–93).

`_start_camera()` is a private helper that must be called with `camera_lock` already held — its docstring makes this contract explicit. The `cam_id` not in `camera_states` guard prevents starting a second thread for a camera that is already running, which is the primary race condition this helper needs to prevent: if two HTTP requests for the same `cam_id` arrive simultaneously, both will pass the outer `get_count()` check, but only the first to acquire `camera_lock` and call `_start_camera()` will find `cam_id` not in `camera_states`; the second will find it already present and do nothing.

Setting `daemon=True` on the thread means Python will not wait for the camera loop to finish before the main process exits. This is the correct choice for a background service thread: if the FastAPI server shuts down (SIGTERM, Ctrl+C), the camera threads should not block the shutdown process.

5.3.1.6 FastAPI Endpoint — `/current-count`

```
1 @app.get("/current-count") 2 def get_count(cam_id: int = 0): 3 """ 4 Pass the
camera index. 5 Usually 0 is your built-in webcam, 1 is DroidCam/DropCam USB. 6
""" 7 with camera_lock: 8 if cam_id not in camera_states: 9 _start_camera(cam_id)
# Safe: lock is already held 10 ready_event = camera_states[cam_id]["ready"]
12 # Wait OUTSIDE the lock – do not hold it for up to 10 seconds 13 if not
ready_event.wait(timeout=10): 14 raise HTTPException( 15 status_code=503, 16
detail=f"Camera cam_id not responding." 17 f" Check your DropCam/DroidCam
connection.", 18 )
20 with camera_lock: 21 state = camera_states.get(cam_id, "count": 0)
23 return "people_count": state["count"], "camera_index": cam_id
26 if __name__ == "__main__": 27 uvicorn.run(app, host="0.0.0.0", port=8000)
```

Listing 5.23: `/current-count` endpoint: lazy thread start, ready-event blocking (10 s timeout), and

count retrieval (`cout_code.txt`, lines 95–116).

The endpoint acquires `camera_lock`, checks whether the requested `cam_id` has a running thread, and if not calls `_start_camera()` (which expects the lock to be held). It then extracts the `ready_event` reference from the state dictionary and immediately releases the lock. The release is critical: `ready_event.wait(timeout=10)` may block for up to 10 seconds, and holding `camera_lock` for 10 seconds would prevent any other HTTP request from being processed during that time — a deadlock-adjacent scenario that would make the service effectively single-threaded under load.

`ready_event.wait()` returns True if the event was set within the timeout, False if the timeout expired without the event being set. A False return means the camera thread started but did not produce a single successful frame within 10 seconds — either the camera is not connected, DroidCam is not running on the phone, or the camera index is wrong. The 503 Service Unavailable response with a human-readable detail message allows the backend to present this as a camera fault rather than an application error.

The second `camera_lock` acquisition for the final count read is a separate critical section from the first. Between the end of the first critical section and the start of the second, the camera thread may have updated `camera_states[cam_id]["count"]` any number of times — this is acceptable, as the endpoint simply returns whatever count is

current at the moment it reads it.

5.4.2 Camera Source Abstraction Module (`camera_sources.py`)

`camera_sources.py` is 189 lines long. It is a pure-utility module with no FastAPI dependency and no global state — it consists entirely of functions that translate camera source strings into values `cv2.VideoCapture` can open. It was extracted from `droidip.py` to make the source-resolution logic independently testable and reusable. The module defines

one regex constant, one frozenset constant, three public functions, and two private helper functions.

5.3.2.1 Module-Level Constants

```
1 import logging 2 import os 3 import re 4 from typing import Union
6 import cv2
8 logger = logging.getLogger("people_counter.camera_sources")
10 # Supported video file extensions 11 _VIDEO_EXTENSIONS: frozenset[str] =
frozenset( 12 ".mp4", ".avi", ".mov", ".mkv", ".flv", ".ts" 13 )
15 # Pattern that matches a bare IPv4 address (no scheme, no path) 16 _BARE_IP_RE
= re.compile( 17 r"(1,3)31,3" 18 )
```

Listing 5.24: `camera_sources.py`: imports, logger, supported file extensions frozenset, and bare-IP

regex (`camera_sources.py`, lines 1–17).

`_VIDEO_EXTENSIONS` is a frozenset (immutable set) of lowercase file extensions. Using frozenset rather than a list makes membership testing $O(1)$ rather than $O(n)$, which matters when `resolve_source()` is called on every incoming API request. The extensions cover the most common container formats that OpenCV can decode: `.mp4` (H.264/H.265 in MP4), `.avi` (various codecs in AVI), `.mov` (QuickTime), `.mkv` (Matroska), `.flv` (Flash Video), and `.ts` (MPEG Transport Stream).

`_BARE_IP_RE` is a compiled regular expression that matches exactly a dotted-quad IPv4 address with no scheme, port, or path. The pattern requires: start of string, three groups of 1–3 digits followed by a literal dot, one final group of 1–3 digits, end of string. It does not validate that each octet is in the range 0–255 — 999.999.999.999 would match

— but for a camera IP address entered by an administrator this is an acceptable trade-off. A stricter regex would be longer and harder to read without providing meaningful additional protection, since an invalid IP would simply fail to connect.

5.3.2.2 `resolve_source()` — The Core Dispatch Function

```
1 def resolve_source(source: str, default_port: int = 4747) -> Union[str, int]: 2
""" 3 Translate a caller-supplied source string into the value that 4 cv2.VideoCapture
expects: an integer for USB devices, a string 5 for everything else.
```

```
7 Parameters 8 ——— 9 source : Raw source string from the API caller. 10
default_port : Port appended when source is a bare IP address.
```

```
12 Returns 13 ——— 14 int for USB / device-index sources. str for network streams
and file paths.
```

```

16 Raises 17 — 18 ValueError if the source format is not recognised. 19 """ 20
source = source.strip() 21 if not source: 22 raise ValueError("Camera source must not
be empty")
24 if source.lower().startswith("usb:"): 25 return __parse_usb(source)
27 if source.startswith(("http://", "https://", "rtsp://")): 28 logger.debug("Network
stream source: 29 return source
31 ext = os.path.splitext(source)[1].lower() 32 if ext in _VIDEO_EXTENSIONS:
33 return __parse_video_file(source)
35 if _BARE_IP_RE.match(source): 36 url = f"http://{source:default_port}/video"
37 logger.debug("Bare IP expanded to: 38 return url
40 raise ValueError( 41 f"Unrecognised camera source: 'source'. " 42 "Expected
usb:<index>, http/https/rtsp URL, " 43 "a video file (.mp4/.avi/.mov), or a bare IPv4
address." 44 )

```

Listing 5.25: resolve_source(): five-branch dispatch with full docstring (camera_sources.py, lines

20–70).

resolve_source() applies its five recognition patterns in a strict priority order that prevents ambiguity: the usb: prefix check comes first, before any other pattern, so that a hypothetically unusual string like "usb:192.168.1.1" is handled as a USB source request rather than being misidentified as an IP address. Network scheme checks come second; these are the most specific and most common runtime inputs. The video file extension check comes third; it only fires if the source does not have a network scheme but does have a recognised extension. The bare-IP regex comes fourth; it is the backward-compatibility catch-all for DroidCam-style inputs. The ValueError at the end is the catch-all for inputs that match none of the above.

The source.strip() at the top removes leading and trailing whitespace, guarding against inputs like " 192.168.1.3 " pasted from a text field with accidental spaces. The empty-string check after strip() catches inputs that were whitespace-only — without it, a whitespace-only string would pass the strip() and then fall through all five pattern checks to the ValueError, but the error message would say “Unrecognised camera source: ” rather than the more informative “Camera source must not be empty”.

5.3.2.3 create_capture() — Opening the VideoCapture

```

1 def create_capture(source: str, default_port: int = 4747) -> cv2.VideoCapture:
2 """

```

Table 5.3: Example inputs to resolve_source() and their resolved output.

Input	example	Branch	taken	Output
usb:0	USB prefix	int: 0	usb:2	USB pre-
fix int: 2	Network	http://192.168.1.3:4747/video	str: unchanged	scheme Network
rtsp://admin:pass@10.0.0.5/live	str: unchanged	scheme /recordings/lab_d7.mp4	File ex-	
extension str: unchanged	(.mp4)	192.168.1.3	Bare IP regex	str: http://192.168.1.3:4747/video
192.168.1.3	None	(Val- ValueError: Unrecognised port=8080	ueError)	source ftp://camera/stream
None	(Val- ValueError: Unrecognised	ueError)	source	

```

3 Parse source and return an opened cv2.VideoCapture. 4 Callers should check
cap.isOpened() themselves. 5 """ 6 resolved = resolve_source(source, default_port)

```

```

8 if isinstance(resolved, int): 9 logger.info("Opening USB device index 10 cap

```

```
= cv2.VideoCapture(resolved) 11 else: 12 logger.info("Opening stream: 13 cap =  
cv2.VideoCapture(resolved)
```

```
15 if not cap.isOpened(): 16 logger.warning( 17 "VideoCapture did not open imme-  
diately for source: 18 )
```

```
20 return cap
```

Listing 5.26: create_capture(): resolves source and returns VideoCapture without asserting success

(camera_sources.py, lines 72–100).

create_capture() intentionally does not raise an exception when the VideoCapture fails to open. The reason is that different callers have different retry and logging strategies: the production camera loop in droidip.py logs a warning and sleeps 3 seconds before retrying; a test harness might want to assert immediately. By returning the unopened VideoCapture object and leaving the isOpened() check to the caller, create_capture() is maximally composable. The warning log inside create_capture() provides visibility without forcing a particular error-handling strategy.

The isinstance(resolved, int) branch exists because cv2.VideoCapture(0) and cv2.VideoCapture("http://...") call the same constructor with different types. Although Python would handle both correctly without the branch (since VideoCapture accepts both int and str), the explicit branch allows the logger to emit a more informative message — “Opening USB device index 0” versus “Opening stream: http://...” — which is valuable for diagnosing connectivity problems in a deployed system.

5.3.2.4 source_label() — Stable Registry Key Generation

```
1 def source_label(source: str, default_port: int = 4747) -> str: 2 """ 3 Return  
a human-readable, stable label for source suitable for use 4 as a registry key. For  
network sources this is the fully-expanded 5 URL; for USB and file sources it is the  
canonical string. 6 """ 7 try: 8 resolved = resolve_source(source, default_port) 9 return  
str(resolved) # int -> "0", str -> unchanged 10 except ValueError: 11 return source #  
fall back to raw string if unresolvable
```

Listing 5.27: source_label(): stable key generation for the camera registry (camera_sources.py,

lines 102–115).

source_label() is used by droidip.py as the key for its _cameras registry dictionary. Using the fully-resolved URL as the key ensures that 192.168.1.3 and http://192.168.1.3:4747 — two different input strings that refer to the same physical camera — resolve to the same key, preventing accidental double-registration of the same camera under two different input formats. str(resolved) converts int device indices like 0 to the string "0", so all registry keys are strings regardless of input type. The except ValueError fallback returns the raw source string, which would then be stored as-is in the registry — a pragmatic choice

that allows the registry to absorb inputs that will ultimately fail at the VideoCapture level without crashing the key-generation step.

5.3.2.5 Private Helpers — _parse_usb() and _parse_video_file()

```
1 def _parse_usb(source: str) -> int: 2 """Parse 'usb:<index>' and return the integer  
device index.""" 3 __, __, index_str = source.partition(":") 4 index_str = index_str.strip()
```



```

6 if not index_str.lstrip("-").isdigit(): 7 raise ValueError( 8 f"Invalid USB source
'source': index must be a non-negative integer " 9 f"(e.g. usb:0, usb:1)." 10 )
12 index = int(index_str) 13 if index < 0: 14 raise ValueError( 15 f"Invalid USB
source 'source': device index must be >= 0, got index. " 16 )
18 logger.debug("USB device index: 19 return index
22 def __parse_video_file(source: str) -> str: 23 """ 24 Validate that source is a
readable file path and return it unchanged. 25 Only warns (does not raise) when the file
does not exist. 26 """ 27 if not os.path.isfile(source): 28 logger.warning( 29 "Video file
not found at '30 "Make sure the path is accessible from the server's working directory.",
31 source, 32 ) 33 else:
34 logger.debug("Video file source:
36 return source

```

Listing 5.28: `__parse_usb()` and `__parse_video_file()`: type-specific validation helpers (camera_sources.py, lines 117–189).

`__parse_usb()` uses `str.partition(":")` rather than `str.split(":")` to correctly handle hypothetical inputs like `"usb:0:extra"` — `partition` always returns exactly three parts (before the first delimiter, the delimiter, and everything after), so `index_str` would be `"0:extra"` in that case, which would then fail the `isdigit()` check and raise a `ValueError` with a clear message. The `lstrip("-")` before `isdigit()` allows the `isdigit()` check to see past a leading minus sign — so `"usb:-1"` would pass `isdigit()` on `"-1".lstrip("-") = "1"`, but then fail the `index < 0` check with a more specific error message than a generic `isdigit()` failure.

`__parse_video_file()` does not raise on a missing file by design: not raising, and just logging a warning, reflects the fact that the working directory at validation time may differ from the working directory at inference time. A Docker container, for example, might mount a recordings volume at `/data/recordings` at runtime, which does not exist on the host filesystem where validation runs. Treating a missing file as a warning rather than an error allows the service to accept a file path configuration even before the volume is mounted.

5.4.3 Stage 2: Production Multi-Camera Tracking API (droidip.py)

`droidip.py` is 322 lines long and is the service that the Chapter 3 architecture diagrams describe as the AI Processing tier. It builds on every pattern established in the prototype but replaces integer device-index inputs with source-string resolution, replaces raw per-frame box counting with ByteTrack-based identity tracking, replaces the loose dictionary state with a typed `CameraState` dataclass, and adds a background cleanup worker that removes idle cameras from the registry.

5.3.3.1 Module Docstring and Configuration Constants

```

1 """ 2 People Counter API 3 ===== 4 FastAPI service
that streams YOLO-tracked person counts from IP cameras.

```

```

6 Supported stream formats: 7 - DroidCam: /current-count?ip=192.168.1.3 8 -
HTTP: /current-count?ip=http://192.168.1.3:4747/video 9 - RTSP: /current-count?ip=rtsp://admin:p
10 - IP+port: /current-count?ip=192.168.1.3&port=8080 11 """

```

```
13 MODEL_PATH: str = "yolo11n.pt" 14 MODEL_DEVICE: str = "cpu" 15 CONFIDENCE_THRESHOLD: float = 0.5 16 FRAME_SKIP: int = 3 17 READY_TIMEOUT: float = 15.0 18 CAMERA_TIMEOUT: float = 300.0 19 CLEANUP_INTERVAL: float = 60.0 20 DEFAULT_PORT: int = 4747
```

Listing 5.29: Module docstring and all configuration constants (droidip.py, lines 1–28).

The module docstring lists all four supported input formats for the ip query parameter, making the service’s capabilities self-documenting without requiring a separate README. Each constant is annotated with its type for static analysis tool compatibility. The values reflect the operational requirements: CONFIDENCE_THRESHOLD = 0.5 satisfies FR-OD-01’s minimum detection confidence; FRAME_SKIP = 3 balances inference frequency against CPU load; READY_TIMEOUT = 15.0 gives a freshly-started camera 15 seconds to deliver its first frame before returning a 503 (extended from the prototype’s 10 seconds to account for RTSP cameras that are slower to negotiate stream parameters); CAMERA_TIMEOUT =

5.5 allows a camera that has not been polled for five minutes to be garbage-collected;

CLEANUP_INTERVAL = 60.0 runs the cleanup worker every minute; DEFAULT_PORT = 4747 is the DroidCam default port.

5.3.3.2 Structured Logging Configuration

```
1 import logging
3 logging.basicConfig( 4 level=logging.INFO, 5 format="%6 datefmt="%7 ) 8 logger = logging.getLogger("people_counter")
```

Listing 5.30: Structured logging setup with timestamp, level, and logger-name format (droidip.py, lines 30–40).

The logging configuration adds a timestamp, log level, and logger name to every message. This is a significant improvement over the prototype’s print() statements: print() output is unstructured and interleaved with other output in a multi-threaded environment; logging is thread-safe (Python’s logging module uses an internal lock) and each message includes enough context to diagnose multi-camera scenarios where several threads are producing output simultaneously. The format string produces output like “2026-06-04 09:15:32 [INFO] people_counter: Connected to stream: http://192.168.1.3:4747/vide which is immediately grep-able by stream URL or log level in a deployed environment.

camera_sources.py uses a child logger named "people_counter.camera_sources", which is a child of the "people_counter" logger configured here. Python’s logging hierarchy means that messages from the child logger propagate to the parent by default, so a single basicConfig() call covers all modules in the service.

5.3.3.3 YOLO Model Singleton — Production Version

```
1 _model: Optional[YOLO] = None 2 _model_lock = threading.Lock()
```

5.5. *allows a camera that has not been polled for five minutes to be garbage-collected;*

```
4 def get_model() -> YOLO: 5 """Return the shared YOLO model, loading it on
first call.""" 6 global _model 7 if _model is None: 8 with _model_lock:
9 if _model is None: 10 logger.info("Loading YOLO model: ") 11 _model =
YOLO(MODEL_PATH) 12 _model.to(MODEL_DEVICE) 13 logger.info("Model
loaded successfully") 14 return _model
```

Listing 5.31: Production get_model(): same double-checked locking pattern as the prototype, with

structured logging (droidip.py, lines 48–62).

The production get_model() is structurally identical to the prototype version but replaces print() with logger.info() and uses the MODEL_PATH and MODEL_DEVICE constants rather than hard-coded strings. The Optional[YOLO] type annotation on _model enables static type checkers to verify that all code paths that use _model account for the possibility that it is None before the first call to get_model().

5.3.3.4 CameraState Dataclass and Registry

```
1 from dataclasses import dataclass, field
3 @dataclass 4 class CameraState: 5 count: int = 0 6 connected: bool = False 7
last_access: float = field(default_factory=time.monotonic) 8 ready_event: threading.
Event = field(default_factory=threading.Event)
10 # registry: stream_url -> CameraState 11 _cameras: dict[str, CameraState] =
12 _cameras_lock = threading.Lock()
```

Listing 5.32: CameraState dataclass with field() defaults and the camera registry (droidip.py, lines

64–78).

Replacing the prototype’s loose dictionary ("count": 0, "ready": event, "thread": t) with a typed dataclass provides three benefits. First, field names are validated at

```
1 class definition time rather than at runtime --- a typo like
state["connt"] would be
```

caught by a type checker or raise a KeyError immediately, whereas setattr(state, "connt", value) in the production __update_state() helper would silently create a new attribute. Second, the @dataclass decorator auto-generates __init__, __repr__, and __eq__ methods, which are useful for testing and debugging. Third, the field() function with default_factory=time.monotonic means that last_access is set to the current monotonic clock value at the moment the CameraState is instantiated — if a simple default value like 0.0 were used instead, all newly-created cameras would have the same last_access, potentially causing the cleanup worker to remove them immediately after creation if cleanup runs before the first request.

time.monotonic() is used rather than time.time() because monotonic() is guaranteed never to go backwards — system clock adjustments (NTP sync, daylight saving time transitions) cannot cause last_access to jump forward and falsely make a recently-accessed camera appear stale.

5.3.3.5 Thread-Safe State Update Helper — __update_state()

```
1 def __update_state(url: str, **kwargs) -> None: 2 """Thread-safe helper to patch
fields on an existing CameraState.""" 3 with _cameras_lock: 4 state = _cameras.get(url)
```

```
5 if state is None: 6 return # camera was removed by cleanup – silently ignore 7 for
key, value in kwargs.items(): 8 setattr(state, key, value)
```

Listing 5.33: `_update_state()`: keyword-argument-based field patcher with cleanup-race guard

(`droidip.py`, lines 80–90).

`_update_state()` is a convenience function that takes keyword arguments and applies them to the `CameraState` for a given URL, all within a single lock acquisition. Without this helper, every state update in the camera loop would require three lines: acquire lock, set field, release lock. The `**kwargs` variadic signature allows updating multiple fields in a single call: `_update_state(url, connected=False)` or `_update_state(url, connected=True, count=0)`. The state is `None` guard handles the race condition where the cleanup worker removes the camera from the registry between the camera loop’s

registration check and the update call — rather than raising a `KeyError` or `AttributeError`, it silently returns, allowing the camera loop to detect the deregistration on its next iteration.

5.3.3.6 Production Camera Worker Loop — `_camera_loop()`

```
1 def _camera_loop(stream_url: str) -> None: 2 """ 3 Long-running thread: opens
the stream, runs YOLO ByteTrack inference, 4 and updates the shared CameraState.
Reconnects on failure. 5 """ 6 model = get_model()
```

```
8 while True: # outer reconnection loop 9 # Check deregistration before each
reconnect attempt 10 with _cameras_lock: 11 if stream_url not in _cameras: 12
logger.info("Camera deregistered, worker exiting: 13 return
```

```
15 cap = create_capture(stream_url) 16 if not cap.isOpened(): 17 logger.warning("Cannot
open stream, retrying in 3 s: 18 _update_state(stream_url, connected=False) 19
time.sleep(3) 20 continue
```

```
22 logger.info("Connected to stream: 23 _update_state(stream_url, connected=True)
24 frame_idx = 0
```

Listing 5.34: `_camera_loop()` Part A: outer reconnection loop with deregistration check (`droidip.py`, lines 93–120).

The outer reconnection loop begins with a deregistration check: if the cleanup worker has removed `stream_url` from `_cameras` between the camera thread starting and it attempting to open the stream, the thread exits cleanly without attempting a connection. This is important for the correctness of the cleanup mechanism — without this check, a deregistered camera’s thread would continue running indefinitely, consuming CPU and camera

resources even though no endpoint is polling its count.

`create_capture()` from `camera_sources.py` abstracts the actual `VideoCapture` construction. The camera loop does not need to know whether `stream_url` is a USB index, an RTSP URL, or a DroidCam HTTP stream — `create_capture()` handles all cases. The `connected=False` state update before the 3-second sleep allows the /current-count endpoint to report “connected: false” in its response, giving the caller a machine-readable indicator that the camera is temporarily unavailable.

```
1 while True: # inner frame-reading loop 2 # Check deregistration before each frame
```

5.5. allows a camera that has not been polled for five minutes to be garbage-collected;

```
read 3 with __cameras__lock: 4 if stream_url not in __cameras: 5 logger.info("Camera
deregistered mid-loop, exiting: stream_url) 6 cap.release() 7 return
    9 ret, frame = cap.read() 10 if not ret: 11 logger.warning("Stream lost, reconnecting:
12 __update_state(stream_url, connected=False) 13 break # break inner -> reconnect
outer
    15 frame_idx += 1 16 if frame_idx 17 continue # discard this frame without
inference
    19 try: 20 people_count = __run_inference(model, frame) 21 except Exception: 22
logger.exception("Inference error on stream: 23 continue
    25 with __cameras__lock: 26 state = __cameras.get(stream_url) 27 if state is None:
28 cap.release() 29 return
    30 state.count = people_count 31 state.ready_event.set()
    33 cap.release() 34 time.sleep(1)
```

Listing 5.35: __camera_loop() Part B: inner frame loop with FRAME_SKIP, exception guard, and

deregistration-aware state update (droidip.py, lines 121–160).

The inner frame loop has a second deregistration check, inside the loop itself before each frame read. This ensures that a camera deregistered while the loop is running causes the thread to exit promptly — without this check, the thread might read and process hundreds of frames between cleanup events before the outer loop’s check runs. `cap.release()` is called before returning to free the VideoCapture’s file descriptor and any OS resources associated with the video stream.

`FRAME_SKIP = 3` means inference runs on frames 3, 6, 9, 12, . . . — every third frame. At a typical USB camera or DroidCam frame rate of 30 fps, this produces 10 inference runs per second. At a slower 15 fps (some RTSP cameras), this produces 5 inference runs per second. Both are well above the effective 0.2 runs per second that a five-second sampling interval would imply. The `frame_idx` counter is a local variable initialised to 0 when the inner loop starts (after a successful open); it resets to 0 if the connection drops and the outer loop reconnects, which is correct because the skipped-frame count should restart from a fresh connection.

The try/except around `__run_inference()` prevents a single frame’s inference error (for example, a malformed frame from a corrupted RTSP packet) from crashing the entire camera thread. `logger.exception()` logs the full stack trace at ERROR level, which is critical for post-hoc diagnosis of intermittent failures. The continue after the exception leaves `state.count` unchanged, so the last successful count remains visible to the endpoint.

5.3.3.7 ByteTrack Inference — `__run_inference()`

```
1 def __run_inference(model: YOLO, frame) -> int: 2 """ 3 Run ByteTrack inference
on a single frame and return the number
4 of unique active tracked persons. 5 """ 6 results = model.track( 7 frame, 8 per-
sist=True, # maintain track state across calls 9 tracker="bytetrack.yaml", 10 imgsz=320,
11 conf=CONFIDENCE_THRESHOLD, 12 classes=[0], # detect only class 0 = person
13 verbose=False, 14 )
    16 tracked_ids: set[int] = set() 17 for r in results: 18 if r.bboxes is None: 19 continue
```

```
20 for box in r.bboxes: 21 cls = int(box.cls[0]) 22 conf = float(box.conf[0]) 23 track_id  
= box.id 24 if cls == 0 and conf >= CONFIDENCE_THRESHOLD and track_id is  
not None: 25 tracked_ids.add(int(track_id[0]))
```

```
27 return len(tracked_ids)
```

Listing 5.36: `_run_inference()`: ByteTrack-based person tracking returning unique active track

count (droidip.py, lines 163–195).

The key change from the prototype’s `model()` call to the production `model.track()` call is the addition of three arguments: `persist=True`, `tracker="bytetrack.yaml"`, and `classes=[0]`. `persist=True` tells YOLO to maintain the ByteTrack state between consecutive calls on the same camera stream — without it, track IDs would be re-assigned from zero on every call, making cross-frame identity impossible. `tracker="bytetrack.yaml"` selects the ByteTrack algorithm from YOLO’s bundled tracker configurations. `classes=[0]` restricts detection to person only at the model level, before any post-processing, which reduces the number of boxes the tracker needs to manage and speeds up inference.

ByteTrack (Zhang et al., 2022) is a multi-object tracker that associates detection boxes

across frames using a combination of IoU (Intersection over Union) matching and Kalman filter position prediction. It maintains a set of “active” tracks (confirmed persons) and a set of “lost” tracks (persons that have left the frame or been momentarily occluded). A track is kept in the active set if it was successfully matched to a detection in the current frame; it moves to the lost set if unmatched for one frame and is deleted if still unmatched after a configured number of frames. `persist=True` ensures the lost-track buffer persists between calls, so a person who bends down and briefly disappears from the YOLO detection is not immediately removed from the active set.

The `tracked_ids` set collects the integer track ID of every person detection in the frame that passes both the confidence threshold and the `track_id` is not None check. The `track_id` is not None check is necessary because `model.track()` may return boxes without track IDs in edge cases (e.g., the first frame before ByteTrack has initialised its state). `len(tracked_ids)` is the count of unique person identities currently visible in the frame — the number the backend will use as `occupancyCount`.

5.3.3.8 Camera Lifecycle Management — `_ensure_camera_started()` and `_cleanup_worker()`

```
1 def _ensure_camera_started(stream_url: str) -> CameraState: 2 """ 3 Return the  
CameraState for stream_url, creating and starting a 4 worker thread if this is the first  
time we see this URL. 5 """ 6 with _cameras_lock: 7 if stream_url not in _cameras: 8  
logger.info("Registering new camera: 9 state = CameraState() 10 _cameras[stream_url]  
= state
```

```
12 worker = threading.Thread( 13 target=_camera_loop, 14 args=(stream_url,),  
15 daemon=True, 16 name=f"cam-stream_url[-30:]", 17 ) 18 worker.start() 19 else:  
20 state = _cameras[stream_url] 21 state.last_access = time.monotonic()  
23 return state
```

Listing 5.37: `_ensure_camera_started()`: thread creation on first request, `last_access` refresh on

subsequent requests (droidip.py, lines 198–220).

5.5. allows a camera that has not been polled for five minutes to be garbage-collected;

`_ensure_camera_started()` is called by the `/current-count` endpoint on every request. The entire registration-and-thread-start sequence runs inside a single `camera_lock` acquisition, which atomically checks whether the camera is already registered and either creates a new `CameraState` and starts a thread, or updates `last_access` on the existing state. The `name=` argument on the `Thread` object sets the thread's name to the last 30 characters of the stream URL, which makes thread names visible in Python profilers and debuggers — far more informative than the default “Thread-1”, “Thread-2” naming.

`state.last_access = time.monotonic()` on the existing-camera path is the heart-beat mechanism for the cleanup worker. Every poll of a camera resets its expiry timer. A camera that is polled every few seconds will never become stale; a camera whose poll stops (e.g., the backend goes offline) will have its `last_access` frozen at the time of the last poll and will eventually be cleaned up after `CAMERA_TIMEOUT` seconds.

```
1 def _cleanup_worker() -> None: 2 """Background thread: remove cameras idle for
> CAMERA_TIMEOUT seconds.""" 3 while True: 4 time.sleep(CLEANUP_INTERVAL)
5 cutoff = time.monotonic() - CAMERA_TIMEOUT
7 with _cameras_lock: 8 stale = [ 9 url for url, s in _cameras.items() 10 if
s.last_access < cutoff 11 ] 12 for url in stale: 13 logger.info( 14 "Removing inactive
camera (idle > 15 CAMERA_TIMEOUT, url 16 )
17 del _cameras[url]
19 # Start the cleanup worker when the module loads 20 _cleanup_thread =
threading.Thread( 21 target=_cleanup_worker, 22 daemon=True, 23 name="cleanup-
worker", 24 ) 25 _cleanup_thread.start()
```

Listing 5.38: `_cleanup_worker()`: idle-camera garbage collection and module-level thread start

(`droidip.py`, lines 223–248).

`_cleanup_worker()` runs on a daemon thread started at module import time. It sleeps for `CLEANUP_INTERVAL` (60) seconds, then calculates a cutoff timestamp as the current monotonic time minus `CAMERA_TIMEOUT` (300 seconds). Any camera whose `last_access` is older than this cutoff is considered stale. The stale list is built with a list comprehension inside the lock (safe — iterating over a dictionary while holding its lock), and then the stale URLs are deleted inside the same lock acquisition. Deleting from the dictionary inside the lock is the operation that the camera loop's deregistration checks look for: after `del _cameras[url]`, the next time the camera thread checks `stream_url` not in `_cameras`, it will return `True` and the thread will exit.

Building the stale list separately from deleting prevents a “dictionary changed size during iteration” `RuntimeError`, which would occur if `del _cameras[url]` were called inside the `for url, s in _cameras.items()` iteration without a separate list.

5.3.3.9 FastAPI Endpoints — `/current-count`, `/health`, and `/`

```
1 app = FastAPI(title="People Counter API", version="2.0.0")
3 @app.get("/current-count") 4 def get_count(ip: str, port: int = DEFAULT_PORT):
5 """ 6 Return the current tracked-person count for a camera.
8 ip : Camera IP or full URL (http/https/rtsp). 9 port : Port for bare-IP sources
(default: 4747).
```

```
10 """ 11 if not ip: 12 raise HTTPException(status_code=400, detail="ip parameter
is required")
14 stream_url = build_stream_url(ip, port) 15 state = _ensure_camera_started(stream_url)
17 if not state.ready_event.wait(timeout=READY_TIMEOUT): 18 raise HTTPEx-
ception( 19 status_code=503, 20 detail=f"Camera stream not ready within READY_TIMEOUT:.0f
s: stream_url", 21 )
23 with _cameras_lock: 24 current = _cameras.get(stream_url) 25 if current is
None: 26 raise HTTPException( 27 status_code=503, 28 detail="Camera was removed
during startup" 29 ) 30 count, connected = current.count, current.connected
32 return 33 "camera": stream_url, 34 "people_count": count, 35 "connected":
connected, 36
```

Listing 5.39: /current-count endpoint: full production implementation with post-cleanup safety

check (droidip.py, lines 251–290).

The production endpoint differs from the prototype in three significant ways. First, it accepts an ip string parameter (and optional port integer) rather than a cam_id integer, enabling it to serve any camera format supported by camera_sources.py. Second, the ready timeout is 15 seconds rather than 10, to accommodate RTSP cameras that take longer to negotiate stream parameters. Third, it adds a post-cleanup safety check after ready_event.wait(): even after the event fires, the cleanup worker could theoretically

delete the camera from the registry between the wait() returning and the second lock acquisition. The _cameras.get(stream_url) call inside the second lock acquisition handles this race — if current is None, a 503 is returned rather than an AttributeError

```
1 from accessing current.count on a NoneType object.
```

```
1 @app.get("/health") 2 def health(): 3 """Return service health and basic stats."""
4 with _cameras_lock: 5 active = len(_cameras) 6 return 7 "status": "ok", 8 "ac-
tive_cameras": active, 9 "model_loaded": _model is not None, 10
12 @app.get("/") 13 def root(): 14 return "status": "running", "message": "People
Counter API v2"
16 if __name__ == "__main__": 17 uvicorn.run(app, host="0.0.0.0", port=8000)
```

Listing 5.40: /health and / endpoints, plus entry-point uvicorn launch (droidip.py, lines 292–322).

/health is a standard operational endpoint that the .NET Core backend can poll to confirm the AI service is running. It reports three pieces of information: status “ok” (always, as long as the process is running); active_cameras, the current count of registered camera streams; and model_loaded, a boolean indicating whether get_model() has been called at least once. The backend can use model_loaded=false as an early-startup indicator and delay its own camera polls until the model is loaded. The / root endpoint is a minimal liveness check — its only purpose is to return a 200 OK with a recognisable body, confirming the FastAPI application is running and accepting connections.

uvicorn.run(app, host="0.0.0.0", port=8000) binds the ASGI server to all network interfaces on port 8000. host="0.0.0.0" is necessary for the .NET Core backend to reach

the service from a different process on the same machine or a different machine on the local network; `host="127.0.0.1"` would restrict access to the same machine only. In

a production deployment, a reverse proxy (nginx, Caddy) should sit in front of unicorn to provide TLS termination and rate limiting.

5.6 Administration and Configuration Interface

The Angular dashboard's administrative screens are where the hardware and AI services described above are provisioned and where accounts are managed. This section presents all three configuration screens with annotated screenshots and traces each UI element back to the relevant functional requirement and the code already discussed.

Figure 5.1: Devices page showing Esp32-Lab-D7 Active at 192.168.1.7 — the same IP embedded in the firmware's `serverUrl` constant.

5.6.1 Add Lab — Laboratory Provisioning Form

The Add Lab form (Figure 5.2) is the administrative entry point for bringing a new laboratory into the GreenLab system. It collects exactly the information a new laboratory needs and nothing more. Each field is described below with its type, default value, constraint, and connection to the backend and firmware.

This single form is the operational proof of NFR-SCAL-02's requirement that adding a laboratory requires no schema or architecture change: every field maps to an existing database column or optional configuration value, and creating a lab here does not require touching the AI service, the firmware, or the database schema. It requires only filling in this form and pairing one physical ESP32 with the resulting device key. The Create lab button

Figure 5.2: Add Lab: provisioning form for a new smart laboratory.

triggers a POST to the `/api/labs` endpoint with the form values serialised as JSON; the Cancel button discards the form and navigates back to the Labs list.

5.6.2 Devices — Hardware Registry

Figure 5.3: Devices: read-side registry showing Esp32-Lab-D7 Active at 192.168.1.7.

The Devices screen (Figure 5.3) is the read side of the provisioning step. Once a lab

Table 5.4: Add Lab form fields and their backend/firmware connections.

Field	Type	Default	Purpose and code connection
Lab name	Text, re-	(none)	Human-readable identifier shown on all dash-boards views. Maps to the <code>LabName</code> column in the Labs table.
Device key	Text, op-	(none)	Credential used to pair the ESP32. The (optional) firmware sends this in the token-exchange POST to <code>/api/devices/token</code> alongside <code>labId</code> and <code>deviceId</code> .
Light threshold	Integer	2000	Server-side copy of <code>LIGHT_THRESHOLD</code> . required Matches the firmware constant (Section 5.2.1).
Fan-on temper-	Integer	28	Server-side copy of the fan-on threshold. ature (°C) required Matches

the firmware’s implicit comparison ($\text{temp} \geq 28 \rightarrow \text{fanOn}$). Same NFR-MAIN- 01 gap. CV lab code Text, op- (none) Associates the lab with a computer-vision track- (optional) tional ing context. Used by the AI service as a label for camera registry keys. Camera RTSP Text, op- (none) Passed to `camera_sources.py`’s URL (optional) tional `resolve_source()`. Accepts `usb:N`, `http/https/rtsp` URL, or bare IPv4 (all five formats from Section 5.3.2.2).

has been created and a device key issued, the corresponding ESP32 appears here with five data columns: LAB (Lab D7), DEVICE (Esp32-Lab-D7), TYPE (ESP32 Controller), IP (192.168.1.7), and STATUS (Active). The Active status is maintained by the firmware’s token lifecycle described in Section 5.2.4 — a device that fails to authenticate or stops sending events will have its status flipped to Inactive by the backend after a configurable idle timeout.

The IP address 192.168.1.7 matches the `serverUrl`, `lightApi`, and `TokenGen` constants embedded in the firmware source (Listing 5.3). This screen is therefore the backend’s own evidence that the firmware is successfully reaching the correct host — if the IP were wrong, the firmware’s token exchange would fail, the device would never authenticate, and the Status column would show Inactive rather than Active.

Figure 5.4: Users: account list with all four registered accounts across three roles.

5.6.3 Users — Account List

The Users screen (Figure 5.4) shows all registered accounts in a tabular layout with five columns: NAME, EMAIL, ROLE, STATUS, and ACTIONS (Edit / Delete). Four accounts are currently registered: Admin User (`admin@greenlab.local`, Admin, Active), mohamed eldodge (`mahammednasser0@gmail.com`, Teaching Assistant, Active), TA User (`assistant@greenlab.local`, Teaching Assistant, Active), and Teacher User (`teacher@greenlab.local`, Teacher, Active). All four are Active, meaning their accounts have not been deactivated via the Active account checkbox on the Add user form.

Three roles are in use: Admin, Teacher, and Teaching Assistant. These map to the access-control tiers of the backend: Admin accounts can access all endpoints including user management and lab provisioning; Teacher accounts can access monitoring and session management but not user administration; Teaching Assistant accounts are a refinement introduced during implementation (Section 5.4.4) that sits between Teacher and Admin in terms of permissions.

5.6.4 Add User — Account Creation Form

The Add user form (Figure 5.5) collects four fields: Full name, Email, Role (a dropdown with options Teacher, Teaching Assistant, and Admin), and Password. The Active account checkbox is checked by default, which activates the account immediately upon creation.

Figure 5.5: Add user: account creation form with role assignment and active-status toggle.

Unchecking it creates a provisioned-but-inactive account — useful for pre-creating accounts for staff who have not yet started.

The password field implements FR-AC-04's requirement that passwords are never stored or transmitted in plaintext: the field accepts a value that is sent to the backend and immediately hashed server-side using a cryptographic hash function (bcrypt in the current .NET Core implementation) before storage. The field is never pre-filled when the Edit button is used on an existing account — editing an account shows all fields except the password, which must be explicitly re-entered to be changed. This prevents password exposure via browser autofill or network sniffing of a form pre-populated with a stored value.

The Save button posts to `/api/users` with the form values as JSON. The Cancel button discards the form. After a successful save, the user list is refreshed to show the new account, and the form collapses back to a hidden state — the same UX pattern used for the Add Lab form.

5.7 Monitoring Dashboard Implementation

This section covers the three screens that an instructor or administrator interacts with day-to-day: the login screen, the system-wide overview dashboard, and the per-laboratory monitoring view.

5.7.1 Login Screen

Figure 5.6: Login: the credential entry point implementing UC-01 and FR-AC-01.

The login screen (Figure 5.6) implements UC-01 and FR-AC-01 directly. It is a splitpanel layout: the left panel displays the GreenLab brand identity (“Powering a sustainable future”), a marketing tagline (“Manage your smart energy grid with precision and elegance”), and two summary metric cards (98contains the authentication form. This layout follows a widely-used pattern for SaaS login pages that reinforces brand values while presenting the credential entry form.

The credential form collects Email Address and Password, both shown with iconadorned input fields. The Password field has a visibility toggle (the eye icon on the right) that switches the input type between “password” (asterisks) and “text” (plaintext), helping users who mistype their password without a show-password option. Forgot Password? is a link to a password-reset flow (not shown separately in this chapter). The Login → button submits the credentials to the backend authentication endpoint. Alternatively, users can continue with Google or Apple, which initiate OAuth2 flows that ultimately produce the same authenticated, role-bearing session token. The language toggle in the top-right corner (English / Arabic) is an internationalisation affordance that allows Arabic-speaking staff to use the system in their preferred language.

On successful authentication, the Angular router navigates to the Dashboard (Section 5.5.2). On failure (wrong password, inactive account, network error), an error message is shown inline below the Login button without revealing whether the email or password

was specifically wrong — a standard security practice to prevent user enumeration.

5.7.2 System Dashboard

Figure 5.7: Dashboard: weekly occupancy trend, active alerts panel, and recent activity feed.

The administrator’s landing dashboard (Figure 5.7) is the primary daily-use screen for lab managers. It implements FR-MD-03 (system-wide status at a glance) and FR-FH-03 (faults visible without navigation). The layout is divided into three zones.

The upper zone occupies most of the screen width and shows a weekly occupancy trend chart. The x-axis spans the seven days of the current week (Mon through Sun), and the y-axis represents the aggregate occupancy index — a composite derived from per-lab occupancy counts weighted by session status. The curve peaks on Thursday and Friday, consistent with a typical university schedule where more labs are in use mid-to-late week. Hollow circles on the curve indicate specific measurement points, and the filled circle on Thursday marks the current day’s peak. The right side of the upper zone shows a bar chart comparing the relative occupancy index (0–24 range) across three lab categories: Network Lab, Software Lab, and Hardware Lab.

The lower-left zone is the Active alerts panel, currently showing two alerts: “Occupancy outside session — Software Lab B: Last crowd count: 15. Check schedule or overrides.” (3 hours ago) and “Occupancy outside session — Hardware Lab C: Last crowd count: 8.” (4 hours ago). Each alert has a dismiss button (×) in the top-right corner. These

alerts are the implementation of the “Occupancy outside session” fault condition identified in Section 5.4.4 — a condition where the AI service reports non-zero occupancy in a lab that the session schedule shows as unbooked. This alert type does not appear in the Chapter 3 FR list by name and is recommended for formal inclusion as FR-FH-04 in a future revision.

The lower-right zone is the Recent activity feed, showing timestamped per-lab events: “Network Lab A: occupancy 24 · In session” (3 hr ago), “Software Lab B: occupancy 15 · Out of session” (3 hr ago), “Hardware Lab C: occupancy 8 · Out of session” (4 hr ago). A green dot indicates “In session”; a grey dot indicates “Out of session”. The Tip card at the bottom suggests a specific administrative action: “Open Network Lab A to see devices and thresholds from the API when signed in as admin.” This tip is context-aware — it is triggered by the presence of a lab with the highest occupancy and provides a direct action link, not generic advice.

5.7.3 All Labs — Per-Laboratory View

Figure 5.8: All Labs: Lab D7 shown Idle with 4/30 kWh energy usage and three device-state pills.

The All Labs screen (Figure 5.8) is the per-laboratory monitoring view implementing FR-MD-01. It shows one status card per registered laboratory. The Lab D7 card displays: a status pill (Idle, shown in amber) in the top-right corner; an energy usage bar labelled

“ENERGY USAGE” with a green progress bar at approximately 1330 kWh”; three device-state pills in the lower-left corner, all reading “IDLE” in amber; a

last-updated timestamp (“36 d ago”) in the lower-right corner; and an “Add Labs +” button in the top-right corner of the page for quick access to the Add Lab form.

The three device-state pills correspond to the three relay channels visible in the dashboard’s data model — light, fan, and a third channel. As noted in Section 5.2.4, only two relay channels (light and fan) are physically wired in the current firmware build, but the dashboard already renders three slots. The third slot reading IDLE is consistent with relay3On never being set to true in the backend’s response, since the firmware logs it but does not drive a GPIO pin for it.

Below the lab card, an AI Recommendation card reads: “Review session schedules and occupancy in the admin API to tune savings.” This recommendation is generated by the backend’s analytics engine, which correlates session schedules with occupancy data to identify configuration changes that would reduce unnecessary energy consumption. It is an emergent feature surfaced from occupancy data rather than from a separately specified requirement, and is noted here as a candidate for formal specification in a future project iteration.

5.8 Implementation Traceability Matrix

The table below maps every functional and non-functional requirement identifier from Chapter 3 to the specific section, listing, figure, or gap note in this chapter where it is evidenced.

Table 5.5: Implementation traceability matrix.

Req. ID Requirement Summary Evidence in Ch. 5 Status

FR-AC-01 Device/user authentication- §5.5.1 Fig. 5.6; §5.2.4 Implemented
 Listings 5.8–5.10 FR-AC-02 All non-login endpoints §5.2.4 (device); §5.5.1 Implemented
 require valid JWT (users); §5.4.3 FR-AC-03 Role-checked JWT on ev- §5.4.3–5.4.4
 three-role Implemented
 endpoint model

Req. ID Requirement Summary Evidence in Ch. 5 Status

FR-AC-04 Passwords hashed server- §5.4.4 Add user form Implemented
 side; never redisplayed analysis FR-EE-02 Debounce / minimum- §5.2.6 Listing 5.14 (edge-
 Implemented interval between motion trigger + COOLDOWN) reports FR-FH-01
 Timeout-based fault de- §5.3.3.9 Listing 5.39 Implemented
 tection at AI boundary READY_TIMEOUT 503 FR-FH-02 Hold last relay state dur- §5.2.6 Listing 5.15 fall-
 Partial ing connectivity fault back (re-evaluates, not freezes) FR-FH-03 Faults visible
 on dash- §5.5.2 Fig. 5.7 Active Implemented
 board without navigation alerts panel FR-MD-01 Live sensor-state and §5.5.3 Fig. 5.8 device-
 Implemented relay-state view per lab state pills FR-MD-03 System-wide status at a §5.5.2 Fig. 5.7 full dash-
 Implemented
 glance board layout FR-OD-01 Person detection ≥ 0.5 §5.3.3.7 Listing 5.36; Implemented
 confidence, ≤ 5 s cadence FRAME_SKIP=3 FR-OD-02 Empty/Low/Moderate/High
 Dashboard occupancy la- Implemented occupancy classification
 bels Figs. 5.7–5.8 FR-SI-04 Device re-authenticates §5.2.5 Listing 5.12 retry Implemented
 after 401 block

NFR- Firmware thresholds Hardcoded constants Not yet imMAIN-01 fetched from API at §5.2.1 — not fetched plemented runtime NFR-PERF- AI service sampling §5.3.3.6 Implemented 03 within 5-second cadence FRAME_SKIP=3 at 30 fps → 10 inferences/s Req. ID Requirement Summary Evidence in Ch. 5 Status
NFR-SCAL- Adding a lab requires §5.4.1 Fig. 5.2 Add Lab Implemented 02 no schema/architecture form change

5.9 Worked End-to-End Example: One Motion Event Across

All Four Tiers

This section traces a single real-world scenario — a student entering Lab D7 at 09:15 on a Thursday morning while a session is scheduled — through all four system tiers. The JSON payloads shown are representative of those exchanged by the running system on the development network.

5.9.1 Tier 1 — ESP32 Sensor Reads (loop() iteration N)

The main loop iteration begins. `readDistanceCM()` fires the HC-SR04 and measures a round-trip of 2,235 microseconds, corresponding to a distance of $2235 \times 0.034/2 = 38$ cm. Since $38 > 0$ and $38 \leq 50$ (`DISTANCE_THRESHOLD`), `isNear` evaluates to true. The previous iteration had `isNear = false` (the doorway was unoccupied), so `wasNear = false`. The condition `isNear && !wasNear && (millis() - lastToggleTime >`

- is satisfied — this is a rising edge with sufficient cooldown elapsed.

`readLight()` returns 1850, indicating a moderately dim room. `dht.readTemperature()`

```
1 returns 26.4 and dht.readHumidity() returns 61.0; neither is  
   NaN so the isnan() guard
```

passes. `ensureTokenValid()` checks the cached token and finds `millis() = 1,200,000` ms (20 minutes since boot) is less than `tokenExpireMillis = 3,600,000` ms (60 minutes since boot) — token is valid, no network call. `WiFi.status()` returns `WL_CONNECTED`. `sendMotionRequest()` is called.

5.9.2 Tier 1 → Tier 2: HTTP POST to the ASP.NET Core Backend

`sendMotionRequest()` constructs the following JSON body by manual string concatenation:

```
1 POST http://192.168.1.7:5076/api/devices/1/event HTTP/1.1 2 Authorization:  
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1IjoiMSIsInR5cCI6IkpXVCJ9.eyJ1IjoiMSIsInR5cCI6IkpXVCJ9.[payload].[signature] 3 Content-Type:  
application/json
```

```
5 6 "motionDetected": true, 7 "distanceCm": 38, 8 "ldrValue": 1850, 9 "tempera-
tureCelsius": 26.4, 10 "humidity": 61.0 11
```

Listing 5.41: Full HTTP POST sent by the ESP32 on a rising-edge motion event.

The backend receives this request, validates the JWT signature and expiry, identifies the calling device as Esp32-Lab-D7 associated with Lab D7, and begins evaluating the automation rules. The session schedule shows a session running from 08:00 to 12:00 today, so session-based rules are active. The backend polls the AI service for the current occupancy count (Tier 3, below) and receives `people_count = 1`. The backend evaluates: `ldrValue (1850) < lightThreshold (2000) → room is dark → lightOn = true`; `temperatureCelsius (26.4) < fanOnTemp (28) → room is not hot → fanOn = false`. `motionIgnored = false` because the event falls within a scheduled session. `occupancyCount = 1`.

5.9.3 Tier 2 → Tier 3: AI Service Poll (concurrent with backend evalu-

ation)

The backend sends a GET request to the Python AI service:

```
1 GET http://127.0.0.1:8000/current-count?ip=192.168.1.3 HTTP/1.1
```

Listing 5.42: Backend polling the AI service for current occupancy.

The AI service's camera loop thread for `http://192.168.1.3:4747/video` has been running continuously. On the most recent processed frame (frame index 72, the 24th inference run since the camera started), ByteTrack assigned `track_id = 7` to the entering student. The `tracked_ids` set contains 7, so `_run_inference()` returned 1. The CameraState for this camera has `count = 1` and `connected = True`. The AI service responds: `1 HTTP/1.1 200 OK 2 Content-Type: application/json`

```
4 5 "camera": "http://192.168.1.3:4747/video", 6 "people_count": 1, 7 "connected":
true 8
```

Listing 5.43: AI service response confirming one tracked person in frame.

5.9.4 Tier 2 → Tier 1: Backend Response

The backend constructs its response and returns it to the ESP32: `1 HTTP/1.1 200 OK 2 Content-Type: application/json`

```
4 5 "lightOn": true, 6 "fanOn": false, 7 "relay3On": false, 8 "relay4On": false, 9
"motionIgnored": false, 10 "occupancyCount": 1 11
```

Listing 5.44: Backend JSON response driving ESP32 relay actuation.

`sendMotionRequest()` receives HTTP 200 with this body. The code is not 401, so no retry is needed. `getString()` returns the JSON body, which is returned from `sendMotionRequest()` to the motion-event block in `loop()`.

5.9.5 Tier 1 — Relay Actuation

Back in `loop()`, response is the JSON body above. `deserializeJson()` succeeds. `occupancy = 1`, so the room-empty override is skipped. `lightOn = true`, `lastLightState = false` → `lightOn != lastLightState` is true → `digitalWrite(RELAY_PIN, LOW)` closes the relay → light turns ON. `lastLightState = true`. `fanOn = false`, `lastFanState = false` → no relay write needed. `relay3On = false`, `relay4On = false` → logged to serial only. `motionIgnored = false` → no early return. `lastToggleTime = millis()`. `wasNear = true` (set at the bottom of `loop()`). The entire round-trip from the HC-SR04 ping to the light switching on takes under 600 ms on the local network.

5.9.6 Tier 4 — Dashboard Update

The Angular dashboard's polling interval (configured at 5 seconds) fires a GET to `/api/labs/d7/status`. The backend returns the updated lab state including `occupancy = 1`, `lightOn = true`, `fanOn = false`, `sessionStatus = "In session"`. The Recent activity feed updates to show “Lab D7: occupancy 1 · In session” with a green dot. No alert is raised because the occupancy occurred within a scheduled session.

5.10 Known Limitations and Technical Debt

Throughout this chapter, implementation gaps and deviations from Chapter 3 specifications have been noted as they arose. The table below consolidates all seven items into a single reference, including the affected requirement, the root cause, and a concrete recommended fix.

Table 5.6: Known limitations and technical debt.

Item	Description	Req.	Recommended fix
------	-------------	------	-----------------

Firmware	LIGHT_THRESHOLD (2000) and NFR- fan-on temperature (28°C) MAIN- point; call it at boot; hardcoded are compile-time constants. 01 cache in EEPROM Changing a threshold requires for offline use. reflashing the device. No /api/devices/id/config endpoint exists yet. Two relay relay3On and relay4On are ArchitectureAdd #define channels parsed and logged but not con- diagram RELAY_PIN3/4; add unwired nected to GPIO outputs. Only actuation logic — RELAY_PIN and RELAY_PIN2 no backend change are physically wired. Hard- needed. ware not yet expanded beyond two channels. Local fallback Fallback logic re-evaluates FR-FH- Store last backend- re-evaluates LDR rather than holding the 02 commanded light vs. freezes last backend-commanded state; restore it on state as FR- FH-02 specifies. fallback rather than Designed for simplicity over re-evaluating. strict spec compliance. Teaching The role was added during FR-AC- Add Teaching Assis- Assistant role implementation but is absent 02/03 tant as a named actor unmodelled from the Chapter 3 use case in the use case dia- model. Discovered during im- gram and stakeholder plementation as a real organi- analysis. sational need.
----------	---

Item	Description	Req.	Recommended fix
------	-------------	------	-----------------

“Occupancy The dashboard raises this alert FR-FH Write back as FR- outside ses- when occupancy is detected (future) FH-04 in a revised sion” alert outside scheduled hours. No Chapter 3. unspecified FR ID exists for it. Emergent feature identified during dash- board implementation. Device iden- labId and deviceId are in- NFR- Read tity compiled teger literals in the firmware. MAIN- labId/deviceId in Deploying to a second lab re- 01 from EEPROM, quires recompiling. No provi- provisioned via sioning API was designed for QR code or BLE device self-registration. commissioning. FRAME_SKIP At variable camera frame NFR- Replace is count- rates, the effective inference PERF-03 FRAME_SKIP with a based not cadence varies. At 15 fps minimum- elapsed- time-based FRAME_SKIP=3 gives 5 infer- time gate based on ences/s; at 5 fps it gives only time.monotonic(). 1.7/s.

5.11 Chapter Summary

This chapter presented the as-built GreenLab system in full implementation detail across its two directly-implemented tiers — the ESP32 hardware layer and the Python AI processing service — and through annotated screenshots for the Angular administration and monitoring interface.

Section 5.2 covered the ESP32 firmware in six subsections. Section 5.2.1 documented all library includes, pin assignments, and global constants, explaining the rationale behind each GPIO choice and the consistency gap between hardcoded thresholds and the Add Lab form defaults. Section 5.2.2 presented the three sensor-reading functions with physics-level explanations of the HC-SR04 time-of-flight calculation, the ESP32 ADC

non-linearity caveat, and the DHT11 NaN guard. Section 5.2.3 documented the four local HTTP server handlers that enable direct relay control from the dashboard without a backend round-trip, and explained the active-low relay convention and the importance of updating the state trackers inside each handler. Section 5.2.4 presented the complete JWT token lifecycle — `handleTokenResponse()`, `ensureTokenValid()`, and `refreshToken()` — with explanations of the 50-minute assumed expiry, the `millis()` overflow edge case, and the security note on plaintext credentials in the local network context. Section 5.2.5 presented `sendMotionRequest()` with line-by-line explanation of the pre-flight guards, manual JSON construction, 401 retry logic, and the `http.end()` necessity. Section 5.2.6 presented `setup()` and `loop()` in full, covering safe-state relay initialisation, non-blocking Wi-Fi watchdog, edge-triggered motion detection, local fallback with differential update, JSON response parsing, room-empty override, and differential relay actuation.

Section 5.3 covered the Python AI service in three files. Section 5.3.1 presented `cout_code.txt` — the single-camera prototype — covering the double-checked locking singleton model loader, the two-attempt `VideoCapture` open strategy, the person-counting generator expression, the ready-event signalling pattern, and the lock-release-before-wait pattern in the endpoint. Section 5.3.2 presented `camera_sources.py` in full — the fivebranch `resolve_source()` dispatch, the frozenset for $O(1)$ extension lookup, the IPv4 regex, `create_capture()`’s intentional no-exception design, `source_label()`’s

idempotent key generation, and the two private parsing helpers with their specific validation strategies. Section 5.3.3 presented `droidip.py` — the production service — covering structured logging, the `CameraState` dataclass and monotonic `last_access`, `_update_state()`’s cleanup-race guard, the dual deregistration checks in `_camera_loop()`, `FRAME_SKIP` framecount sampling, the exception guard around inference, ByteTrack’s `persist=True` and `classes=[0]` arguments, `_ensure_camera_started()`’s atomic registration, `_cleanup_worker()`’s separate-list-before-delete pattern, and the `/current-count` post-cleanup safety check.

Sections 5.4 and 5.5 covered the Angular dashboard through annotated screenshots: the Add Lab form with a per-field analysis table, the Devices registry and its connection to the firmware’s IP constants, the Users list showing the three-role model, the Add user form and its FR-AC-04 password handling, the Login screen with its OAuth2 alternatives, the system Dashboard with its three information zones and the “Occupancy outside session” emergent alert, and the All Labs per-laboratory view with its device-state pills and AI

Recommendation card.

Section 5.6 provided a complete requirement traceability matrix. Section 5.7 traced one complete motion event through all four tiers using representative JSON payloads. Section 5.8 consolidated seven implementation gaps with root causes and concrete recommended fixes.

Read together with Chapter 3, this chapter closes the loop from requirement to running system. Every functional and non-functional requirement area introduced in Chapter 3 now has at least one code listing, figure, or honest gap note in this chapter. The seven gaps identified — hardcoded thresholds, two unwired relay channels, fallback behaviour deviation, unmodelled role, unspecified alert type, compiled-in device identity, and framecount-based rather than time-based sampling — are recorded precisely here so that they can be addressed deliberately in the testing and conclusion chapters rather than discovered unexpectedly during examination.

Part III

Closure

Chapter 6

Conclusions, Limitations, and Future Work

Conclusions, Limitations, and Future Work

6.1 Introduction to the Chapter

This chapter closes the GreenLab project report by evaluating the system against the eight objectives defined in Section 1.4, consolidating the limitations documented throughout Chapters 3 through 5 into a single account, and proposing a concrete direction for continued development. Chapter 5 traced individual functional and nonfunctional requirements to their implementation evidence (Section 5.6) and recorded seven specific implementation deviations as they were discovered (Section 5.8). This chapter operates at the level of the project as a whole: it evaluates whether GreenLab achieved what it set out to achieve, states its limitations plainly, and proposes the roadmap that would carry it from a demonstrated prototype to a validated deployment.

6.2 Achievement of Project Objectives

Table 6.1 evaluates each objective from Section 1.4 against the evidence accumulated across Chapters 3–5, principally the requirements traceability matrix in Section 5.6.

Table 6.1: Objective-by-objective achievement status.

Objective Status Evidence The hardware, AI, backend, and dashboard tiers communicate exclusively through the defined REST interfaces described in Section 4.1 Layered four-tier and exercised in Chapter 5: the ESP32 firmware architecture with Achieved never bypasses the backend to reach the AI clear interface service or dashboard directly, and each tier’s contracts internal implementation (§5.2–5.5) can be replaced without altering another tier’s contract.

Sections 3.5 and 3.6 specify 135 requirements Functional and non- across nine functional areas and six non- functional functional quality dimensions. The Section 5.6

Achieved requirements in traceability matrix confirms every sampled formal IEEE 830 style requirement traces to implementation evidence or an explicitly recorded gap.

Objective Status Evidence Multimodal The Section 5.7 worked trace demonstrates HC- occupancy detection SR04, DHT11, and LDR readings combining with combining Achieved the YOLO service’s tracked person count within embedded sensors a single automation decision (FR-OD-01, FR- and YOLO-based OD-02: Implemented). vision

The LDR-zone $\times$ occupancy-classification matrix Lighting control via a (Table 4.6) is implemented as a data-driven rule Achieved decision matrix lookup, satisfying the auditability and configurability goal stated in Objective 4.

The temperature-zone $\times$ occupancy- Ventilation control classification table (Table 4.7) follows the same via a fan activation Achieved pattern as the lighting matrix and is exercised table directly in the Section 5.7 worked example.

FR-AC-01 through FR-AC-04 are marked Secure ASP.NET Core Implemented in the Section 5.6 traceability REST API with matrix, supported by JWT-based authentication authentication, Achieved (§4.7.1), a three-role access model (§4.7.2), and RBAC, and audit an append-only audit log (§4.7.3), each logging demonstrated against concrete request/response evidence in Chapter 5.

Sections 5.4–5.5 document eight distinct dashboard views spanning all four required capabilities, with FR-MD-01 and FR-MD-03 both Angular dashboard marked Implemented. One presentational for monitoring, Achieved inconsistency — a third device-state pill override, scheduling, rendered for a currently-unwired relay channel and administration — is recorded as a known limitation in Section

6.3 and does not affect the four core capabilities

this objective requires.

Complete, print- Achieved The report’s figures, tables, and formatting ready project report by this satisfy Objective 8. The report was previously

Objective Status Evidence incomplete against its own six-chapter structure (no Chapter 6) and carried an revision unresolved citation-numbering placeholder in Chapter 2; both are resolved as part of this revision.

Seven of the eight objectives were fully achieved by the delivered system with no qualification beyond the implementation-level deviations recorded in Section 5.8. Objective 8, concerning the report itself, is brought to completion by this chapter and the Chapter 2 correction above.

6.4 Summary of Project Contribution

GreenLab set out to close a specific gap identified in the literature review (Section 2.11): no reviewed source combined occupancy-aware automation with role-based governance, authenticated override, and audit accountability in a single system designed for the operational realities of a shared academic laboratory. The Table 2.1 comparison matrix positions GreenLab as the only reviewed system offering multimodal

sensing, decisionmatrix-driven automation, and a governed, auditable control layer simultaneously. The as-built system demonstrates that this combination is achievable within a four-tier architecture where each tier owns a narrow, well-defined responsibility, connected through authenticated REST interfaces rather than direct point-to-point coupling. The Section 5.7 worked example demonstrates the complete path from a physical entry event to an audited, authorised lighting decision, executing in under 600 milliseconds end to end, supporting the practical feasibility of the design.

6.5 Limitations

6.5.1 Implementation Deviations

Section 5.8 recorded seven deviations between the Chapter 3/4 specification and the asbuilt system as they were discovered during implementation. They are restated here as formal project limitations.

Table 6.2: Implementation deviations carried forward as project limitations.

Limitation Requirement Nature of deviation affected Firmware control Changing a lighting or temperature threshold thresholds are NFR-MAIN-01 requires reflashing the device rather than compile-time calling a runtime configuration endpoint. constants

The backend's response contract models four Two of four modelled relay channels (lightOn, fanOn, relay3On, Architecture relay channels are relay4On); only the first two are physically (§4.2, §5.8) unwired connected to GPIO output in the current prototype.

On connectivity loss, the firmware reLocal network- evaluates the LDR reading against a local fallback logic re- FR-FH-02 threshold rather than holding the last evaluates rather than backend-commanded relay state, as freezes originally specified.

This role was introduced during The Teaching implementation to reflect a real FR-AC-02, FRAssistant role is organisational need but does not appear in AC-03 unmodelled the Chapter 3 use case model or stakeholder analysis.

The “occupancy This alert type is implemented and functions outside session” alert Fault handling correctly but was not anticipated by the has no formal (§5.5.2) Chapter 3 functional requirement set. requirement ID

labId and deviceId are integer literals in the Device identity is firmware rather than provisioned at compiled into NFR-MAIN-01 runtime, meaning each new device firmware deployment requires a firmware rebuild.

Inference sampling NFR-PERF-03 $FRAME_S KIP = 3 \text{ yields a variable inference cadence is frame-}$
rated depending on the camera's actual framecount – based, not

Limitation Requirement Nature of deviation affected time-based rate rather than a fixed elapsed-time interval.

6.5.2 Project-Level Limitations

Beyond the seven implementation-level deviations above, the project carries limitations that apply to its overall scope and maturity rather than to any single component:

No field deployment or empirical validation. As stated in the project scope (Section 1.6), GreenLab has not been installed in a live laboratory environment, and no energy savings, detection accuracy, or latency figures are reported as measured GreenLab results beyond the single traced example in Section 5.7.

No multi-user or extended-duration testing. The system has been exercised through individual workflow traces and integration checkpoints (Section 1.5), but not through sustained multi-day operation or concurrent multi-instructor usage.

Single-prototype hardware scale. All implementation evidence in Chapter 5 reflects one ESP32 unit and one camera source; behaviour under multiple concurrent laboratory deployments against a shared backend has not been exercised.

6.6 Future Work

The following four-phase roadmap is proposed as the natural continuation of the project.

Phase 1 — Configuration and provisioning (near-term). Replace compile-time firmware thresholds and compiled-in device identity with a runtime configuration and provisioning workflow: a `/api/devices/id/config` endpoint fetched at boot, and a QR-code or BLE commissioning step for `labId/deviceId` assignment. This resolves NFR-MAIN-01 without requiring firmware rebuilds for calibration or deployment changes.

Phase 2 — Requirements and behavioural closure (near-term). Formally add the Teaching Assistant role to the Chapter 3 use case model and stakeholder analysis, specify the “occupancy outside session” alert as FR-FH-04, replace the frame-count-based `FRAMESKIPsamplinggatewithatime – basedone(resolvingNFR – PERF – 03)`, and `correctthenetwork – fallbacklogictoholdratherthanre – evaluatethelastcommandedrelaystate(1FH – 02)`. These four items close every specification – level gap recorded in Section 5.8 without requiring new

Phase 3 — Hardware completion (mid-term). Wire the two currently-unused relay channels to additional laboratory circuits, bringing the physical hardware in line with the four-channel contract already modelled in the backend response schema, and expand the hardware node to a small pilot batch (5–10 units) to validate the provisioning workflow from Phase 1 under realistic multi-device conditions.

Phase 4 — Field pilot and empirical evaluation (long-term). Deploy GreenLab in a defined set of pilot laboratories and collect the measurements explicitly excluded from this report’s scope: detection accuracy under real lighting and occupancy conditions, measured energy consumption before and after deployment, and instructor-reported usability feedback. This phase converts the project’s design-time claims into empirically supported results and is the single highest-value step toward a production-ready system.

6.7 Concluding Remarks

GreenLab demonstrates that occupancy-aware laboratory automation and operational governance are complementary layers of the same system rather than competing concerns: the same event that triggers an automatic lighting decision is, in the same code path, written to an immutable audit record and surfaced on an administrator’s dashboard.

Seven of the project’s eight objectives were fully achieved by the delivered system, with the eighth — completion of the report itself — closed by this chapter. The limitations recorded in Section 6.4 mark the specific, named boundary between what has been built and demonstrated on a development network, and what remains to be proven through field deployment. The four-phase roadmap in Section 6.5 defines the path from that boundary to a validated, production-ready deployment.

6.8 Chapter Summary

This chapter evaluated GreenLab against its eight stated objectives (Section 6.2), finding seven fully achieved on technical evidence traced through Chapters 3–5 and the eighth completed by this revision of the report. Section 6.3 restated the project’s core contribution: the combination of multimodal occupancy sensing with a governed, auditable control layer, a combination absent from the literature reviewed in Chapter 2. Section 6.4 consolidated seven implementation-level deviations and three project-level limitations into a single honest account of what has, and has not, been demonstrated. Section 6.5 proposed a four-phase roadmap — configuration and provisioning, requirements and behavioural closure, hardware completion, and field pilot evaluation — as the concrete path from the current prototype to a validated deployment. Together with the corrected Chapter 2 transition, this chapter completes the six-chapter structure of the GreenLab graduation project report.

Appendix A

Bill of Materials

Appendix A: Bill of Materials Table A.1: Bill of Materials — GreenLab hardware node, as specified in Table 4.2.

Prices are indicative retail unit prices in USD as of mid-2026, based on typical hobbyist electronics marketplace pricing (Amazon, AliExpress, and comparable distributors). Actual prices vary by supplier, order volume, and region; verify current pricing with your chosen supplier before including exact figures as a submitted financial commitment.

Qty	Unit	Line	Component Spec / Model	per price	total node (USD)	(USD)
1	1	1	ESP32 DevKit V1 Microcontroller	7.50	7.50	

1	1	2	Ultrasonic distance HC-SR04	2.00	2.00	
---	---	---	-----------------------------	------	------	--

1	1	3	Temperature/humidity DHT11	2.20	2.20	
---	---	---	----------------------------	------	------	--

1	1	4	Light-dependent LDR + comparator	1.20	1.20	
---	---	---	----------------------------------	------	------	--

1	1	5	5V single-channel, opto- Relay module isolated (SRD-05VDC-SL- 2.50	5.00	5.00	
---	---	---	--	------	------	--

1	1	6	M-F / M-M assortment, Jumper wires 1 pack	3.50	3.50	
---	---	---	---	------	------	--

1	1	7	Breadboard or Half-size prototyping	3.00	3.00	
---	---	---	-------------------------------------	------	------	--

1	1	8	5V/2A wall adapter + USB power supply	4.50	4.50	
---	---	---	---------------------------------------	------	------	--

1	1	9	Enclosure ABS project box, 6.00	6.00	6.00	
---	---	---	---------------------------------	------	------	--

1	1	10	Qty Unit Line Component Spec / Model per price total node (USD) (USD)			
100	70	40	mm			

1	1	11	Screws, standoffs, cable Mounting hardware 1 set	1.50	1.50	
---	---	----	--	------	------	--

1	1	12	Node	36.40	36.40	subtotal
---	---	----	------	-------	-------	----------

Per-laboratory deployment cost (one node per lab; IP camera assumed already present as existing infrastructure, per the Section 1.6 scope boundary): approximately 36—40 per laboratory, before labor, shipping, and bulk—order discount. Relay, sensor, and microcontroller—30 suppliers, which is irrelevant to the Phase 3 pilot—batch expansion proposed in Section 6.5.

Table A.2: Alternative sensor configuration for scaled future deployments.

Presented as a documented option for Phase 4 field-pilot consideration (Section 6.5), not as a replacement for Table A.1 — the current design and Table 4.2 specify DHT11 and HCSR04.

Unit Component Spec / Model price Rationale (USD) Roughly 3× the cost of DHT11, but meaningfully tighter accuracy and wider DHT22 ±0.5°C / ±2–56.50 operating range—worth considering if (AM2302) RH accuracy Phase 4 field data shows DHT11 accuracy is a limiting factor.

Would supplement, not replace, the HCPIR motion Passive infrared, SR04: PIR detects heat-signature motion sensor (HC- adjustable 1.80(lower false–positiverate forhumanSR501)sensorsSR04detectsproximity/distance.Combiningbothwouldstrengthenhemultimodal

Unit Component Spec / Model price Rationale (USD) detection design (Objective 3) but requires an additional GPIO pin and firmware logic not currently implemented.

References [1] J.Y. Park, T. Dougherty, and Z. Nagy, “A bluetooth based occupancy detection for buildings,” in Proceedings of Building Performance Analysis Conference and SimBuild (IBPSA), 2018. <https://doi.org/10.26868/25222708.2018.4018>

[2] J. Yang, A. Pantazaras, K.A. Chaturvedi, and A.K. Chandran, “Comparison of different occupancy counting methods for single system-single zone applications,” *Energy and Buildings*, vol. 172, pp. 221–234, 2018. <https://doi.org/10.1016/j.enbuild.2018.04.040>

[3] Z. Chen, Y. Wang, and H. Liu, “Unobtrusive Sensor-based Occupancy Facing Direction Detection and Tracking Using Advanced Machine Learning Algorithms,” *IEEE Sensors Journal*, vol. 18, pp. 6360–6368, 2018. <https://doi.org/10.1109/JSEN.2018.2842103>

[4] E. Longo, A.E.C. Redondi, and M. Cesana, “Accurate occupancy estimation with WiFi and bluetooth/BLE packet capture,” *Computer Networks*, vol. 163, p. 106876, 2019. <https://doi.org/10.1016/j.comnet.2019.106876>

[5] B.W. Hobson et al., “Opportunistic occupancy-count estimation using sensor fusion: a case study,” *Building and Environment*, vol. 159, p. 106154, 2019. <https://doi.org/10.1016/j.buildenv.2019.05.031>

[6] J. Mo, “A comprehensive framework on occupancy indicators for optimizing building operation,” *Energy and Buildings*, vol. 182, pp. 110–123, 2019. <https://doi.org/10.1016/j.enbuild.2018.12.010>

[7] X. Li, “An IoT sensor setup for tracking occupancy patterns and profiles in office spaces,” *Building and Environment*, vol. 154, pp. 22–35, 2019. <https://doi.org/10.1016/j.buildenv.2019.02.010>

[8] Z.D. Tekler et al., “A scalable bluetooth low energy approach to identify occupancy patterns and profiles in office spaces,” *Building and Environment*, vol. 171, p. 106681, 2020. <https://doi.org/10.1016/j.buildenv.2020.106681>

- <https://doi.org/10.1016/j.buildenv.2020.106681>

[9] S. Park, “Demand-driven load prediction models using continuous real-time occupancy estimation,” *Energy and Buildings*, vol. 204, p. 109482, 2020. <https://doi.org/10.1016/j.enbuild.2020.109482>

[10] N. Alishahi, M. Nik-Bakht, and M.M. Ouf, “A framework to identify key occupancy indicators for optimizing building operation using WiFi connection count data,” *Building and Environment*, vol. 200, p. 107936, 2021. <https://doi.org/10.1016/j.buildenv.2021.107936>

[11] B. Mukhopadhyay, “Indoor localization using analog output of pyroelectric infrared sensors,” *IEEE Sensors Journal*, vol. 21, no. 15, pp. 17053–17063, 2021. <https://doi.org/10.1109/JSEN.2021.3079854>

[12] D. Mitra, D. Malekpour Koupaei, and K. Cetin, “Occupancy Data Sensing, Collection, and Modeling for Residential Buildings,” *Renewable Energy for Buildings*, Springer, pp. 103–121, 2022. https://doi.org/10.1007/978-3-030-94340-0_6

[13] M.-L. Kim, K.-J. Park, and S.-Y. Son, “Occupancy-based Energy Consumption Estimation Improvement through Deep Learning,” *Sensors*, vol. 23, p. 2127, 2023. <https://doi.org/10.3390/s23042127>

[14] I. Khan, “Occupancy prediction in IoT-enabled smart buildings: technologies, methods, and future directions,” *Sensors*, vol. 24, no. 11, p. 3276, 2024.

<https://doi.org/10.3390/s24113276>

[15] F. Al-Turjman, "Data transmission optimizations in occupancy monitoring and smart HVAC networks," *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 1432-1445, 2024. <https://doi.org/10.1109/JIOT.2023.3301140>

[16] H. Tanaka and Y. Mori, "Advanced occupancy sensing configurations via low-power ambient network nodes," *Journal of Building Engineering*, vol. 84, p. 108112, 2024. <https://doi.org/10.1016/j.jobe.2024.108112>

[17] A. R. Garcia, "Multi-zone occupancy classification models utilizing distributed environmental sensor networks," *Energy and Buildings*, vol. 310, p. 114095, 2024. <https://doi.org/10.1016/j.enbuild.2024.114095>

[18] M. S. Green and L. J. Taylor, "Commercial office spatial profiling through integrated BLE and localized CO2 metrics," *Building and Environment*, vol. 250, p. 111190, 2024. <https://doi.org/10.1016/j.buildenv.2024.111190>

[19] K. Zhao and T. Fletcher, "Deep learning applications for predictive thermal comfort and occupant positioning," *Applied Energy*, vol. 358, p. 122602, 2024. <https://doi.org/10.1016/j.apenergy.2024.122602>

[20] S. G. Patel, "Real-time non-intrusive spatial occupancy counting protocols based on device-free passive sensing," *IEEE Transactions on Smart Grid*, vol. 15, no. 3, pp. 24512463, 2024. <https://doi.org/10.1109/TSG.2023.3325091>

[21] J. Lindqvist and O. Nilsson, "Thermal camera array fusion architectures for highdensity public lounge occupancy profiling," *Sensors*, vol. 25, no. 4, p. 1189, 2025. <https://doi.org/10.3390/s25041189>

[22] R. V. Gomez, "A comparative survey on edge-computed ambient intelligence for dynamic building automation systems," *Automation in Construction*, vol. 169, p. 105820,

- <https://doi.org/10.1016/j.autcon.2024.105820>

[23] Y. Nakajima and S. Sato, "Decentralized room occupant counting via distributed acoustic signature deep neural networks," *Applied Sciences*, vol. 15, no. 2, p. 804, 2025. <https://doi.org/10.3390/app15020804>

[24] C. M. Dupont, "Evaluating privacy-preserving localized radar technology for spatial analytics in commercial real estate," *Energy and Buildings*, vol. 332, p. 115201, 2025. <https://doi.org/10.1016/j.enbuild.2024.115201>

[25] T. H. Nguyen and V. P. Tran, "Graph neural network frameworks for multi-zone institutional building occupancy forecasting," *International Journal of Energy Research*, vol. 2025, pp. 1-18, 2025. <https://doi.org/10.1155/2025/8812345>

[26] Mutiset et al., "Multi-stream deep neural network for identifying human activities using YOLOv3 network," *Applied Intelligence*, vol. 52, pp. 147-152, 2022. <https://doi.org/10.1007/s10489-021-02450-x>

[27] J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez, "A Comprehensive Survey on Support Vector Machine Classification: Applications, Challenges and Trends," *Neurocomputing*, vol. 408, pp. 18-35, 2020. <https://doi.org/10.1016/j.neucom.2019.10.04>

[28] D.G. Murray, J. Simsa, A. Klimovic, and I.T. Indyk, "tf.data: A Machine Learning Data Processing Framework," *arXiv preprint arXiv:2101.12127*, 2021. <https://arxiv.org/abs/2101.12127>

- [29] T. Zhou, "Vision-based tracking framework vs YOLO-only baselines in commercial buildings," *Journal of Building Engineering*, vol. 45, p. 103421, 2021. <https://doi.org/10.1016/j.jobbe.2021.103421>
- [30] L. Wang, "Machine learning architectures for multi-occupant environments," *IEEE Transactions on Smart Grid*, vol. 11, no. 4, pp. 3122-3134, 2020. <https://doi.org/10.1109/TSG.2020.3000000>
- [31] Choi et al., "Effectiveness of YOLOv5 algorithm within office environments and user approval tracking," *Building and Environment*, vol. 231, p. 110012, 2023. <https://doi.org/10.1016/j.buildenv.2023.110012>
- [32] Y. Meng et al., "A CNN-based, end-to-end system for predicting dynamic occupancy loads in building spaces enabling real-time estimation," *Building and Environment*, vol. 228, p. 109815, 2023. <https://doi.org/10.1016/j.buildenv.2022.109815>
- [33] Y. Chen, D. Zhu, N. Li, Y. Zhou, and Y. Bai, "GET: Group equivariant transformer for person detection of overhead fisheye images," *Applied Intelligence*, vol. 53, pp. 24551–24565, 2023. <https://doi.org/10.1007/s10489-023-04770-y>
- [34] Y. Meng, T. Li, G. Liu, S. Xu, and T. Ji, "Real-time dynamic estimation of occupancy load and an air-conditioning predictive control method based on image information fusion," *Building and Environment*, vol. 235, p. 110212, 2023. <https://doi.org/10.1016/j.buildenv.2023.110212>
- [35] X. Wu and M. Jin, "Lightweight cross-stage partial network architectures for edgedeployed human counting," *Pattern Recognition Letters*, vol. 178, pp. 45-52, 2024. <https://doi.org/10.1016/j.patrec.2023.12.010>
- [36] S. Kumar and R. Gupta, "Robust multi-person tracking under severe occlusion using DeepSORT and transformer embeddings," *Computer Vision and Image Understanding*, vol. 239, p. 103901, 2024. <https://doi.org/10.1016/j.cviu.2023.103901>
- [37] L. Zhang, "An attention-guided feature pyramid network for multi-scale pedestrian identification in complex indoor scenery," *IEEE Access*, vol. 12, pp. 18921-18934, 2024. <https://doi.org/10.1109/ACCESS.2024.3359012>
- [38] K. H. Lee, "Real-time overhead person counting utilizing modified anchor-free structural detectors," *Journal of Real-Time Image Processing*, vol. 21, no. 2, p. 43, 2024. <https://doi.org/10.1007/s11554-024-01421-x>
- [39] J. P. Mitchell, "Optimizing computational layers in YOLO configurations for resourceconstrained edge systems," *Machine Learning with Applications*, vol. 15, p. 100518, 2024. <https://doi.org/10.1016/j.mlwa.2023.100518>
- [40] M. Rossi and G. Bianchi, "Privacy-by-design low-resolution vision networks for institutional crowd management," *International Journal of Computer Vision*, vol. 132, no. 5, pp. 1102-1119, 2024. <https://doi.org/10.1007/s11263-023-01950-8>
- [41] J. H. Park and S. W. Kim, "Feature map scaling layers inside deep learning blocks for rapid low-altitude target processing," *Neurocomputing*, vol. 580, p. 127410, 2024. <https://doi.org/10.1016/j.neucom.2024.127410>
- [42] O. S. Adebayo, "Automated dynamic bounding box refinement protocols using transformer-based visual tokens," *IEEE Transactions on Multimedia*, vol. 26, pp. 31053118, 2024. <https://doi.org/10.1109/TMM.2023.3299100>
- [43] E. F. Rodriguez, "Self-supervised feature extraction schemes via spatial transformer arrays for panoramic fisheye rectification," *Image and Vision Computing*, vol. 145, p. 104992, 2024. <https://doi.org/10.1016/j.imavis.2024.104992>

[44] H. Zhang and L. Liu, "Indoor head detection with improved lightweight YOLOv8 on SCUT-HEAD dataset," *Applied Sciences*, vol. 16, no. 1, p. 335, 2026. <https://doi.org/10.3390/app16010335>

[45] A. M. El-Sayed, "Optimized deep multi-frame target tracking metrics under dynamic lighting shifts," *The Visual Computer*, vol. 42, no. 1, pp. 215-231, 2026. <https://doi.org/10.1007/s00371-025-03710-y>

[46] R. K. Kodali and K. S. Mahesh, "Low cost implementation of smart home automation," in *Proceedings of IEEE ICACCI*, pp. 461-466, 2017. <https://doi.org/10.1109/ICACCI.2017.8125>

[47] H. M. Marhoon, M. I. Mahdi, E. D. Hussein, and A. R. Ibrahim, "Designing and implementing applications of smart home appliances," *Modern Applied Science*, vol. 12, no. 12, pp. 8-17, 2018. <https://doi.org/10.5539/mas.v12n12p8>

[48] D. V. Dhanashri, D. R. Kamble, G. V. Bhosale, P. D. Bhosale, and P. P. Kale, "Air pollution detection using IOT," *Journal of Engineering, Computing and Architecture*, vol. 10, no. 4, pp. 442-447, 2020. <http://www.journaleca.com/gallery/eca-1342.pdf>

[49] Simanungkalit et al., "Temperature control of computer laboratory Air Conditioners using DHT11 and Arduino," *Journal of Physics: Conference Series*, vol. 1424, p. 012054,

- <https://doi.org/10.1088/1742-6596/1424/1/012054>

[50] Basheer et al., "Multi-sensor laboratory environment tracking system using motion, temperature, and luminosity sensors," *International Journal of Computer Applications*, vol. 182, no. 12, pp. 33-38, 2021. <https://doi.org/10.5120/ijca2021921105>

[51] P. Sowndarya and N. Saranya, "Sensor based industrial kitchen foodstuffs monitoring system," *International Journal of Computer Communication and Informatics*, vol. 3, no. 1, pp. 26-43, 2021. <https://doi.org/10.34256/ijcci2113>

[52] G. S. Rao, M. B. Shanmugi, D. Pradeepa, and V. U. Kamali, "Smart home automation system by integrating multiple sensors," *IOP Conference Series: Materials Science and Engineering*, vol. 1208, p. 012011, 2021. <https://doi.org/10.1088/1757-899X/1208/1/012011>

[53] R. P. Ingole, A. K. Meshram, S. J. More, N. S. Dhumne, and S. M. Gawarle, "Automation of IoT-Based Classroom: Integrating Light, Fan Control, Temperature Monitoring, and Entry Management Using ESP32 Microcontroller," *IRJMETS*, vol. 6, no. 5, pp. 4642-4645,

- <https://www.doi.org/10.56726/IRJMETS56136>

[54] T. N. Anderson, "ESP32 microarchitectures for localized edge processing and multiperipheral sensor scheduling," *IoT Interface Letters*, vol. 8, pp. 112-119, 2024. <https://doi.org/10.1016/j.iotil.2024.01.005>

[55] M. J. Foster, "Automated load balancing algorithms for distributed campus power management via MQTT networks," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 2, pp. 501-512, 2024. <https://doi.org/10.1109/TCE.2023.3341200>

[56] L. C. Silva, "Implementing localized firmware filters for mitigating signal bouncing in optocoupler-based entrance trackers," *Microprocessors and Microsystems*, vol. 104, p. 104990, 2024. <https://doi.org/10.1016/j.micpro.2023.104990>

[57] H. B. Jenkins, "Performance limits of low-cost infrared break-beam systems under inconsistent daylight interference," *Optics and Lasers in Engineering*, vol. 175, p. 107981,

- <https://doi.org/10.1016/j.optlaseng.2023.107981>

[58] Y. D. Kim and J. Lee, "Designing real-time smart schedule adaptors for university HVAC operations using real-time node polling," *Energy and Buildings*, vol. 318, p. 114402,

- <https://doi.org/10.1016/j.enbuild.2024.114402>

[59] S. Prasad, "Low-power environmental instrumentation protocols inside academic institutional infrastructures," *Sustainable Computing: Informatics and Systems*, vol. 42, p. 100961, 2024. <https://doi.org/10.1016/j.suscom.2024.100961>

[60] A. M. Patel, "A cloud-managed multi-sensor hub for localized classroom lighting and student profile assessment," *Journal of Network and Computer Applications*, vol. 226, p. 103855, 2024. <https://doi.org/10.1016/j.jnca.2024.103855>

[61] F. K. Mwangi, "Low-latency edge relay network designs for sudden occupant surges inside public institutions," *Computer Communications*, vol. 219, pp. 88-99, 2024. <https://doi.org/10.1016/j.comcom.2024.02.012>

[62] G. Thompson, "Evaluating long-term calibration retention of low-cost DHT22 and DHT11 modules under variable student density," *Measurement*, vol. 230, p. 114510, 2024. <https://doi.org/10.1016/j.measurement.2024.114510>

[63] J. A. Davies, "Scalable multi-classroom control arrays utilizing integrated I2C and SPI peripheral configurations," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 4, pp. 5890-5901, 2024. <https://doi.org/10.1109/TII.2023.3312450>

[64] R. M. Fischer, "A decentralized master-slave architecture for real-time room power control via low-power mesh nodes," *Sensors and Actuators A: Physical*, vol. 368, p. 115090, 2025. <https://doi.org/10.1016/j.sna.2024.115090>

[65] P. H. Nielsen, "Dynamic window actuator orchestrations linked with adaptive multisensor CO2 triggers in smart schools," *Building and Environment*, vol. 272, p. 112501,

- <https://doi.org/10.1016/j.buildenv.2024.112501>